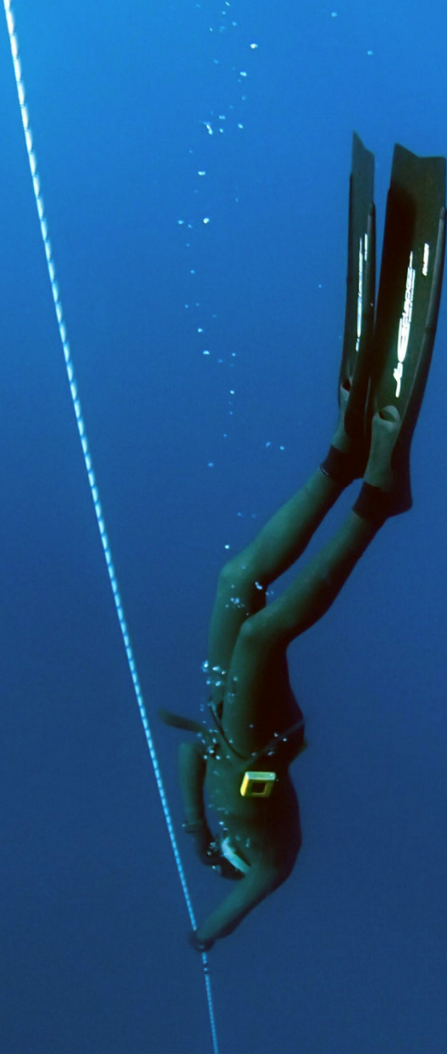




Deep JavaScript

Theory and techniques

Dr. Axel Rauschmayer

Deep JavaScript

Dr. Axel Rauschmayer

2020

Copyright © 2020 by Dr. Axel Rauschmayer

Cover photo by [Jakob Boman](#) on [Unsplash](#)

All rights reserved. This book or any portion thereof may not be reproduced or used in any manner whatsoever without the express written permission of the publisher except for the use of brief quotations in a book review or scholarly journal.

exploringjs.com

Contents

I	Frontmatter	7
1	About this book	9
1.1	Where is the homepage of this book?	9
1.2	What is in this book?	9
1.3	What do I get for my money?	10
1.4	How can I preview the content?	10
1.5	How do I report errors?	10
1.6	Tips for reading	10
1.7	Notations and conventions	10
1.8	Acknowledgements	11
II	Types, values, variables	13
2	Type coercion in JavaScript	15
2.1	What is type coercion?	15
2.2	Operations that help implement coercion in the ECMAScript specification	18
2.3	Intermission: expressing specification algorithms in JavaScript	20
2.4	Example coercion algorithms	21
2.5	Operations that coerce	29
2.6	Glossary: terms related to type conversion	32
3	The destructuring algorithm	33
3.1	Preparing for the pattern matching algorithm	33
3.2	The pattern matching algorithm	35
3.3	Empty object patterns and Array patterns	37
3.4	Applying the algorithm	38
4	A detailed look at global variables	41
4.1	Scopes	41
4.2	Lexical environments	42
4.3	The global object	42
4.4	In browsers, <code>globalThis</code> does not point directly to the global object	42
4.5	The global environment	43
4.6	Conclusion: Why does JavaScript have both normal global variables and the global object?	45
4.7	Further reading and sources of this chapter	47

5	% is a remainder operator, not a modulo operator (bonus)	49
5.1	Remainder operator <code>rem</code> vs. modulo operator <code>mod</code>	49
5.2	An intuitive understanding of the remainder operation	50
5.3	An intuitive understanding of the modulo operation	50
5.4	Similarities and differences between <code>rem</code> and <code>mod</code>	51
5.5	The equations behind remainder and modulo	51
5.6	Where are <code>rem</code> and <code>mod</code> used in programming languages?	53
5.7	Further reading and sources of this chapter	54
III	Working with data	55
6	Copying objects and Arrays	57
6.1	Shallow copying vs. deep copying	57
6.2	Shallow copying in JavaScript	58
6.3	Deep copying in JavaScript	62
6.4	Further reading	65
7	Updating data destructively and non-destructively	67
7.1	Examples: updating an object destructively and non-destructively	67
7.2	Examples: updating an Array destructively and non-destructively	68
7.3	Manual deep updating	69
7.4	Implementing generic deep updating	69
8	The problems of shared mutable state and how to avoid them	71
8.1	What is shared mutable state and why is it problematic?	71
8.2	Avoiding sharing by copying data	73
8.3	Avoiding mutations by updating non-destructively	75
8.4	Preventing mutations by making data immutable	76
8.5	Libraries for avoiding shared mutable state	76
IV	OOP: object property attributes	79
9	Property attributes: an introduction	81
9.1	The structure of objects	82
9.2	Property descriptors	84
9.3	Retrieving descriptors for properties	85
9.4	Defining properties via descriptors	86
9.5	<code>Object.create()</code> : Creating objects via descriptors	88
9.6	Use cases for <code>Object.getPrototypeOf()</code>	89
9.7	Omitting descriptor properties	91
9.8	What property attributes do built-in constructs use?	92
9.9	API: property descriptors	95
9.10	Further reading	97
10	Protecting objects from being changed	99
10.1	Levels of protection: preventing extensions, sealing, freezing	99
10.2	Preventing extensions of objects	100
10.3	Sealing objects	101

10.4 Freezing objects	102
10.5 Further reading	104
11 Properties: assignment vs. definition	105
11.1 Assignment vs. definition	106
11.2 Assignment and definition in theory (optional)	106
11.3 Definition and assignment in practice	110
11.4 Which language constructs use definition, which assignment?	114
11.5 Further reading and sources of this chapter	115
12 Enumerability of properties	117
12.1 How enumerability affects property-iterating constructs	117
12.2 The enumerability of pre-defined and created properties	122
12.3 Use cases for enumerability	122
12.4 Conclusion	128
V OOP: techniques	129
13 Techniques for instantiating classes	131
13.1 The problem: initializing a property asynchronously	131
13.2 Solution: Promise-based constructor	132
13.3 Solution: static factory method	133
13.4 Subclassing a Promise-based constructor (optional)	137
13.5 Conclusion	138
13.6 Further reading	138
14 Copying instances of classes: <code>.clone()</code> vs. copy constructors	141
14.1 <code>.clone()</code> methods	141
14.2 Static factory methods	142
14.3 Acknowledgements	143
15 Immutable wrappers for collections	145
15.1 Wrapping objects	145
15.2 An immutable wrapper for Maps	146
15.3 An immutable wrapper for Arrays	147
VI Regular expressions	149
16 Regular expressions: lookahead assertions by example	151
16.1 Cheat sheet: lookahead assertions	151
16.2 Warnings for this chapter	152
16.3 Example: Specifying what comes before or after a match (positive lookahead)	152
16.4 Example: Specifying what does not come before or after a match (negative lookahead)	153
16.5 Interlude: pointing lookahead assertions inward	154
16.6 Example: match strings not starting with 'abc'	154
16.7 Example: match substrings that do not contain '.mjs'	155

16.8 Example: skipping lines with comments	155
16.9 Example: smart quotes	156
16.10 Acknowledgements	157
16.11 Further reading	157
17 Composing regular expressions via re-template-tag (bonus)	159
17.1 The basics	159
17.2 A tour of the features	160
17.3 Why is this useful?	160
17.4 re and named capture groups	160
VII Miscellaneous topics	163
18 Exploring Promises by implementing them	165
18.1 Refresher: states of Promises	166
18.2 Version 1: Stand-alone Promise	166
18.3 Version 2: Chaining .then() calls	169
18.4 Convenience method .catch()	170
18.5 Omitting reactions	170
18.6 The implementation	171
18.7 Version 3: Flattening Promises returned from .then() callbacks	172
18.8 Version 4: Exceptions thrown in reaction callbacks	175
18.9 Version 5: Revealing constructor pattern	177
19 Metaprogramming with Proxies (early access)	179
19.1 Overview	180
19.2 Programming versus metaprogramming	181
19.3 Proxies explained	182
19.4 Use cases for Proxies	190
19.5 The design of the Proxy API	200
19.6 FAQ: Proxies	209
19.7 Reference: the Proxy API	209
19.8 Conclusion	215
19.9 Further reading	215
20 The property .name of functions (bonus)	217
20.1 Names of functions	218
20.2 Constructs that provide names for functions	219
20.3 Things to look out for with names of functions	223
20.4 Changing the names of functions	223
20.5 The function property .name in the ECMAScript specification	224

Part I

Frontmatter

Chapter 1

About this book

Contents

1.1	Where is the homepage of this book?	9
1.2	What is in this book?	9
1.3	What do I get for my money?	10
1.4	How can I preview the content?	10
1.5	How do I report errors?	10
1.6	Tips for reading	10
1.7	Notations and conventions	10
1.7.1	What is a type signature? Why am I seeing static types in this book?	10
1.7.2	What do the notes with icons mean?	11
1.8	Acknowledgements	11

1.1 Where is the homepage of this book?

The homepage of “Deep JavaScript” is exploringjs.com/deep-js/

1.2 What is in this book?

This book dives deeply into JavaScript:

- It teaches practical techniques for using the language better.
- It teaches how the language works and why. What it teaches is firmly grounded in the ECMAScript specification (which the book explains and refers to).
- It covers only the language (ignoring platform-specific features such as browser APIs) but not exhaustively. Instead, it focuses on a selection of important topics.

1.3 What do I get for my money?

If you buy this book, you get:

- The current content in four DRM-free versions:
 - PDF file
 - ZIP archive with ad-free HTML
 - EPUB file
 - MOBI file
- Any future content that is added to this edition. How much I can add depends on the sales of this book.

The current price is introductory. It will increase as more content is added.

1.4 How can I preview the content?

[On the homepage of this book](#), there are extensive previews for all versions of this book.

1.5 How do I report errors?

- The HTML version of this book has a link to comments at the end of each chapter.
- They jump to GitHub issues, which [you can also access directly](#).

1.6 Tips for reading

- You can read the chapters in any order. Each one is self-contained but occasionally, there are references to other chapters with further information.
- The headings of some sections are marked with “(optional)” meaning that they are non-essential. You will still understand the remainders of their chapters if you skip them.

1.7 Notations and conventions

1.7.1 What is a type signature? Why am I seeing static types in this book?

For example, you may see:

```
Number.isFinite(num: number): boolean
```

That is called the *type signature* of `Number.isFinite()`. This notation, especially the static types `number` of `num` and `boolean` of the result, are not real JavaScript. The notation is borrowed from the compile-to-JavaScript language TypeScript (which is mostly just JavaScript plus static typing).

Why is this notation being used? It helps give you a quick idea of how a function works. The notation is explained in detail in [a 2ality blog post](#), but is usually relatively intuitive.

1.7.2 What do the notes with icons mean?

**Reading instructions**

Explains how to best read the content.

**External content**

Points to additional, external, content.

**Tip**

Gives a tip related to the current content.

**Question**

Asks and answers a question pertinent to the current content (think FAQ).

**Warning**

Warns about pitfalls, etc.

**Details**

Provides additional details, complementing the current content. It is similar to a footnote.

1.8 Acknowledgements

- Thanks to Allen Wirfs-Brock for his advice via Twitter and blog post comments. It helped make this book better.
- More people who contributed are acknowledged in the chapters.

Part II

Types, values, variables

Chapter 2

Type coercion in JavaScript

Contents

2.1	What is type coercion?	15
2.1.1	Dealing with type coercion	17
2.2	Operations that help implement coercion in the ECMAScript specification	18
2.2.1	Converting to primitive types and objects	18
2.2.2	Converting to numeric types	18
2.2.3	Converting to property keys	19
2.2.4	Converting to Array indices	19
2.2.5	Converting to Typed Array elements	20
2.3	Intermission: expressing specification algorithms in JavaScript	20
2.4	Example coercion algorithms	21
2.4.1	ToPrimitive()	21
2.4.2	ToString() and related operations	24
2.4.3	ToPropertyKey()	28
2.4.4	ToNumeric() and related operations	28
2.5	Operations that coerce	29
2.5.1	Addition operator (+)	29
2.5.2	Abstract Equality Comparison (==)	30
2.6	Glossary: terms related to type conversion	32

In this chapter, we examine the role of *type coercion* in JavaScript. We will go relatively deeply into this subject and, e.g., look into how the ECMAScript specification handles coercion.

2.1 What is type coercion?

Each operation (function, operator, etc.) expects its parameters to have certain types. If a value doesn't have the right type for a parameter, three common options for, e.g., a function are:

1. The function can throw an exception:

```
function multiply(x, y) {
  if (typeof x !== 'number' || typeof y !== 'number') {
    throw new TypeError();
  }
  // ...
}
```

2. The function can return an error value:

```
function multiply(x, y) {
  if (typeof x !== 'number' || typeof y !== 'number') {
    return NaN;
  }
  // ...
}
```

3. The function can convert its arguments to useful values:

```
function multiply(x, y) {
  if (typeof x !== 'number') {
    x = Number(x);
  }
  if (typeof y !== 'number') {
    y = Number(y);
  }
  // ...
}
```

In (3), the operation performs an implicit type conversion. That is called *type coercion*.

JavaScript initially didn't have exceptions, which is why it uses coercion and error values for most of its operations:

```
// Coercion
assert.equal(3 * true, 3);

// Error values
assert.equal(1 / 0, Infinity);
assert.equal(Number('xyz'), NaN);
```

However, there are also cases (especially when it comes to newer features) where it throws exceptions if an argument doesn't have the right type:

- Accessing properties of null or undefined:

```
> undefined.prop
TypeError: Cannot read property 'prop' of undefined
> null.prop
TypeError: Cannot read property 'prop' of null
> 'prop' in null
TypeError: Cannot use 'in' operator to search for 'prop' in null
```

- Using symbols:

```
> 6 / Symbol()
TypeError: Cannot convert a Symbol value to a number
```

- Mixing bigints and numbers:

```
> 6 / 3n
TypeError: Cannot mix BigInt and other types
```

- New-calling or function-calling values that don't support that operation:

```
> 123()
TypeError: 123 is not a function
> (class {}]()
TypeError: Class constructor cannot be invoked without 'new'

> new 123
TypeError: 123 is not a constructor
> new (() => {})
TypeError: (intermediate value) is not a constructor
```

- Changing read-only properties (only throws in strict mode):

```
> 'abc'.length = 1
TypeError: Cannot assign to read only property 'length'
> Object.freeze({prop:3}).prop = 1
TypeError: Cannot assign to read only property 'prop'
```

2.1.1 Dealing with type coercion

Two common ways of dealing with coercion are:

- A caller can explicitly convert values so that they have the right types. For example, in the following interaction, we want to multiply two numbers encoded as strings:

```
let x = '3';
let y = '2';
assert.equal(Number(x) * Number(y), 6);
```

- A caller can let the operation make the conversion for them:

```
let x = '3';
let y = '2';
assert.equal(x * y, 6);
```

I usually prefer the former, because it clarifies my intention: I expect `x` and `y` not to be numbers, but want to multiply two numbers.

2.2 Operations that help implement coercion in the ECMAScript specification

The following sections describe the most important internal functions used by the ECMAScript specification to convert actual parameters to expected types.

For example, in TypeScript, we would write:

```
function isNaN(number: number) {
  // ...
}
```

In the specification, this looks [as follows](#) (translated to JavaScript, so that it is easier to understand):

```
function isNaN(number) {
  let num = ToNumber(number);
  // ...
}
```

2.2.1 Converting to primitive types and objects

Whenever primitive types or objects are expected, the following conversion functions are used:

- `ToBoolean()`
- `ToNumber()`
- `ToBigInt()`
- `ToString()`
- `ToObject()`

These internal functions have analogs in JavaScript that are very similar:

```
> Boolean(0)
false
> Boolean(1)
true

> Number('123')
123
```

After the introduction of bigints, which exists alongside numbers, the specification often uses `ToNumeric()` where it previously used `ToNumber()`. Read on for more information.

2.2.2 Converting to numeric types

At the moment, JavaScript has [two built-in numeric types](#): `number` and `bigint`.

- `ToNumeric()` returns a numeric value `num`. Its callers usually invoke a method `mthd` of the specification type of `num`:

```
Type(num)::mthd(...)
```

Among others, the following operations use `ToNumeric`:

- Prefix and postfix `++` operator
- `*` operator
- `ToInteger(x)` is used whenever a number without a fraction is expected. The range of the result is often restricted further afterwards.
 - It calls `ToNumber(x)` and removes the fraction (similar to `Math.trunc()`).
 - Operations that use `ToInteger()`:
 - * `Number.prototype.toString(radix?)`
 - * `String.prototype.codePointAt(pos)`
 - * `Array.prototype.slice(start, end)`
 - * Etc.
- `ToInt32()`, `ToUint32()` coerce numbers to 32-bit integers and are used by bitwise operators (see [tbl. 2.1](#)).
 - `ToInt32()`: signed, range $[-2^{31}, 2^{31}-1]$ (limits are included)
 - `ToUint32()`: unsigned (hence the `U`), range $[0, 2^{32}-1]$ (limits are included)

Table 2.1: Coercion of the operands of bitwise number operators (BigInt operators don't limit the number of bits).

Operator	Left operand	Right operand	result type
<code><<</code>	<code>ToInt32()</code>	<code>ToUint32()</code>	<code>Int32</code>
signed <code>>></code>	<code>ToInt32()</code>	<code>ToUint32()</code>	<code>Int32</code>
unsigned <code>>>></code>	<code>ToInt32()</code>	<code>ToUint32()</code>	<code>Uint32</code>
<code>&, ^, </code>	<code>ToInt32()</code>	<code>ToUint32()</code>	<code>Int32</code>
<code>~</code>	—	<code>ToInt32()</code>	<code>Int32</code>

2.2.3 Converting to property keys

`ToPropertyKey()` returns a string or a symbol and is used by:

- The bracket operator `[]`
- Computed property keys in object literals
- The left-hand side of the `in` operator
- `Object.defineProperty(_, P, _)`
- `Object.fromEntries()`
- `Object.getOwnPropertyDescriptor()`
- `Object.prototype.hasOwnProperty()`
- `Object.prototype.propertyIsEnumerable()`
- Several methods of `Reflect`

2.2.4 Converting to Array indices

- `ToLength()` is used (directly) mainly for string indices.
 - Helper function for `ToIndex()`
 - Range of result `l`: $0 \leq l \leq 2^{53}-1$

- `ToIndex()` is used for Typed Array indices.
 - Main difference with `ToLength()`: throws an exception if argument is out of range.
 - Range of result i : $0 \leq i \leq 2^{53}-1$
- `ToUint32()` is used for Array indices.
 - Range of result i : $0 \leq i < 2^{32}-1$ (the upper limit is excluded, to leave room for the `.length`)

2.2.5 Converting to Typed Array elements

When we set the value of a Typed Array element, one of the following conversion functions is used:

- `ToInt8()`
- `ToUint8()`
- `ToUint8Clamp()`
- `ToInt16()`
- `ToUint16()`
- `ToInt32()`
- `ToUint32()`
- `ToBigInt64()`
- `ToBigUint64()`

2.3 Intermission: expressing specification algorithms in JavaScript

In the remainder of this chapter, we'll encounter several specification algorithms, but "implemented" as JavaScript. The following list shows how some frequently used patterns are translated from specification to JavaScript:

- Spec: If `Type(value)` is String
 JavaScript: `if (TypeOf(value) === 'string')`
 (very loose translation; `TypeOf()` is defined below)
- Spec: If `IsCallable(method)` is true
 JavaScript: `if (IsCallable(method))`
 (`IsCallable()` is defined below)
- Spec: Let `numValue` be `ToNumber(value)`
 JavaScript: `let numValue = Number(value)`
- Spec: Let `isArray` be `isArray(O)`
 JavaScript: `let isArray = Array.isArray(O)`
- Spec: If `O` has a `[[NumberData]]` internal slot
 JavaScript: `if ('__NumberData__' in O)`
- Spec: Let `tag` be `Get(O, @@toStringTag)`
 JavaScript: `let tag = O[Symbol.toStringTag]`

- Spec: Return the string-concatenation of “[object”, tag, and ”].
JavaScript: return '[object ' + tag + ']';

let (and not const) is used to match the language of the specification.

Some things are omitted – for example, the [ReturnIfAbrupt shorthands](#) ? and !.

```
/**
 * An improved version of typeof
 */
function TypeOf(value) {
  const result = typeof value;
  switch (result) {
    case 'function':
      return 'object';
    case 'object':
      if (value === null) {
        return 'null';
      } else {
        return 'object';
      }
    default:
      return result;
  }
}

function IsCallable(x) {
  return typeof x === 'function';
}
```

2.4 Example coercion algorithms

2.4.1 ToPrimitive()

The operation `ToPrimitive()` is an intermediate step for many coercion algorithms (some of which we’ll see later in this chapter). It converts an arbitrary values to primitive values.

`ToPrimitive()` is used often in the spec because most operators can only work with primitive values. For example, we can use the addition operator (+) to add numbers and to concatenate strings, but we can’t use it to concatenate Arrays.

This is what the JavaScript version of `ToPrimitive()` looks like:

```
/**
 * @param hint Which type is preferred for the result:
 *             string, number, or don't care?
 */
function ToPrimitive(input: any,
  hint: 'string' | 'number' | 'default' = 'default') {
  if (TypeOf(input) === 'object') {
```

```

    let exoticToPrim = input[Symbol.toPrimitive]; // (A)
    if (exoticToPrim !== undefined) {
      let result = exoticToPrim.call(input, hint);
      if (TypeOf(result) !== 'object') {
        return result;
      }
      throw new TypeError();
    }
    if (hint === 'default') {
      hint = 'number';
    }
    return OrdinaryToPrimitive(input, hint);
  } else {
    // input is already primitive
    return input;
  }
}

```

`ToPrimitive()` lets objects override the conversion to primitive via `Symbol.toPrimitive` (line A). If an object doesn't do that, it is passed on to `OrdinaryToPrimitive()`:

```

function OrdinaryToPrimitive(O: object, hint: 'string' | 'number') {
  let methodNames;
  if (hint === 'string') {
    methodNames = ['toString', 'valueOf'];
  } else {
    methodNames = ['valueOf', 'toString'];
  }
  for (let name of methodNames) {
    let method = O[name];
    if (IsCallable(method)) {
      let result = method.call(O);
      if (TypeOf(result) !== 'object') {
        return result;
      }
    }
  }
  throw new TypeError();
}

```

2.4.1.1 Which hints do callers of `ToPrimitive()` use?

The parameter `hint` can have one of three values:

- 'number' means: if possible, input should be converted to a number.
- 'string' means: if possible, input should be converted to a string.
- 'default' means: there is no preference for either numbers or strings.

These are a few examples of how various operations use `ToPrimitive()`:

- `hint === 'number'`. The following operations prefer numbers:

- `ToNumeric()`
- `ToNumber()`
- `ToBigInt(), BigInt()`
- Abstract Relational Comparison (`<`)
- `hint === 'string'`. The following operations prefer strings:
 - `ToString()`
 - `ToPropertyKey()`
- `hint === 'default'`. The following operations are neutral w.r.t. the type of the returned primitive value:
 - Abstract Equality Comparison (`==`)
 - Addition Operator (`+`)
 - `new Date(value)` (`value` can be either a number or a string)

As we have seen, the default behavior is for `'default'` being handled as if it were `'number'`. Only instances of `Symbol` and `Date` override this behavior (shown later).

2.4.1.2 Which methods are called to convert objects to Primitives?

If the conversion to primitive isn't overridden via `Symbol.toPrimitive`, `OrdinaryToPrimitive()` calls either or both of the following two methods:

- `'toString'` is called first if `hint` indicates that we'd like the primitive value to be a string.
- `'valueOf'` is called first if `hint` indicates that we'd like the primitive value to be a number.

The following code demonstrates how that works:

```
const obj = {
  toString() { return 'a' },
  valueOf() { return 1 },
};

// String() prefers strings:
assert.equal(String(obj), 'a');

// Number() prefers numbers:
assert.equal(Number(obj), 1);
```

A method with the property key `Symbol.toPrimitive` overrides the normal conversion to primitive. That is only done twice in the standard library:

- `Symbol.prototype[Symbol.toPrimitive](hint)`
 - If the receiver is an instance of `Symbol`, this method always returns the wrapped symbol.
 - The rationale is that instances of `Symbol` have a `.toString()` method that returns strings. But even if `hint` is `'string'`, `.toString()` should not be called so that we don't accidentally convert instances of `Symbol` to strings (which are a completely different kind of property key).
- `Date.prototype[Symbol.toPrimitive](hint)`
 - Explained in more detail next.

2.4.1.3 Date.prototype[Symbol.toPrimitive]()

This is how Dates handle being converted to primitive values:

```
Date.prototype[Symbol.toPrimitive] = function (
  hint: 'default' | 'string' | 'number') {
  let 0 = this;
  if (TypeOf(0) !== 'object') {
    throw new TypeError();
  }
  let tryFirst;
  if (hint === 'string' || hint === 'default') {
    tryFirst = 'string';
  } else if (hint === 'number') {
    tryFirst = 'number';
  } else {
    throw new TypeError();
  }
  return OrdinaryToPrimitive(0, tryFirst);
};
```

The only difference with the default algorithm is that 'default' becomes 'string' (and not 'number'). This can be observed if we use operations that set hint to 'default':

- The `==` operator coerces objects to primitives (with a default hint) if the other operand is a primitive value other than undefined, null, and boolean. In the following interaction, we can see that the result of coercing the date is a string:

```
const d = new Date('2222-03-27');
assert.equal(
  d == 'Wed Mar 27 2222 01:00:00 GMT+0100'
    + ' (Central European Standard Time)',
  true);
```

- The `+` operator coerces both operands to primitives (with a default hint). If one of the results is a string, it performs string concatenation (otherwise it performs numeric addition). In the following interaction, we can see that the result of coercing the date is a string because the operator returns a string.

```
const d = new Date('2222-03-27');
assert.equal(
  123 + d,
  '123Wed Mar 27 2222 01:00:00 GMT+0100'
    + ' (Central European Standard Time)');
```

2.4.2 ToString() and related operations

This is the JavaScript version of ToString():

```
function ToString(argument) {
  if (argument === undefined) {
    return 'undefined';
  }
```

```

    } else if (argument === null) {
      return 'null';
    } else if (argument === true) {
      return 'true';
    } else if (argument === false) {
      return 'false';
    } else if (TypeOf(argument) === 'number') {
      return Number.toString(argument);
    } else if (TypeOf(argument) === 'string') {
      return argument;
    } else if (TypeOf(argument) === 'symbol') {
      throw new TypeError();
    } else if (TypeOf(argument) === 'bigint') {
      return BigInt.toString(argument);
    } else {
      // argument is an object
      let primValue = ToPrimitive(argument, 'string'); // (A)
      return ToString(primValue);
    }
  }
}

```

Note how this function uses `ToPrimitive()` as an intermediate step for objects, before converting the primitive result to a string (line A).

`ToString()` deviates in an interesting way from how `String()` works: If argument is a symbol, the former throws a `TypeError` while the latter doesn't. Why is that? The default for symbols is that converting them to strings throws exceptions:

```

> const sym = Symbol('sym');

> ''+sym
TypeError: Cannot convert a Symbol value to a string
> `${sym}`
TypeError: Cannot convert a Symbol value to a string

```

That default is overridden in `String()` and `Symbol.prototype.toString()` (both are described in the next subsections):

```

> String(sym)
'Symbol(sym)'
> sym.toString()
'Symbol(sym)'

```

2.4.2.1 String()

```

function String(value) {
  let s;
  if (value === undefined) {
    s = '';
  } else {
    if (new.target === undefined && TypeOf(value) === 'symbol') {

```

```

    // This function was function-called and value is a symbol
    return SymbolDescriptiveString(value);
  }
  s = ToString(value);
}
if (new.target === undefined) {
  // This function was function-called
  return s;
}
// This function was new-called
return StringCreate(s, new.target.prototype); // simplified!
}

```

`String()` works differently, depending on whether it is invoked via a function call or via `new`. It uses `new.target` to distinguish the two.

These are the helper functions `StringCreate()` and `SymbolDescriptiveString()`:

```

/**
 * Creates a String instance that wraps `value`
 * and has the given prototype.
 */
function StringCreate(value, prototype) {
  // ...
}

function SymbolDescriptiveString(sym) {
  assert.equal(TypeOf(sym), 'symbol');
  let desc = sym.description;
  if (desc === undefined) {
    desc = '';
  }
  assert.equal(TypeOf(desc), 'string');
  return 'Symbol('+desc+')';
}

```

2.4.2.2 `Symbol.prototype.toString()`

In addition to `String()`, we can also use method `.toString()` to convert a symbol to a string. Its specification looks as follows.

```

Symbol.prototype.toString = function () {
  let sym = thisSymbolValue(this);
  return SymbolDescriptiveString(sym);
};
function thisSymbolValue(value) {
  if (TypeOf(value) === 'symbol') {
    return value;
  }
  if (TypeOf(value) === 'object' && '__SymbolData__' in value) {

```

```

    let s = value.__SymbolData__;
    assert.equal(TypeOf(s), 'symbol');
    return s;
  }
}

```

2.4.2.3 Object.prototype.toString

The default specification for `.toString()` looks as follows:

```

Object.prototype.toString = function () {
  if (this === undefined) {
    return '[object Undefined]';
  }
  if (this === null) {
    return '[object Null]';
  }
  let O = ToObject(this);
  let isArray = Array.isArray(O);
  let builtinTag;
  if (isArray) {
    builtinTag = 'Array';
  } else if ('__ParameterMap__' in O) {
    builtinTag = 'Arguments';
  } else if ('__Call__' in O) {
    builtinTag = 'Function';
  } else if ('__ErrorData__' in O) {
    builtinTag = 'Error';
  } else if ('__BooleanData__' in O) {
    builtinTag = 'Boolean';
  } else if ('__NumberData__' in O) {
    builtinTag = 'Number';
  } else if ('__StringData__' in O) {
    builtinTag = 'String';
  } else if ('__DateValue__' in O) {
    builtinTag = 'Date';
  } else if ('__RegExpMatcher__' in O) {
    builtinTag = 'RegExp';
  } else {
    builtinTag = 'Object';
  }
  let tag = O[Symbol.toStringTag];
  if (TypeOf(tag) !== 'string') {
    tag = builtinTag;
  }
  return '[object ' + tag + ']';
};

```

This operation is used if we convert plain objects to strings:

```
> String({})
'[object Object]'
```

By default, it is also used if we convert instances of classes to strings:

```
class MyClass {}
assert.equal(
  String(new MyClass()), '[object Object]');
```

Normally, we would override `.toString()` in order to configure the string representation of `MyClass`, but we can also change what comes after “object” inside the string with the square brackets:

```
class MyClass {}
MyClass.prototype[Symbol.toStringTag] = 'Custom!';
assert.equal(
  String(new MyClass()), '[object Custom!]');
```

It is interesting to compare the overriding versions of `.toString()` with the original version in `Object.prototype`:

```
> ['a', 'b'].toString()
'a,b'
> Object.prototype.toString.call(['a', 'b'])
'[object Array]'

> /^abc$/.toString()
'/^abc$/'
> Object.prototype.toString.call(/^abc$/)
'[object RegExp]'
```

2.4.3 ToPropertyKey()

`ToPropertyKey()` is used by, among others, the bracket operator. This is how it works:

```
function ToPropertyKey(argument) {
  let key = ToPrimitive(argument, 'string'); // (A)
  if (TypeOf(key) === 'symbol') {
    return key;
  }
  return ToString(key);
}
```

Once again, objects are converted to primitives before working with primitives.

2.4.4 ToNumeric() and related operations

`ToNumeric()` is used by, among others, by the multiplication operator (`*`). This is how it works:

```
function ToNumeric(value) {
  let primValue = ToPrimitive(value, 'number');
  if (TypeOf(primValue) === 'bigint') {
```

```

    return primValue;
  }
  return ToNumber(primValue);
}

```

2.4.4.1 ToNumber()

ToNumber() works as follows:

```

function ToNumber(argument) {
  if (argument === undefined) {
    return NaN;
  } else if (argument === null) {
    return +0;
  } else if (argument === true) {
    return 1;
  } else if (argument === false) {
    return +0;
  } else if (TypeOf(argument) === 'number') {
    return argument;
  } else if (TypeOf(argument) === 'string') {
    return parseTheString(argument); // not shown here
  } else if (TypeOf(argument) === 'symbol') {
    throw new TypeError();
  } else if (TypeOf(argument) === 'bigint') {
    throw new TypeError();
  } else {
    // argument is an object
    let primValue = ToPrimitive(argument, 'number');
    return ToNumber(primValue);
  }
}

```

The structure of ToNumber() is similar to the structure of ToString().

2.5 Operations that coerce

2.5.1 Addition operator (+)

This is how JavaScript's addition operator is specified:

```

function Addition(leftHandSide, rightHandSide) {
  let lprim = ToPrimitive(leftHandSide);
  let rprim = ToPrimitive(rightHandSide);
  if (TypeOf(lprim) === 'string' || TypeOf(rprim) === 'string') { // (A)
    return ToString(lprim) + ToString(rprim);
  }
  let lnum = ToNumeric(lprim);
  let rnum = ToNumeric(rprim);

```

```

    if (TypeOf(lnum) !== TypeOf(rnum)) {
        throw new TypeError();
    }
    let T = Type(lnum);
    return T.add(lnum, rnum); // (B)
}

```

Steps of this algorithm:

- Both operands are converted to primitive values.
- If one of the results is a string, both are converted to strings and concatenated (line A).
- Otherwise, both operands are converted to numeric values and added (line B). `Type()` returns the ECMAScript specification type of `lnum`. `.add()` is a method of [numeric types](#).

2.5.2 Abstract Equality Comparison (==)

```

/** Loose equality (==) */
function abstractEqualityComparison(x, y) {
    if (TypeOf(x) === TypeOf(y)) {
        // Use strict equality (===)
        return strictEqualityComparison(x, y);
    }

    // Comparing null with undefined
    if (x === null && y === undefined) {
        return true;
    }
    if (x === undefined && y === null) {
        return true;
    }

    // Comparing a number and a string
    if (TypeOf(x) === 'number' && TypeOf(y) === 'string') {
        return abstractEqualityComparison(x, Number(y));
    }
    if (TypeOf(x) === 'string' && TypeOf(y) === 'number') {
        return abstractEqualityComparison(Number(x), y);
    }

    // Comparing a bigint and a string
    if (TypeOf(x) === 'bigint' && TypeOf(y) === 'string') {
        let n = StringToBigInt(y);
        if (Number.isNaN(n)) {
            return false;
        }
        return abstractEqualityComparison(x, n);
    }
}

```

```

    if (TypeOf(x) === 'string' && TypeOf(y) === 'bigint') {
      return abstractEqualityComparison(y, x);
    }

    // Comparing a boolean with a non-boolean
    if (TypeOf(x) === 'boolean') {
      return abstractEqualityComparison(Number(x), y);
    }
    if (TypeOf(y) === 'boolean') {
      return abstractEqualityComparison(x, Number(y));
    }

    // Comparing an object with a primitive
    // (other than undefined, null, a boolean)
    if (['string', 'number', 'bigint', 'symbol'].includes(TypeOf(x))
      && TypeOf(y) === 'object') {
      return abstractEqualityComparison(x, ToPrimitive(y));
    }
    if (TypeOf(x) === 'object'
      && ['string', 'number', 'bigint', 'symbol'].includes(TypeOf(y))) {
      return abstractEqualityComparison(ToPrimitive(x), y);
    }

    // Comparing a bigint with a number
    if ((TypeOf(x) === 'bigint' && TypeOf(y) === 'number')
      || (TypeOf(x) === 'number' && TypeOf(y) === 'bigint')) {
      if ([NaN, +Infinity, -Infinity].includes(x)
        || [NaN, +Infinity, -Infinity].includes(y)) {
        return false;
      }
      if (isSameMathematicalValue(x, y)) {
        return true;
      } else {
        return false;
      }
    }

    return false;
  }

```

The following operations are not shown here:

- `strictEqualityComparison()`
- `StringToBigInt()`
- `isSameMathematicalValue()`

2.6 Glossary: terms related to type conversion

Now that we have taken a closer look at how JavaScript's type coercion works, let's conclude with a brief glossary of terms related to type conversion:

- In *type conversion*, we want the output value to have a given type. If the input value already has that type, it is simply returned unchanged. Otherwise, it is converted to a value that has the desired type.
- *Explicit type conversion* means that the programmer uses an operation (a function, an operator, etc.) to trigger a type conversion. Explicit conversions can be:
 - *Checked*: If a value can't be converted, an exception is thrown.
 - *Unchecked*: If a value can't be converted, an error value is returned.
- What *type casting* is, depends on the programming language. For example, in Java, it is explicit checked type conversion.
- *Type coercion* is implicit type conversion: An operation automatically converts its arguments to the types it needs. Can be checked or unchecked or something in-between.

[Source: [Wikipedia](#)]

Chapter 3

The destructuring algorithm

Contents

3.1	Preparing for the pattern matching algorithm	33
3.1.1	Using declarative rules for specifying the matching algorithm	34
3.1.2	Evaluating expressions based on the declarative rules	35
3.2	The pattern matching algorithm	35
3.2.1	Patterns	35
3.2.2	Rules for variable	35
3.2.3	Rules for object patterns	35
3.2.4	Rules for Array patterns	36
3.3	Empty object patterns and Array patterns	37
3.4	Applying the algorithm	38
3.4.1	Background: passing parameters via matching	38
3.4.2	Using move2()	39
3.4.3	Using move1()	39
3.4.4	Conclusion: Default values are a feature of pattern parts	40

In this chapter, we look at destructuring from a different angle: as a recursive pattern matching algorithm.

The algorithm will give us a better understanding of default values. That will be useful at the end, where we'll try to figure out how the following two functions differ:

```
function move({x=0, y=0} = {}) { ... }
function move({x, y} = { x: 0, y: 0 }) { ... }
```

3.1 Preparing for the pattern matching algorithm

A destructuring assignment looks like this:

```
«pattern» = «value»
```

We want to use `pattern` to extract data from `value`.

We will now look at an algorithm for performing this kind of assignment. This algorithm is known in functional programming as *pattern matching* (short: *matching*). It specifies the operator `←` (“match against”) that matches a `pattern` against a `value` and assigns to variables while doing so:

```
«pattern» ← «value»
```

We will only explore destructuring assignment, but destructuring variable declarations and destructuring parameter definitions work similarly. We won’t go into advanced features, either: Computed property keys, property value shorthands, and object properties and array elements as assignment targets, are beyond the scope of this chapter.

The specification for the match operator consists of declarative rules that descend into the structures of both operands. The declarative notation may take some getting used to, but it makes the specification more concise.

3.1.1 Using declarative rules for specifying the matching algorithm

The declarative rules used in this chapter operate on input and produce the result of the algorithm via side effects. This is one such rule (which we’ll see again later):

- (2c) {key: «pattern», «properties»} ← obj

```
«pattern» ← obj.key
{«properties»} ← obj
```

This rule has the following parts:

- (2c) is the *number* of the rule. The number is used to refer to the rule.
- The *head* (first line) describes what the input must look like so that this rule can be applied.
- The *body* (remaining lines) describes what happens if the rule is applied.

In rule (2c), the head means that this rule can be applied if there is an object pattern with at least one property (whose key is `key`) and zero or more remaining properties. The effect of this rule is that execution continues with the property value pattern being matched against `obj.key` and the remaining properties being matched against `obj`.

Let’s consider one more rule from this chapter:

- (2e) {} ← obj (no properties left)

```
// We are finished
```

In rule (2e), the head means that this rule is executed if the empty object pattern {} is matched against a value `obj`. The body means that, in this case, we are done.

Together, rule (2c) and rule (2e) form a declarative loop that iterates over the properties of the pattern on the left-hand side of the arrow.

3.1.2 Evaluating expressions based on the declarative rules

The complete algorithm is specified via a sequence of declarative rules. Let's assume we want to evaluate the following matching expression:

```
{first: f, last: l} ← obj
```

To apply a sequence of rules, we go over them from top to bottom and execute the first applicable rule. If there is a matching expression in the body of that rule, the rules are applied again. And so on.

Sometimes the head includes a condition that also determines if a rule is applicable – for example:

- (3a) [«elements»] ← non_iterable
if (!isIterable(non_iterable))

 throw new **TypeError**();

3.2 The pattern matching algorithm

3.2.1 Patterns

A pattern is either:

- A variable: `x`
- An object pattern: {«properties»}
- An Array pattern: [«elements»]

The next three sections specify rules for handling these three cases in matching expressions.

3.2.2 Rules for variable

- (1) `x ← value` (including undefined and null)

 `x = value`

3.2.3 Rules for object patterns

- (2a) {«properties»} ← undefined (illegal value)

 throw new **TypeError**();
- (2b) {«properties»} ← null (illegal value)

 throw new **TypeError**();
- (2c) {key: «pattern», «properties»} ← obj

 «pattern» ← obj.**key**
 {«properties»} ← obj
- (2d) {key: «pattern» = default_value, «properties»} ← obj

```

const tmp = obj.key;
if (tmp !== undefined) {
  «pattern» ← tmp
} else {
  «pattern» ← default_value
}
{«properties»} ← obj

```

- (2e) {} ← obj (no properties left)

```
// We are finished
```

Rules 2a and 2b deal with illegal values. Rules 2c–2e loop over the properties of the pattern. In rule 2d, we can see that a default value provides an alternative to match against if there is no matching property in obj.

3.2.4 Rules for Array patterns

Array pattern and iterable. The algorithm for Array destructuring starts with an Array pattern and an iterable:

- (3a) [«elements»] ← non_iterable (illegal value)
 if (!isIterable(non_iterable))
 throw new **TypeError**();
- (3b) [«elements»] ← iterable
 if (isIterable(iterable))
 const iterator = iterable[**Symbol.iterator**]();
 «elements» ← iterator

Helper function:

```

function isIterable(value) {
  return (value !== null
    && typeof value === 'object'
    && typeof value[Symbol.iterator] === 'function');
}

```

Array elements and iterator. The algorithm continues with:

- The elements of the pattern (left-hand side of the arrow)
- The iterator that was obtained from the iterable (right-hand side of the arrow)

These are the rules:

- (3c) «pattern», «elements» ← iterator
 «pattern» ← getNext(iterator) // undefined after last item
 «elements» ← iterator
- (3d) «pattern» = default_value, «elements» ← iterator
 const tmp = getNext(iterator); // undefined after last item
 if (tmp !== undefined) {

```

    «pattern» ← tmp
  } else {
    «pattern» ← default_value
  }
  «elements» ← iterator

```

- (3e), «elements» ← iterator (hole, elision)


```

        getNext(iterator); // skip
        «elements» ← iterator
      
```
- (3f) ...«pattern» ← iterator (always last part!)


```

        const tmp = [];
        for (const elem of iterator) {
          tmp.push(elem);
        }
        «pattern» ← tmp
      
```
- (3g) ← iterator (no elements left)


```

        // We are finished
      
```

Helper function:

```

function getNext(iterator) {
  const {done,value} = iterator.next();
  return (done ? undefined : value);
}

```

An iterator being finished is similar to missing properties in objects.

3.3 Empty object patterns and Array patterns

Interesting consequence of the algorithm's rules: We can destructure with empty object patterns and empty Array patterns.

Given an empty object pattern {}: If the value to be destructured is neither undefined nor null, then nothing happens. Otherwise, a `TypeError` is thrown.

```

const {} = 123; // OK, neither undefined nor null
assert.throws(
  () => {
    const {} = null;
  },
  /^TypeError: Cannot destructure 'null' as it is null.$/)

```

Given an empty Array pattern []: If the value to be destructured is iterable, then nothing happens. Otherwise, a `TypeError` is thrown.

```

const [] = 'abc'; // OK, iterable
assert.throws(
  () => {
    const [] = 123; // not iterable
  }
)

```

```

},
/^TypeError: 123 is not iterable$/

```

In other words: Empty destructuring patterns force values to have certain characteristics, but have no other effects.

3.4 Applying the algorithm

In JavaScript, named parameters are simulated via objects: The caller uses an object literal and the callee uses destructuring. This simulation is explained in detail in “JavaScript for impatient programmers”. The following code shows an example: function `move1()` has two named parameters, `x` and `y`:

```

function move1({x=0, y=0} = {}) { // (A)
  return [x, y];
}
assert.deepEqual(
  move1({x: 3, y: 8}), [3, 8]);
assert.deepEqual(
  move1({x: 3}), [3, 0]);
assert.deepEqual(
  move1({}), [0, 0]);
assert.deepEqual(
  move1(), [0, 0]);

```

There are three default values in line A:

- The first two default values allow us to omit `x` and `y`.
- The third default value allows us to call `move1()` without parameters (as in the last line).

But why would we define the parameters as in the previous code snippet? Why not as follows?

```

function move2({x, y} = { x: 0, y: 0 }) {
  return [x, y];
}

```

To see why `move1()` is correct, we are going to use both functions in two examples. Before we do that, let’s see how the passing of parameters can be explained via matching.

3.4.1 Background: passing parameters via matching

For function calls, *formal parameters* (inside function definitions) are matched against *actual parameters* (inside function calls). As an example, take the following function definition and the following function call.

```

function func(a=0, b=0) { ... }
func(1, 2);

```

The parameters `a` and `b` are set up similarly to the following destructuring.

```
[a=0, b=0] ← [1, 2]
```

3.4.2 Using move2()

Let's examine how destructuring works for `move2()`.

Example 1. The function call `move2()` leads to this destructuring:

```
[{x, y} = { x: 0, y: 0 }] ← []
```

The single Array element on the left-hand side does not have a match on the right-hand side, which is why `{x, y}` is matched against the default value and not against data from the right-hand side (rules 3b, 3d):

```
{x, y} ← { x: 0, y: 0 }
```

The left-hand side contains *property value shorthands*. It is an abbreviation for:

```
{x: x, y: y} ← { x: 0, y: 0 }
```

This destructuring leads to the following two assignments (rules 2c, 1):

```
x = 0;
y = 0;
```

This is what we wanted. However, in the next example, we are not as lucky.

Example 2. Let's examine the function call `move2({z: 3})` which leads to the following destructuring:

```
[{x, y} = { x: 0, y: 0 }] ← [{z: 3}]
```

There is an Array element at index 0 on the right-hand side. Therefore, the default value is ignored and the next step is (rule 3d):

```
{x, y} ← { z: 3 }
```

That leads to both `x` and `y` being set to undefined, which is not what we want. The problem is that `{x, y}` is not matched against the default value, anymore, but against `{z: 3}`.

3.4.3 Using move1()

Let's try `move1()`.

Example 1: `move1()`

```
[{x=0, y=0} = {}] ← []
```

We don't have an Array element at index 0 on the right-hand side and use the default value (rule 3d):

```
{x=0, y=0} ← {}
```

The left-hand side contains property value shorthands, which means that this destructuring is equivalent to:

```
{x: x=0, y: y=0} ← {}
```

Neither property `x` nor property `y` have a match on the right-hand side. Therefore, the default values are used and the following destructurings are performed next (rule 2d):

```
x ← 0
y ← 0
```

That leads to the following assignments (rule 1):

```
x = 0
y = 0
```

Here, we get what we want. Let's see if our luck holds with the next example.

Example 2: `move1({z: 3})`

```
[{x=0, y=0} = {}] ← [{z: 3}]
```

The first element of the Array pattern has a match on the right-hand side and that match is used to continue destructuring (rule 3d):

```
{x=0, y=0} ← {z: 3}
```

Like in example 1, there are no properties `x` and `y` on the right-hand side and the default values are used:

```
x = 0
y = 0
```

It works as desired! This time, the pattern with `x` and `y` being matched against `{z:3}` is not a problem, because they have their own local default values.

3.4.4 Conclusion: Default values are a feature of pattern parts

The examples demonstrate that default values are a feature of pattern parts (object properties or Array elements). If a part has no match or is matched against `undefined` then the default value is used. That is, the pattern is matched against the default value, instead.

Chapter 4

A detailed look at global variables

Contents

4.1	Scopes	41
4.2	Lexical environments	42
4.3	The global object	42
4.4	In browsers, <code>globalThis</code> does not point directly to the global object	42
4.5	The global environment	43
4.5.1	Script scope and module scopes	44
4.5.2	Creating variables: declarative record vs. object record	44
4.5.3	Getting or setting variables	45
4.5.4	Global ECMAScript variables and global host variables	45
4.6	Conclusion: Why does JavaScript have both normal global variables and the global object?	45
4.7	Further reading and sources of this chapter	47

In this chapter, we take a detailed look at how JavaScript’s global variables work. Several interesting phenomena play a role: the scope of scripts, the so-called *global object*, and more.

4.1 Scopes

The *lexical scope* (short: *scope*) of a variable is the region of a program where it can be accessed. JavaScript’s scopes are *static* (they don’t change at runtime) and they can be nested – for example:

```
function func() { // (A)
  const aVariable = 1;
  if (true) { // (B)
    const anotherVariable = 2;
```

```
    }
  }
```

The scope introduced by the `if` statement (line B) is nested inside the scope of function `func()` (line A).

The innermost surrounding scope of a scope *S* is called the *outer scope* of *S*. In the example, `func` is the outer scope of `if`.

4.2 Lexical environments

In the JavaScript language specification, scopes are “implemented” via *lexical environments*. They consist of two components:

- An *environment record* that maps variable names to variable values (think dictionary). This is the actual storage space for the variables of the scope. The name-value entries in the record are called *bindings*.
- A reference to the *outer environment* – the environment for the outer scope.

The tree of nested scopes is therefore represented by a tree of environments linked by outer environment references.

4.3 The global object

The global object is an object whose properties become global variables. (We’ll examine soon how exactly it fits into the tree of environments.) It can be accessed via the following global variables:

- Available on all platforms: `globalThis`. The name is based on the fact that it has the same value as `this` in global scope.
- Other variables for the global object are not available on all platforms:
 - `window` is the classic way of referring to the global object. It works in normal browser code, but not in *Web Workers* (processes running concurrently to the normal browser process) and not on Node.js.
 - `self` is available everywhere in browsers (including in Web Workers). But it isn’t supported by Node.js.
 - `global` is only available on Node.js.

4.4 In browsers, `globalThis` does not point directly to the global object

In browsers, `globalThis` does not point directly to the global, there is an indirection. As an example, consider an `iframe` on a web page:

- Whenever the `src` of the `iframe` changes, it gets a new global object.
- However, `globalThis` always has the same value. That value can be checked from outside the `iframe`, as demonstrated below (inspired by [an example in the `globalThis` proposal](#)).

File `parent.html`:

```
<iframe src="iframe.html?first"></iframe>
<script>
  const iframe = document.querySelector('iframe');
  const icw = iframe.contentWindow; // `globalThis` of iframe

  iframe.onload = () => {
    // Access properties of global object of iframe
    const firstGlobalThis = icw.globalThis;
    const firstArray = icw.Array;
    console.log(icw.iframeName); // 'first'

    iframe.onload = () => {
      const secondGlobalThis = icw.globalThis;
      const secondArray = icw.Array;

      // The global object is different
      console.log(icw.iframeName); // 'second'
      console.log(secondArray === firstArray); // false

      // But globalThis is still the same
      console.log(firstGlobalThis === secondGlobalThis); // true
    };
    iframe.src = 'iframe.html?second';
  };
</script>
```

File `iframe.html`:

```
<script>
  globalThis.iframeName = location.search.slice(1);
</script>
```

How do browsers ensure that `globalThis` doesn't change in this scenario? They internally distinguish two objects:

- `Window` is the global object. It changes whenever the location changes.
- `WindowProxy` is an object that forwards all accesses to the current `Window`. This object never changes.

In browsers, `globalThis` refers to the `WindowProxy`; everywhere else, it directly refers to the global object.

4.5 The global environment

The global scope is the “outermost” scope – it has no outer scope. Its environment is the *global environment*. Every environment is connected with the global environment via a chain of environments that are linked by outer environment references. The outer environment reference of the global environment is `null`.

The global environment record uses two environment records to manage its variables:

- An *object environment record* has the same interface as a normal environment record, but keeps its bindings in a JavaScript object. In this case, the object is the global object.
- A normal (*declarative*) environment record that has its own storage for its bindings.

Which of these two records is used when will be explained soon.

4.5.1 Script scope and module scopes

In JavaScript, we are only in global scope at the top levels of scripts. In contrast, each module has its own scope that is a subscope of the script scope.

If we ignore the relatively complicated rules for how variable bindings are added to the global environment, then global scope and module scopes work as if they were nested code blocks:

```
{ // Global scope (scope of *all* scripts)

  // (Global variables)

  { // Scope of module 1
    ...
  }
  { // Scope of module 2
    ...
  }
  // (More module scopes)
}
```

4.5.2 Creating variables: declarative record vs. object record

In order to create a variable that is truly global, we must be in global scope – which is only the case at the top level of scripts:

- Top-level `const`, `let`, and `class` create bindings in the declarative environment record.
- Top-level `var` and function declarations create bindings in the object environment record.

```
<script>
  const one = 1;
  var two = 2;
</script>
<script>
  // All scripts share the same top-level scope:
  console.log(one); // 1
  console.log(two); // 2

  // Not all declarations create properties of the global object:
```

```
console.log(globalThis.one); // undefined
console.log(globalThis.two); // 2
</script>
```

4.5.3 Getting or setting variables

When we get or set a variable and both environment records have a binding for that variable, then the declarative record wins:

```
<script>
  let myGlobalVariable = 1; // declarative environment record
  globalThis.myGlobalVariable = 2; // object environment record

  console.log(myGlobalVariable); // 1 (declarative record wins)
  console.log(globalThis.myGlobalVariable); // 2
</script>
```

4.5.4 Global ECMAScript variables and global host variables

In addition to variables created via `var` and function declarations, the global object contains the following properties:

- All built-in global variables of ECMAScript
- All built-in global variables of the host platform (browser, Node.js, etc.)

Using `const` or `let` guarantees that global variable declarations aren't influencing (or influenced by) the built-in global variables of ECMAScript and host platform.

For example, browsers have the global variable `.location`:

```
// Changes the location of the current document:
var location = 'https://example.com';

// Shadows window.location, doesn't change it:
let location = 'https://example.com';
```

If a variable already exists (such as `location` in this case), then a `var` declaration with an initializer behaves like an assignment. That's why we get into trouble in this example.

Note that this is only an issue in global scope. In modules, we are never in global scope (unless we use `eval()` or similar).

Fig. 4.1 summarizes everything we have learned in this section.

4.6 Conclusion: Why does JavaScript have both normal global variables and the global object?

The global object is generally considered to be a mistake. For that reason, newer constructs such as `const`, `let`, and classes create normal global variables (when in script scope).

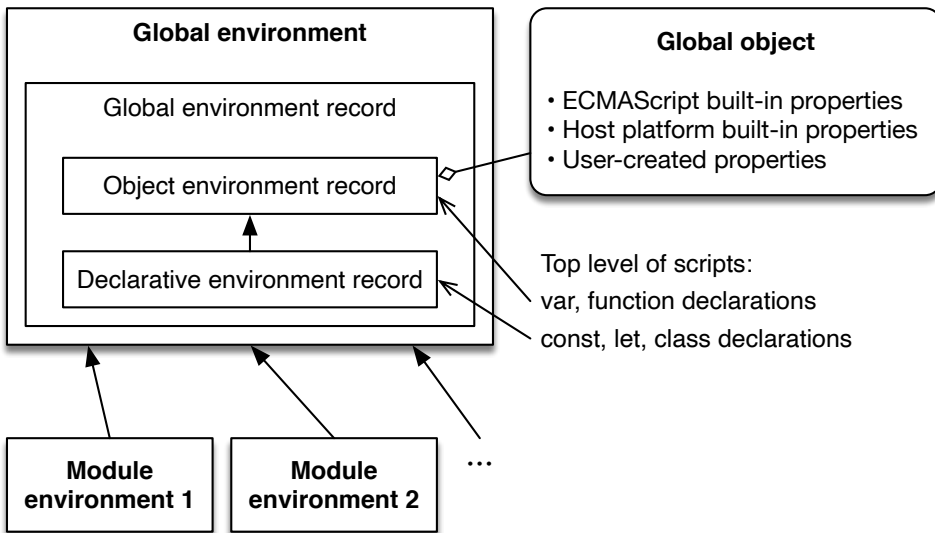


Figure 4.1: The environment for the global scope manages its bindings via a *global environment record* which in turn is based on two environment records: an *object environment record* whose bindings are stored in the global object and a *declarative environment record* that uses internal storage for its bindings. Therefore, global variables can be created by adding properties to the global object or via various declarations. The global object is initialized with the built-in global variables of ECMAScript and the host platform. Each ECMAScript module has its own environment whose outer environment is the global environment.

Thankfully, most of the code written in modern JavaScript, lives in [ECMAScript modules and CommonJS modules](#). Each module has its own scope, which is why the rules governing global variables rarely matter for module-based code.

4.7 Further reading and sources of this chapter

Environments and the global object in the ECMAScript specification:

- [Section “Lexical Environments”](#) provides a general overview over environments.
- [Section “Global Environment Records”](#) covers the global environment.
- [Section “ECMAScript Standard Built-in Objects”](#) describes how ECMAScript manages its built-in objects (which include the global object).

`globalThis`:

- 2ality post [“ES feature: `globalThis`”](#)
- Various ways of accessing the global this value: [“A horrifying `globalThis` polyfill in universal JavaScript”](#) by Mathias Bynens

The global object in browsers:

- Background on what happens in browsers: [“Defining the WindowProxy, Window, and Location objects”](#) by Anne van Kesteren
- Very technical: [section “Realms, settings objects, and global objects”](#) in the WHATWG HTML standard
- In the ECMAScript specification, we can see how web browsers customize global this: [section “InitializeHostDefinedRealm\(\)”](#)

Chapter 5

% is a remainder operator, not a modulo operator (bonus)

Contents

5.1	Remainder operator <code>rem</code> vs. modulo operator <code>mod</code>	49
5.2	An intuitive understanding of the remainder operation	50
5.3	An intuitive understanding of the modulo operation	50
5.4	Similarities and differences between <code>rem</code> and <code>mod</code>	51
5.5	The equations behind remainder and modulo	51
5.5.1	<code>rem</code> and <code>mod</code> perform integer division differently	52
5.5.2	Implementing <code>rem</code>	52
5.5.3	Implementing <code>mod</code>	52
5.6	Where are <code>rem</code> and <code>mod</code> used in programming languages?	53
5.6.1	JavaScript’s <code>%</code> operator computes the remainder	53
5.6.2	Python’s <code>%</code> operator computes the modulus	53
5.6.3	Uses of the modulo operation in JavaScript	53
5.7	Further reading and sources of this chapter	54

Remainder and *modulo* are two similar operations. This chapter explores how they work and reveals that JavaScript’s `%` operator computes the remainder, not the modulus.

5.1 Remainder operator `rem` vs. modulo operator `mod`

In this chapter, we pretend that JavaScript has the following two operators:

- The *remainder operator* `rem`
- The *modulo operator* `mod`

That will help us examine how the underlying operations work.

5.2 An intuitive understanding of the remainder operation

The operands of the remainder operator are called *dividend* and *divisor*:

```
remainder = dividend rem divisor
```

How is the result computed? Consider the following expression:

```
7 rem 3
```

We remove 3 from the dividend until we are left with a value that is smaller than 3:

```
7 rem 3 = 4 rem 3 = 1 rem 3 = 1
```

What do we do if the dividend is negative?

```
-7 rem 3
```

This time, we add 3 to the dividend until we have a value that is smaller than -3:

```
-7 rem 3 = -4 rem 3 = -1 rem 3 = -1
```

It is insightful to map out the results for a fixed divisor:

x:	-7	-6	-5	-4	-3	-2	-1	0	1	2	3	4	5	6	7
x rem 3:	-1	0	-2	-1	0	-2	-1	0	1	2	0	1	2	0	1

Among the results, we can see a symmetry: x and $-x$ produce the same results, but with opposite signs.

5.3 An intuitive understanding of the modulo operation

Once again, the operands are called *dividend* and *divisor* (hinting at how similar `rem` and `mod` are):

```
modulus = dividend mod divisor
```

Consider the following example:

```
x mod 3
```

This operation maps x into the range:

```
[0,3) = {0,1,2}
```

That is, zero is included (opening square bracket), 3 is excluded (closing parenthesis).

How does `mod` perform this mapping? It is easy if x is already inside the range:

```
> 0 mod 3
0
> 2 mod 3
2
```

If x is greater than or equal to the upper boundary of the range, then the upper boundary is subtracted from x until it fits into the range:

```
> 4 mod 3
1
> 7 mod 3
1
```

That means we are getting the following mapping for non-negative integers:

```
x:      0 1 2 3 4 5 6 7
x mod 3: 0 1 2 0 1 2 0 1
```

This mapping is extended as follows, so that it includes negative integers:

```
x:      -7 -6 -5 -4 -3 -2 -1 0 1 2 3 4 5 6 7
x mod 3: 2 0 1 2 0 1 2 0 1 2 0 1 2 0 1
```

Note how the range [0,3) is repeated over and over again.

5.4 Similarities and differences between *rem* and *mod*

rem and *mod* are quite similar – they only differ if dividend and divisor have different signs:

- With *rem*, the result has the same sign as the dividend (first operand):

```
> 5 rem 4
1
> -5 rem 4
-1
> 5 rem -4
1
> -5 rem -4
-1
```

- With *mod*, the result has the same sign as the divisor (second operand):

```
> 5 mod 4
1
> -5 mod 4
3
> 5 mod -4
-3
> -5 mod -4
-1
```

5.5 The equations behind remainder and modulo

The following two equations define both remainder and modulo:

```
dividend = (divisor * quotient) + remainder
|remainder| < |divisor|
```

Given a dividend and a divisor, how do we compute remainder and modulus?

```
remainder = dividend - (divisor * quotient)
quotient = dividend div divisor
```

The `div` operator performs integer division.

5.5.1 `rem` and `mod` perform integer division differently

How can these equations contain the solution for *both* operations? If variable remainder isn't zero, the equations always allow two values for it: one positive, the other one negative. To see why, we turn to the question we are asking when determining the quotient:

How often do we need to multiply the divisor to get as close to the dividend as possible?

For example, that gives us two different results for dividend `-2` and divisor `3`:

- `-2 rem 3 = -2`
 3×0 gets us close to `-2`. The difference between `0` and the dividend `-2` is `-2`.
- `-2 mod 3 = 1`
 3×-1 also gets us close to `-2`. The difference between `-3` and the dividend `-2` is `1`.

It also gives us two different results for dividend `2` and divisor `-3`:

- `2 rem -3 = 2`
 -3×0 gets us close to `2`.
- `2 mod -3 = -1`
 -3×-1 gets us close to `2`.

The results differ depending on how the integer division `div` is implemented.

5.5.2 Implementing `rem`

The following JavaScript function implements the `rem` operator. It performs integer division by performing floating point division and rounding the result to an integer via `Math.trunc()`:

```
function rem(dividend, divisor) {
  const quotient = Math.trunc(dividend / divisor);
  return dividend - (divisor * quotient);
}
```

5.5.3 Implementing `mod`

The following function implements the `mod` operator. This time, integer division is based on `Math.floor()`:

```
function mod(dividend, divisor) {
  const quotient = Math.floor(dividend / divisor);
  return dividend - (divisor * quotient);
}
```

Note that other ways of doing integer division are possible (e.g. based on `Math.ceil()` or `Math.round()`). That means that there are more operations that are similar to *rem* and *mod*.

5.6 Where are *rem* and *mod* used in programming languages?

5.6.1 JavaScript's % operator computes the remainder

For example (Node.js REPL):

```
> 7 % 6
1
> -7 % 6
-1
```

5.6.2 Python's % operator computes the modulus

For example (Python REPL):

```
>>> 7 % 6
1
>>> -7 % 6
5
```

5.6.3 Uses of the modulo operation in JavaScript

The ECMAScript specification uses modulo several times – for example:

- To convert the operands of [the >>> operator](#) to unsigned 32-bit integers (`x mod 2**32`):

```
> 2**32 >>> 0
0
> (2**32)+1 >>> 0
1
> (-1 >>> 0) === (2**32)-1
true
```

- To convert arbitrary numbers so that they fit into Typed Arrays. For example, `x mod 2**8` is used to convert numbers to unsigned 8-bit integers (after first converting them to integers):

```
const tarr = new Uint8Array(1);

tarr[0] = 256;
assert.equal(tarr[0], 0);

tarr[0] = 257;
assert.equal(tarr[0], 1);
```

5.7 Further reading and sources of this chapter

- [The Wikipedia page on “modulo operation”](#) has much information.
- The ECMAScript specification [mentions](#) that % is computed via “truncating division”.
- [Sect. “Rounding”](#) in “JavaScript for impatient programmers” describes several ways of rounding in JavaScript.

Part III

Working with data

Chapter 6

Copying objects and Arrays

Contents

6.1 Shallow copying vs. deep copying	57
6.2 Shallow copying in JavaScript	58
6.2.1 Copying plain objects and Arrays via spreading	58
6.2.2 Shallow copying via <code>Object.assign()</code> (optional)	61
6.2.3 Shallow copying via <code>Object.getOwnPropertyDescriptors()</code> and <code>Object.defineProperties()</code> (optional)	61
6.3 Deep copying in JavaScript	62
6.3.1 Manual deep copying via nested spreading	62
6.3.2 Hack: generic deep copying via JSON	62
6.3.3 Implementing generic deep copying	63
6.4 Further reading	65

In this chapter, we will learn how to copy objects and Arrays in JavaScript.

6.1 Shallow copying vs. deep copying

There are two “depths” with which data can be copied:

- *Shallow copying* only copies the top-level entries of objects and Arrays. The entry values are still the same in original and copy.
- *Deep copying* also copies the entries of the values of the entries, etc. That is, it traverses the complete tree whose root is the value to be copied and makes copies of all nodes.

The next sections cover both kinds of copying. Unfortunately, JavaScript only has built-in support for shallow copying. If we need deep copying, we need to implement it ourselves.

6.2 Shallow copying in JavaScript

Let's look at several ways of shallowly copying data.

6.2.1 Copying plain objects and Arrays via spreading

We can spread [into object literals](#) and [into Array literals](#) to make copies:

```
const copyOfObject = {...originalObject};
const copyOfArray = [...originalArray];
```

Alas, spreading has several issues. Those will be covered in the next subsections. Among those, some are real limitations, others mere peculiarities.

6.2.1.1 The prototype is not copied by object spreading

For example:

```
class MyClass {}

const original = new MyClass();
assert.equal(original instanceof MyClass, true);

const copy = {...original};
assert.equal(copy instanceof MyClass, false);
```

Note that the following two expressions are equivalent:

```
obj instanceof SomeClass
SomeClass.prototype.isPrototypeOf(obj)
```

Therefore, we can fix this by giving the copy the same prototype as the original:

```
class MyClass {}

const original = new MyClass();

const copy = {
  __proto__: Object.getPrototypeOf(original),
  ...original,
};
assert.equal(copy instanceof MyClass, true);
```

Alternatively, we can set the prototype of the copy after its creation, via `Object.setPrototypeOf()`.

6.2.1.2 Many built-in objects have special “internal slots” that aren’t copied by object spreading

Examples of such built-in objects include regular expressions and dates. If we make a copy of them, we lose most of the data stored in them.

6.2.1.3 Only own (non-inherited) properties are copied by object spreading

Given how [prototype chains](#) work, this is usually the right approach. But we still need to be aware of it. In the following example, the inherited property `.inheritedProp` of `original` is not available in `copy` because we only copy own properties and don't keep the prototype.

```
const proto = { inheritedProp: 'a' };
const original = { __proto__: proto, ownProp: 'b' };
assert.equal(original.inheritedProp, 'a');
assert.equal(original.ownProp, 'b');

const copy = {...original};
assert.equal(copy.inheritedProp, undefined);
assert.equal(copy.ownProp, 'b');
```

6.2.1.4 Only enumerable properties are copied by object spreading

For example, the own property `.length` of `Array` instances is not enumerable and not copied. In the following example, we are copying the `Array` `arr` via object spreading (line A):

```
const arr = ['a', 'b'];
assert.equal(arr.length, 2);
assert.equal({}.hasOwnProperty.call(arr, 'length'), true);

const copy = {...arr}; // (A)
assert.equal({}.hasOwnProperty.call(copy, 'length'), false);
```

This is also rarely a limitation because most properties are enumerable. If we need to copy non-enumerable properties, we can use `Object.getOwnPropertyDescriptors()` and `Object.defineProperties()` to copy objects ([how to do that is explained later](#)):

- They consider all attributes (not just value) and therefore correctly copy getters, setters, read-only properties, etc.
- `Object.getOwnPropertyDescriptors()` retrieves both enumerable and non-enumerable properties.

For more information on enumerability, see [§12 “Enumerability of properties”](#).

6.2.1.5 Property attributes aren't always copied faithfully by object spreading

Independently of [the attributes of a property](#), its copy will always be a data property that is writable and configurable.

For example, here we create the property `original.prop` whose attributes `writable` and `configurable` are `false`:

```
const original = Object.defineProperties(
  {}, {
    prop: {
      value: 1,
```

```

    writable: false,
    configurable: false,
    enumerable: true,
  },
});
assert.deepEqual(original, {prop: 1});

```

If we copy `.prop`, then `writable` and `configurable` are both `true`:

```

const copy = {...original};
// Attributes `writable` and `configurable` of copy are different:
assert.deepEqual(
  Object.getOwnPropertyDescriptors(copy),
  {
    prop: {
      value: 1,
      writable: true,
      configurable: true,
      enumerable: true,
    },
  },
);

```

As a consequence, getters and setters are not copied faithfully, either:

```

const original = {
  get myGetter() { return 123 },
  set mySetter(x) {},
};
assert.deepEqual({...original}, {
  myGetter: 123, // not a getter anymore!
  mySetter: undefined,
});

```

The aforementioned `Object.getOwnPropertyDescriptors()` and `Object.defineProperties()` always transfer own properties with all attributes intact (as shown later).

6.2.1.6 Copying is shallow

The copy has fresh versions of each key-value entry in the original, but the values of the original are not copied themselves. For example:

```

const original = {name: 'Jane', work: {employer: 'Acme'}};
const copy = {...original};

// Property .name is a copy: changing the copy
// does not affect the original
copy.name = 'John';
assert.deepEqual(original,
  {name: 'Jane', work: {employer: 'Acme'}});
assert.deepEqual(copy,
  {name: 'John', work: {employer: 'Acme'}});

```

```
// The value of .work is shared: changing the copy
// affects the original
copy.work.employer = 'Spectre';
assert.deepEqual(
  original, {name: 'Jane', work: {employer: 'Spectre'}});
assert.deepEqual(
  copy, {name: 'John', work: {employer: 'Spectre'}});
```

We'll look at deep copying later in this chapter.

6.2.2 Shallow copying via `Object.assign()` (optional)

`Object.assign()` works mostly like spreading into objects. That is, the following two ways of copying are mostly equivalent:

```
const copy1 = {...original};
const copy2 = Object.assign({}, original);
```

Using a method instead of syntax has the benefit that it can be polyfilled on older JavaScript engines via a library.

`Object.assign()` is not completely like spreading, though. It differs in one, relatively subtle point: it creates properties differently.

- `Object.assign()` uses *assignment* to create the properties of the copy.
- Spreading *defines* new properties in the copy.

Among other things, assignment invokes own and inherited setters, while definition doesn't ([more information on assignment vs. definition](#)). This difference is rarely noticeable. The following code is an example, but it's contrived:

```
const original = {['__proto__']: null}; // (A)
const copy1 = {...original};
// copy1 has the own property '__proto__'
assert.deepEqual(
  Object.keys(copy1), ['__proto__']);

const copy2 = Object.assign({}, original);
// copy2 has the prototype null
assert.equal(Object.getPrototypeOf(copy2), null);
```

By using a computed property key in line A, we create `__proto__` as an own property and don't invoke the inherited setter. However, when `Object.assign()` copies that property, it does invoke the setter. (For more information on `__proto__`, see [“JavaScript for impatient programmers”](#).)

6.2.3 Shallow copying via `Object.getOwnPropertyDescriptors()` and `Object.defineProperties()` (optional)

JavaScript lets us create properties via *property descriptors*, objects that specify property attributes. For example, via the `Object.defineProperties()`, which we have already

seen in action. If we combine that method with `Object.getOwnPropertyDescriptors()`, we can copy more faithfully:

```
function copyAllOwnProperties(original) {
  return Object.defineProperties(
    {}, Object.getOwnPropertyDescriptors(original));
}
```

That eliminates two issues of copying objects via spreading.

First, all attributes of own properties are copied correctly. Therefore, we can now copy own getters and own setters:

```
const original = {
  get myGetter() { return 123 },
  set mySetter(x) {},
};
assert.deepEqual(copyAllOwnProperties(original), original);
```

Second, thanks to `Object.getOwnPropertyDescriptors()`, non-enumerable properties are copied, too:

```
const arr = ['a', 'b'];
assert.equal(arr.length, 2);
assert.equal({}.hasOwnProperty.call(arr, 'length'), true);

const copy = copyAllOwnProperties(arr);
assert.equal({}.hasOwnProperty.call(copy, 'length'), true);
```

6.3 Deep copying in JavaScript

Now it is time to tackle deep copying. First, we will deep-copy manually, then we'll examine generic approaches.

6.3.1 Manual deep copying via nested spreading

If we nest spreading, we get deep copies:

```
const original = {name: 'Jane', work: {employer: 'Acme'}};
const copy = {name: original.name, work: {...original.work}};

// We copied successfully:
assert.deepEqual(original, copy);
// The copy is deep:
assert.ok(original.work !== copy.work);
```

6.3.2 Hack: generic deep copying via JSON

This is a hack, but, in a pinch, it provides a quick solution: In order to deep-copy an object `original`, we first convert it to a JSON string and parse that JSON string:

```
function jsonDeepCopy(original) {
  return JSON.parse(JSON.stringify(original));
}
const original = {name: 'Jane', work: {employer: 'Acme'}};
const copy = jsonDeepCopy(original);
assert.deepEqual(original, copy);
```

The significant downside of this approach is that we can only copy properties with keys and values that are supported by JSON.

Some unsupported keys and values are simply ignored:

```
assert.deepEqual(
  jsonDeepCopy({
    // Symbols are not supported as keys
    [Symbol('a')]: 'abc',
    // Unsupported value
    b: function () {},
    // Unsupported value
    c: undefined,
  }),
  {} // empty object
);
```

Others cause exceptions:

```
assert.throws(
  () => jsonDeepCopy({a: 123n}),
  /^TypeError: Do not know how to serialize a BigInt$/);
```

6.3.3 Implementing generic deep copying

The following function generically deep-copies a value original:

```
function deepCopy(original) {
  if (Array.isArray(original)) {
    const copy = [];
    for (const [index, value] of original.entries()) {
      copy[index] = deepCopy(value);
    }
    return copy;
  } else if (typeof original === 'object' && original !== null) {
    const copy = {};
    for (const [key, value] of Object.entries(original)) {
      copy[key] = deepCopy(value);
    }
    return copy;
  } else {
    // Primitive value: atomic, no need to copy
    return original;
  }
}
```

```
    }
  }
}
```

The function handles three cases:

- If `original` is an Array we create a new Array and deep-copy the elements of `original` into it.
- If `original` is an object, we use a similar approach.
- If `original` is a primitive value, we don't have to do anything.

Let's try out `deepCopy()`:

```
const original = {a: 1, b: {c: 2, d: {e: 3}}};
const copy = deepCopy(original);

// Are copy and original deeply equal?
assert.deepEqual(copy, original);

// Did we really copy all levels
// (equal content, but different objects)?
assert.ok(copy !== original);
assert.ok(copy.b !== original.b);
assert.ok(copy.b.d !== original.b.d);
```

Note that `deepCopy()` only fixes one issue of spreading: shallow copying. All others remain: prototypes are not copied, special objects are only partially copied, non-enumerable properties are ignored, most property attributes are ignored.

Implementing copying completely generically is generally impossible: Not all data is a tree, sometimes we don't want to copy all properties, etc.

6.3.3.1 A more concise version of `deepCopy()`

We can make our previous implementation of `deepCopy()` more concise if we use `.map()` and `Object.fromEntries()`:

```
function deepCopy(original) {
  if (Array.isArray(original)) {
    return original.map(elem => deepCopy(elem));
  } else if (typeof original === 'object' && original !== null) {
    return Object.fromEntries(
      Object.entries(original)
        .map(([k, v]) => [k, deepCopy(v)]));
  } else {
    // Primitive value: atomic, no need to copy
    return original;
  }
}
```

6.4 Further reading

- §14 “Copying instances of classes: `.clone()` vs. copy constructors” explains class-based patterns for copying.
- Section “Spreading into object literals” in “JavaScript for impatient programmers”
- Section “Spreading into Array literals” in “JavaScript for impatient programmers”
- Section “Prototype chains” in “JavaScript for impatient programmers”

Chapter 7

Updating data destructively and non-destructively

Contents

7.1	Examples: updating an object destructively and non-destructively	67
7.2	Examples: updating an Array destructively and non-destructively	68
7.3	Manual deep updating	69
7.4	Implementing generic deep updating	69

In this chapter, we learn about two different ways of updating data:

- A *destructive update* of data mutates the data so that it has the desired form.
- A *non-destructive update* of data creates a copy of the data that has the desired form.

The latter way is similar to first making a copy and then changing it destructively, but it does both at the same time.

7.1 Examples: updating an object destructively and non-destructively

The following code shows a function that updates object properties destructively and uses it on an object.

```
function setPropertyDestructively(obj, key, value) {
  obj[key] = value;
  return obj;
}

const obj = {city: 'Berlin', country: 'Germany'};
setPropertyDestructively(obj, 'city', 'Munich');
assert.deepEqual(obj, {city: 'Munich', country: 'Germany'});
```

The following code demonstrates non-destructive updating of an object:

```
function setPropertyNonDestructively(obj, key, value) {
  const updatedObj = {};
  for (const [k, v] of Object.entries(obj)) {
    updatedObj[k] = (k === key ? value : v);
  }
  return updatedObj;
}

const obj = {city: 'Berlin', country: 'Germany'};
const updatedObj = setPropertyNonDestructively(obj, 'city', 'Munich');

// We have created an updated object:
assert.deepEqual(updatedObj, {city: 'Munich', country: 'Germany'});

// But we didn't change the original:
assert.deepEqual(obj, {city: 'Berlin', country: 'Germany'});
```

Spreading makes `setPropertyNonDestructively()` more concise:

```
function setPropertyNonDestructively(obj, key, value) {
  return {...obj, [key]: value};
}
```

Both versions of `setPropertyNonDestructively()` update *shallowly*: They only change the top level of an object.

7.2 Examples: updating an Array destructively and non-destructively

The following code shows a function that updates Array elements destructively and uses it on an Array.

```
function setElementDestructively(arr, index, value) {
  arr[index] = value;
}

const arr = ['a', 'b', 'c', 'd', 'e'];
setElementDestructively(arr, 2, 'x');
assert.deepEqual(arr, ['a', 'b', 'x', 'd', 'e']);
```

The following code demonstrates non-destructive updating of an Array:

```
function setElementNonDestructively(arr, index, value) {
  const updatedArr = [];
  for (const [i, v] of arr.entries()) {
    updatedArr.push(i === index ? value : v);
  }
  return updatedArr;
}
```

```

}

const arr = ['a', 'b', 'c', 'd', 'e'];
const updatedArr = setElementNonDestructively(arr, 2, 'x');
assert.deepEqual(updatedArr, ['a', 'b', 'x', 'd', 'e']);
assert.deepEqual(arr, ['a', 'b', 'c', 'd', 'e']);

```

`.slice()` and spreading make `setElementNonDestructively()` more concise:

```

function setElementNonDestructively(arr, index, value) {
  return [
    ...arr.slice(0, index), value, ...arr.slice(index+1)];
}

```

Both versions of `setElementNonDestructively()` update *shallowly*: They only change the top level of an Array.

7.3 Manual deep updating

So far, we have only updated data shallowly. Let's tackle deep updating. The following code shows how to do it manually. We are changing name and employer.

```

const original = {name: 'Jane', work: {employer: 'Acme'}};
const updatedOriginal = {
  ...original,
  name: 'John',
  work: {
    ...original.work,
    employer: 'Spectre'
  },
};

assert.deepEqual(
  original, {name: 'Jane', work: {employer: 'Acme'}});
assert.deepEqual(
  updatedOriginal, {name: 'John', work: {employer: 'Spectre'}});

```

7.4 Implementing generic deep updating

The following function implements generic deep updating.

```

function deepUpdate(original, keys, value) {
  if (keys.length === 0) {
    return value;
  }
  const currentKey = keys[0];
  if (Array.isArray(original)) {
    return original.map(
      (v, index) => index === currentKey

```

```

    ? deepUpdate(v, keys.slice(1), value) // (A)
    : v); // (B)
} else if (typeof original === 'object' && original !== null) {
  return Object.fromEntries(
    Object.entries(original).map(
      (keyValuePair) => {
        const [k,v] = keyValuePair;
        if (k === currentKey) {
          return [k, deepUpdate(v, keys.slice(1), value)]; // (C)
        } else {
          return keyValuePair; // (D)
        }
      }
    ));
} else {
  // Primitive value
  return original;
}
}

```

If we see `value` as the root of a tree that we are updating, then `deepUpdate()` only deeply changes a single branch (line A and C). All other branches are copied shallowly (line B and D).

This is what using `deepUpdate()` looks like:

```

const original = {name: 'Jane', work: {employer: 'Acme'}};

const copy = deepUpdate(original, ['work', 'employer'], 'Spectre');
assert.deepEqual(copy, {name: 'Jane', work: {employer: 'Spectre'}});
assert.deepEqual(original, {name: 'Jane', work: {employer: 'Acme'}});

```

Chapter 8

The problems of shared mutable state and how to avoid them

Contents

8.1	What is shared mutable state and why is it problematic?	71
8.2	Avoiding sharing by copying data	73
8.2.1	How does copying help with shared mutable state?	73
8.3	Avoiding mutations by updating non-destructively	75
8.3.1	How does non-destructive updating help with shared mutable state?	75
8.4	Preventing mutations by making data immutable	76
8.4.1	How does immutability help with shared mutable state?	76
8.5	Libraries for avoiding shared mutable state	76
8.5.1	Immutable.js	76
8.5.2	Immer	77

This chapter answers the following questions:

- What is shared mutable state?
- Why is it problematic?
- How can its problems be avoided?

8.1 What is shared mutable state and why is it problematic?

Shared mutable state works as follows:

- If two or more parties can change the same data (variables, objects, etc.).
- And if their lifetimes overlap.
- Then there is a risk of one party’s modifications preventing other parties from working correctly.

Note that this definition applies to function calls, cooperative multitasking (e.g., async functions in JavaScript), etc. The risks are similar in each case.

The following code is an example. The example is not realistic, but it demonstrates the risks and is easy to understand:

```
function logElements(arr) {
  while (arr.length > 0) {
    console.log(arr.shift());
  }
}

function main() {
  const arr = ['banana', 'orange', 'apple'];

  console.log('Before sorting:');
  logElements(arr);

  arr.sort(); // changes arr

  console.log('After sorting:');
  logElements(arr); // (A)
}
main();

// Output:
// 'Before sorting:'
// 'banana'
// 'orange'
// 'apple'
// 'After sorting:'
```

In this case, there are two independent parties:

- Function `main()` wants to log an Array before and after sorting it.
- Function `logElements()` logs the elements of its parameter `arr`, but removes them while doing so.

`logElements()` breaks `main()` and causes it to log an empty Array in line A.

In the remainder of this chapter, we look at three ways of avoiding the problems of shared mutable state:

- Avoiding sharing by copying data
- Avoiding mutations by updating non-destructively
- Preventing mutations by making data immutable

In particular, we will come back to the example that we've just seen and fix it.

8.2 Avoiding sharing by copying data

Copying data is one way of avoiding sharing it.



Background

For background on copying data in JavaScript, please refer to the following two chapters in this book:

- §6 “Copying objects and Arrays”
- §14 “Copying instances of classes: `.clone()` vs. copy constructors”

8.2.1 How does copying help with shared mutable state?

As long as we only *read* from shared state, we don’t have any problems. Before *modifying* it, we need to “un-share” it, by copying it (as deeply as necessary).

Defensive copying is a technique to always copy when issues *might* arise. Its objective is to keep the current entity (function, class, etc.) safe:

- Input: Copying (potentially) shared data passed to us, lets us use that data without being disturbed by an external entity.
- Output: Copying internal data before exposing it to an outside party, means that that party can’t disrupt our internal activity.

Note that these measures protect us from other parties, but they also protect other parties from us.

The next sections illustrate both kinds of defensive copying.

8.2.1.1 Copying shared input

Remember that in the motivating example at the beginning of this chapter, we got into trouble because `logElements()` modified its parameter `arr`:

```
function logElements(arr) {  
  while (arr.length > 0) {  
    console.log(arr.shift());  
  }  
}
```

Let’s add defensive copying to this function:

```
function logElements(arr) {  
  arr = [...arr]; // defensive copy  
  while (arr.length > 0) {  
    console.log(arr.shift());  
  }  
}
```

Now `logElements()` doesn’t cause problems anymore, if it is called inside `main()`:

```

function main() {
  const arr = ['banana', 'orange', 'apple'];

  console.log('Before sorting:');
  logElements(arr);

  arr.sort(); // changes arr

  console.log('After sorting:');
  logElements(arr); // (A)
}
main();

// Output:
// 'Before sorting:'
// 'banana'
// 'orange'
// 'apple'
// 'After sorting:'
// 'apple'
// 'banana'
// 'orange'

```

8.2.1.2 Copying exposed internal data

Let's start with a class `StringBuilder` that doesn't copy internal data it exposes (line A):

```

class StringBuilder {
  _data = [];
  add(str) {
    this._data.push(str);
  }
  getParts() {
    // We expose internals without copying them:
    return this._data; // (A)
  }
  toString() {
    return this._data.join('');
  }
}

```

As long as `.getParts()` isn't used, everything works well:

```

const sb1 = new StringBuilder();
sb1.add('Hello');
sb1.add(' world!');
assert.equal(sb1.toString(), 'Hello world!');

```

If, however, the result of `.getParts()` is changed (line A), then the `StringBuilder` ceases to work correctly:

```
const sb2 = new StringBuilder();
sb2.add('Hello');
sb2.add(' world!');
sb2.getParts().length = 0; // (A)
assert.equal(sb2.toString(), ''); // not OK
```

The solution is to copy the internal `._data` defensively before it is exposed (line A):

```
class StringBuilder {
  this._data = [];
  add(str) {
    this._data.push(str);
  }
  getParts() {
    // Copy defensively
    return [...this._data]; // (A)
  }
  toString() {
    return this._data.join('');
  }
}
```

Now changing the result of `.getParts()` doesn't interfere with the operation of `sb` anymore:

```
const sb = new StringBuilder();
sb.add('Hello');
sb.add(' world!');
sb.getParts().length = 0;
assert.equal(sb.toString(), 'Hello world!'); // OK
```

8.3 Avoiding mutations by updating non-destructively

We can avoid mutations if we only update data non-destructively.



Background

For more information on updating data, see §7 “Updating data destructively and non-destructively”.

8.3.1 How does non-destructive updating help with shared mutable state?

With non-destructive updating, sharing data becomes unproblematic, because we never mutate the shared data. (This only works if everyone that accesses the data does that!)

Intriguingly, copying data becomes trivially simple:

```
const original = {city: 'Berlin', country: 'Germany'};
```

```
const copy = original;
```

This works, because we are only making non-destructive changes and are therefore copying the data on demand.

8.4 Preventing mutations by making data immutable

We can prevent mutations of shared data by making that data immutable.



Background

For background on how to make data immutable in JavaScript, please refer to the following two chapters in this book:

- §10 “Protecting objects from being changed”
- §15 “Immutable wrappers for collections”

8.4.1 How does immutability help with shared mutable state?

If data is immutable, it can be shared without any risks. In particular, there is no need to copy defensively.



Non-destructive updating is an important complement to immutable data

If we combine the two, immutable data becomes virtually as versatile as mutable data but without the associated risks.

8.5 Libraries for avoiding shared mutable state

There are several libraries available for JavaScript that support immutable data with non-destructive updating. Two popular ones are:

- [Immutable.js](#) provides immutable data structures for lists, stacks, sets, maps, etc.
- [Immer](#) also supports immutability and non-destructive updating but for plain objects, Arrays, Sets, and Maps. That is, no new data structures are needed.

These libraries are described in more detail in the next two sections.

8.5.1 Immutable.js

In its repository, [the library Immutable.js](#) is described as:

Immutable persistent data collections for JavaScript which increase efficiency and simplicity.

Immutable.js provides immutable data structures such as:

- List
- Stack

- Set (which is different from JavaScript's built-in Set)
- Map (which is different from JavaScript's built-in Map)
- Etc.

In the following example, we use an immutable Map:

```
import {Map} from 'immutable/dist/immutable.es.js';
const map0 = Map([
  [false, 'no'],
  [true, 'yes'],
]);

// We create a modified version of map0:
const map1 = map0.set(true, 'maybe');

// The modified version is different from the original:
assert.ok(map1 !== map0);
assert.equal(map1.equals(map0), false); // (A)

// We undo the change we just made:
const map2 = map1.set(true, 'yes');

// map2 is a different object than map0,
// but it has the same content
assert.ok(map2 !== map0);
assert.equal(map2.equals(map0), true); // (B)
```

Notes:

- Instead of modifying the receiver, “destructive” operations such as `.set()` return modified copies.
- To check if two data structures have the same content, we use the built-in `.equals()` method (line A and line B).

8.5.2 Immer

In its repository, [the library Immer](#) is described as:

Create the next immutable state by mutating the current one.

Immer helps with non-destructively updating (potentially nested) plain objects, Arrays, Sets, and Maps. That is, there are no custom data structures involved.

This is what using Immer looks like:

```
import {produce} from 'immer/dist/immer.module.js';

const people = [
  {name: 'Jane', work: {employer: 'Acme'}},
];

const modifiedPeople = produce(people, (draft) => {
```

```
draft[0].work.employer = 'Cyberdyne';
draft.push({name: 'John', work: {employer: 'Spectre'}});
});

assert.deepEqual(modifiedPeople, [
  {name: 'Jane', work: {employer: 'Cyberdyne'}},
  {name: 'John', work: {employer: 'Spectre'}},
]);
assert.deepEqual(people, [
  {name: 'Jane', work: {employer: 'Acme'}},
]);
```

The original data is stored in `people`. `produce()` provides us with a variable `draft`. We pretend that this variable is `people` and use operations with which we would normally make destructive changes. Immer intercepts these operations. Instead of mutating `draft`, it non-destructively changes `people`. The result is referenced by `modifiedPeople`. As a bonus, it is deeply immutable.

`assert.deepEqual()` works because Immer returns plain objects and Arrays.

Part IV

OOP: object property attributes

Chapter 9

Property attributes: an introduction

Contents

9.1 The structure of objects	82
9.1.1 Internal slots	82
9.1.2 Property keys	83
9.1.3 Property attributes	83
9.2 Property descriptors	84
9.3 Retrieving descriptors for properties	85
9.3.1 <code>Object.getOwnPropertyDescriptor()</code> : retrieving a descriptor for a single property	85
9.3.2 <code>Object.getOwnPropertyDescriptors()</code> : retrieving descriptors for all properties of an object	85
9.4 Defining properties via descriptors	86
9.4.1 <code>Object.defineProperty()</code> : defining single properties via descriptors	86
9.4.2 <code>Object.defineProperties()</code> : defining multiple properties via descriptors	88
9.5 <code>Object.create()</code>: Creating objects via descriptors	88
9.6 Use cases for <code>Object.getOwnPropertyDescriptors()</code>	89
9.6.1 Use case: copying properties into an object	89
9.6.2 Use case for <code>Object.getOwnPropertyDescriptors()</code> : cloning objects	91
9.7 Omitting descriptor properties	91
9.7.1 Omitting descriptor properties when creating properties	91
9.7.2 Omitting descriptor properties when changing properties	92
9.8 What property attributes do built-in constructs use?	92
9.8.1 Own properties created via assignment	92
9.8.2 Own properties created via object literals	93
9.8.3 The own property <code>.length</code> of Arrays	93

9.8.4

Prototype properties of built-in classes

93

9.8.5

Prototype properties and instance properties of user-defined classes

94

9.9

API: property descriptors

95

9.10

Further reading

97

In this chapter, we take a closer look at how the ECMAScript specification sees JavaScript objects. In particular, properties are not atomic in the spec, but composed of multiple *attributes* (think fields in a record). Even the value of a data property is stored in an attribute!

9.1 The structure of objects

In the ECMAScript specification, an object consists of:

- *Internal slots*, which are storage locations that are not accessible from JavaScript, only from operations in the specification.
- A collection of *properties*. Each property associates a *key* with *attributes* (think fields in a record).

9.1.1 Internal slots

The specification describes internal slots as follows. I added bullet points and emphasized one part:

- Internal slots correspond to internal state that is associated with objects and used by various ECMAScript specification algorithms.
- Internal slots are not object properties and they are not inherited.
- Depending upon the specific internal slot specification, such state may consist of:
 - values of any ECMAScript language type or
 - of specific ECMAScript specification type values.
- Unless explicitly specified otherwise, internal slots are allocated as part of the process of creating an object and may not be dynamically added to an object.
- Unless specified otherwise, the initial value of an internal slot is the value undefined.
- Various algorithms within this specification create objects that have internal slots. However, **the ECMAScript language provides no direct way to associate internal slots with an object.**
- Internal methods and internal slots are identified within this specification using names enclosed in double square brackets `[[ ]]`.

There are two kinds of internal slots:

- Method slots for manipulating objects (getting properties, setting properties, etc.)
- Data slots that store values.

Ordinary objects have the following data slots:

- `.[[Prototype]]`: `null` | `object`
 - Stores the prototype of an object.

- Can be accessed indirectly via `Object.getPrototypeOf()` and `Object.setPrototypeOf()`.
- `.[[Extensible]]`: boolean
 - Indicates if it is possible to add properties to an object.
 - Can be set to false via `Object.preventExtensions()`.
- `.[[PrivateFieldValues]]`: EntryList
 - Is used to manage [private class fields](#).

9.1.2 Property keys

The key of a property is either:

- A string
- A symbol

9.1.3 Property attributes

There are two kinds of properties and they are characterized by their attributes:

- A *data property* stores data. Its attribute value holds any JavaScript value.
- An *accessor property* consists of a getter function and/or a setter function. The former is stored in the attribute `get`, the latter in the attribute `set`.

Additionally, there are attributes that both kinds of properties have. The following table lists all attributes and their default values.

Kind of property	Name and type of attribute	Default value
Data property	<code>value</code> : any	undefined
	<code>writable</code> : boolean	false
Accessor property	<code>get</code> : (this: any) => any	undefined
	<code>set</code> : (this: any, v: any) => void	undefined
All properties	<code>configurable</code> : boolean	false
	<code>enumerable</code> : boolean	false

We have already encountered the attributes `value`, `get`, and `set`. The other attributes work as follows:

- `writable` determines if the value of a data property can be changed.
- `configurable` determines if the attributes of a property can be changed. If it is false, then:
 - We cannot delete the property.
 - We cannot change a property from a data property to an accessor property or vice versa.
 - We cannot change any attribute other than `value`.
 - However, one more attribute change is allowed: We can change `writable` from true to false. The rationale behind this anomaly is [historical](#): Property `.length` of Arrays has always been writable and non-configurable. Allowing its `writable` attribute to be changed enables us to freeze Arrays.
- `enumerable` influences some operations (such as `Object.keys()`). If it is false,

then those operations ignore the property. Most properties are enumerable (e.g. those created via assignment or object literals), which is why you'll rarely notice this attribute in practice. If you are still interested in how it works, see §12 "Enumerability of properties".

9.1.3.1 Pitfall: Inherited non-writable properties prevent creating own properties via assignment

If an inherited property is non-writable, we can't use assignment to create an own property with the same key:

```
const proto = {
  prop: 1,
};
// Make proto.prop non-writable:
Object.defineProperty(
  proto, 'prop', {writable: false});

const obj = Object.create(proto);

assert.throws(
  () => obj.prop = 2,
  /^TypeError: Cannot assign to read only property 'prop'/);
```

For more information, see §11.3.4 "Inherited read-only properties prevent creating own properties via assignment".

9.2 Property descriptors

A *property descriptor* encodes the attributes of a property as a JavaScript object. Their TypeScript interfaces look as follows.

```
interface DataPropertyDescriptor {
  value?: any;
  writable?: boolean;
  configurable?: boolean;
  enumerable?: boolean;
}
interface AccessorPropertyDescriptor {
  get?: (this: any) => any;
  set?: (this: any, v: any) => void;
  configurable?: boolean;
  enumerable?: boolean;
}
type PropertyDescriptor = DataPropertyDescriptor | AccessorPropertyDescriptor;
```

The question marks indicate that all properties are optional. §9.7 "Omitting descriptor properties" describes what happens if they are omitted.

9.3 Retrieving descriptors for properties

9.3.1 `Object.getOwnPropertyDescriptor()`: retrieving a descriptor for a single property

Consider the following object:

```
const legoBrick = {
  kind: 'Plate 1x3',
  color: 'yellow',
  get description() {
    return `${this.kind} (${this.color})`;
  },
};
```

Let's first get a descriptor for the data property `.color`:

```
assert.deepEqual(
  Object.getOwnPropertyDescriptor(legoBrick, 'color'),
  {
    value: 'yellow',
    writable: true,
    enumerable: true,
    configurable: true,
  });
```

This is what the descriptor for the accessor property `.description` looks like:

```
const desc = Object.getOwnPropertyDescriptor.bind(Object);
assert.deepEqual(
  Object.getOwnPropertyDescriptor(legoBrick, 'description'),
  {
    get: desc(legoBrick, 'description').get, // (A)
    set: undefined,
    enumerable: true,
    configurable: true
  });
```

Using the utility function `desc()` in line A ensures that `.deepEqual()` works.

9.3.2 `Object.getOwnPropertyDescriptors()`: retrieving descriptors for all properties of an object

```
const legoBrick = {
  kind: 'Plate 1x3',
  color: 'yellow',
  get description() {
    return `${this.kind} (${this.color})`;
  },
};
```

```

const desc = Object.getOwnPropertyDescriptor.bind(Object);
assert.deepEqual(
  Object.getOwnPropertyDescriptors(legoBrick),
  {
    kind: {
      value: 'Plate 1x3',
      writable: true,
      enumerable: true,
      configurable: true,
    },
    color: {
      value: 'yellow',
      writable: true,
      enumerable: true,
      configurable: true,
    },
    description: {
      get: desc(legoBrick, 'description').get, // (A)
      set: undefined,
      enumerable: true,
      configurable: true,
    },
  },
);

```

Using the helper function `desc()` in line A ensures that `.deepEqual()` works.

9.4 Defining properties via descriptors

If we define a property with the key `k` via a property descriptor `propDesc`, then what happens depends:

- If there is no property with key `k`, a new own property is created that has the attributes specified by `propDesc`.
- If there is a property with key `k`, defining changes the property's attributes so that they match `propDesc`.

9.4.1 `Object.defineProperty()`: defining single properties via descriptors

First, let us create a new property via a descriptor:

```

const car = {};

Object.defineProperty(car, 'color', {
  value: 'blue',
  writable: true,
  enumerable: true,
  configurable: true,
});

```

```
assert.deepEqual(  
  car,  
  {  
    color: 'blue',  
  });
```

Next, we change the kind of a property via a descriptor; we turn a data property into a getter:

```
const car = {  
  color: 'blue',  
};  
  
let readCount = 0;  
Object.defineProperty(car, 'color', {  
  get() {  
    readCount++;  
    return 'red';  
  },  
});  
  
assert.equal(car.color, 'red');  
assert.equal(readCount, 1);
```

Lastly, we change the value of a data property via a descriptor:

```
const car = {  
  color: 'blue',  
};  
  
// Use the same attributes as assignment:  
Object.defineProperty(  
  car, 'color', {  
    value: 'green',  
    writable: true,  
    enumerable: true,  
    configurable: true,  
  });  
  
assert.deepEqual(  
  car,  
  {  
    color: 'green',  
  });
```

We have used the same property attributes as assignment.

9.4.2 `Object.defineProperty()`: defining multiple properties via descriptors

`Object.defineProperty()` is the multi-property version of `Object.defineProperty()`:

```
const legoBrick1 = {};
Object.defineProperty(
  legoBrick1,
  {
    kind: {
      value: 'Plate 1x3',
      writable: true,
      enumerable: true,
      configurable: true,
    },
    color: {
      value: 'yellow',
      writable: true,
      enumerable: true,
      configurable: true,
    },
    description: {
      get: function () {
        return `${this.kind} (${this.color})`;
      },
      enumerable: true,
      configurable: true,
    },
  },
  {});

assert.deepEqual(
  legoBrick1,
  {
    kind: 'Plate 1x3',
    color: 'yellow',
    get description() {
      return `${this.kind} (${this.color})`;
    },
  },
  {});
```

9.5 `Object.create()`: Creating objects via descriptors

`Object.create()` creates a new object. Its first argument specifies the prototype of that object. Its optional second argument specifies descriptors for the properties of that object. In the next example, we create the same object as in the previous example.

```
const legoBrick2 = Object.create(
  Object.prototype,
  {
```

```

    kind: {
      value: 'Plate 1x3',
      writable: true,
      enumerable: true,
      configurable: true,
    },
    color: {
      value: 'yellow',
      writable: true,
      enumerable: true,
      configurable: true,
    },
    description: {
      get: function () {
        return `${this.kind} (${this.color})`;
      },
      enumerable: true,
      configurable: true,
    },
  },
});

// Did we really create the same object?
assert.deepEqual(legoBrick1, legoBrick2); // Yes!

```

9.6 Use cases for `Object.getOwnPropertyDescriptors()`

`Object.getOwnPropertyDescriptors()` helps us with two use cases, if we combine it with `Object.defineProperties()` or `Object.create()`.

9.6.1 Use case: copying properties into an object

Since ES6, JavaScript already has had a tool method for copying properties: `Object.assign()`. However, this method uses simple get and set operations to copy a property whose key is key:

```
target[key] = source[key];
```

That means that it only creates a faithful copy of a property if:

- Its attribute `writable` is `true` and its attribute `enumerable` is `true` (because that's how assignment creates properties).
- It is a data property.

The following example illustrates this limitation. Object `source` has a setter whose key is `data`.

```

const source = {
  set data(value) {
    this._data = value;
  }
}

```

```
};

// Property `data` exists because there is only a setter
// but has the value `undefined`.
assert.equal('data' in source, true);
assert.equal(source.data, undefined);
```

If we use `Object.assign()` to copy property data, then the accessor property data is converted to a data property:

```
const target1 = {};
Object.assign(target1, source);

assert.deepEqual(
  Object.getOwnPropertyDescriptor(target1, 'data'),
  {
    value: undefined,
    writable: true,
    enumerable: true,
    configurable: true,
  });

// For comparison, the original:
const desc = Object.getOwnPropertyDescriptor.bind(Object);
assert.deepEqual(
  Object.getOwnPropertyDescriptor(source, 'data'),
  {
    get: undefined,
    set: desc(source, 'data').set,
    enumerable: true,
    configurable: true,
  });
```

Fortunately, using `Object.getOwnPropertyDescriptors()` together with `Object.defineProperties()` does faithfully copy the property data:

```
const target2 = {};
Object.defineProperties(
  target2, Object.getOwnPropertyDescriptors(source));

assert.deepEqual(
  Object.getOwnPropertyDescriptor(target2, 'data'),
  {
    get: undefined,
    set: desc(source, 'data').set,
    enumerable: true,
    configurable: true,
  });
```

9.6.1.1 Pitfall: copying methods that use `super`

A method that uses `super` is firmly connected with its *home object* (the object it is stored in). There is currently no way to copy or move such a method to a different object.

9.6.2 Use case for `Object.getOwnPropertyDescriptors()`: cloning objects

Shallow cloning is similar to copying properties, which is why `Object.getOwnPropertyDescriptors()` is a good choice here, too.

To create the clone, we use `Object.create()`:

```
const original = {
  set data(value) {
    this._data = value;
  }
};

const clone = Object.create(
  Object.getPrototypeOf(original),
  Object.getOwnPropertyDescriptors(original));

assert.deepEqual(original, clone);
```

For more information on this topic, see §6 “Copying objects and Arrays”.

9.7 Omitting descriptor properties

All properties of descriptors are optional. What happens when you omit a property depends on the operation.

9.7.1 Omitting descriptor properties when creating properties

When we create a new property via a descriptor, then omitting attributes means that their default values are used:

```
const car = {};
Object.defineProperty(
  car, 'color', {
    value: 'red',
  });
assert.deepEqual(
  Object.getOwnPropertyDescriptor(car, 'color'),
  {
    value: 'red',
    writable: false,
    enumerable: false,
    configurable: false,
  });
```

9.7.2 Omitting descriptor properties when changing properties

If instead, we change an existing property, then omitting descriptor properties means that the corresponding attributes are not touched:

```
const car = {
  color: 'yellow',
};
assert.deepEqual(
  Object.getOwnPropertyDescriptor(car, 'color'),
  {
    value: 'yellow',
    writable: true,
    enumerable: true,
    configurable: true,
  });
Object.defineProperty(
  car, 'color', {
    value: 'pink',
  });
assert.deepEqual(
  Object.getOwnPropertyDescriptor(car, 'color'),
  {
    value: 'pink',
    writable: true,
    enumerable: true,
    configurable: true,
  });
```

9.8 What property attributes do built-in constructs use?

The general rule (with few exceptions) for property attributes is:

- Properties of objects at the beginning of a prototype chain are usually writable, enumerable, and configurable.
- As described in [the chapter on enumerability](#), most inherited properties are non-enumerable, to hide them from legacy constructs such as `for-in` loops. Inherited properties are usually writable and configurable.

9.8.1 Own properties created via assignment

```
const obj = {};
obj.prop = 3;

assert.deepEqual(
  Object.getOwnPropertyDescriptors(obj),
  {
    prop: {
```

```

    value: 3,
    writable: true,
    enumerable: true,
    configurable: true,
  }
});

```

9.8.2 Own properties created via object literals

```

const obj = { prop: 'yes' };

assert.deepEqual(
  Object.getOwnPropertyDescriptors(obj),
  {
    prop: {
      value: 'yes',
      writable: true,
      enumerable: true,
      configurable: true
    }
  }
});

```

9.8.3 The own property `.length` of Arrays

The own property `.length` of Arrays is non-enumerable, so that it isn't copied by `Object.assign()`, spreading, and similar operations. It is also non-configurable:

```

> Object.getOwnPropertyDescriptor([], 'length')
{ value: 0, writable: true, enumerable: false, configurable: false }
> Object.getOwnPropertyDescriptor('abc', 'length')
{ value: 3, writable: false, enumerable: false, configurable: false }

```

`.length` is a special data property, in that it is influenced by (and influences) other own properties (specifically, index properties).

9.8.4 Prototype properties of built-in classes

```

assert.deepEqual(
  Object.getOwnPropertyDescriptor(Array.prototype, 'map'),
  {
    value: Array.prototype.map,
    writable: true,
    enumerable: false,
    configurable: true
  }
});

```

9.8.5 Prototype properties and instance properties of user-defined classes

```

class DataContainer {
  accessCount = 0;
  constructor(data) {
    this.data = data;
  }
  getData() {
    this.accessCount++;
    return this.data;
  }
}
assert.deepEqual(
  Object.getOwnPropertyDescriptors(DataContainer.prototype),
  {
    constructor: {
      value: DataContainer,
      writable: true,
      enumerable: false,
      configurable: true,
    },
    getData: {
      value: DataContainer.prototype.getData,
      writable: true,
      enumerable: false,
      configurable: true,
    }
  }
);

```

Note that all own properties of instances of `DataContainer` are writable, enumerable, and configurable:

```

const dc = new DataContainer('abc')
assert.deepEqual(
  Object.getOwnPropertyDescriptors(dc),
  {
    accessCount: {
      value: 0,
      writable: true,
      enumerable: true,
      configurable: true,
    },
    data: {
      value: 'abc',
      writable: true,
      enumerable: true,
      configurable: true,
    }
  }
);

```

```
});
```

9.9 API: property descriptors

The following tool methods use property descriptors:

- `Object.defineProperty(obj: object, key: string|symbol, propDesc: PropertyDescriptor): object` ^[ES5]

Creates or changes a property on `obj` whose key is `key` and whose attributes are specified via `propDesc`. Returns the modified object.

```
const obj = {};
const result = Object.defineProperty(
  obj, 'happy', {
    value: 'yes',
    writable: true,
    enumerable: true,
    configurable: true,
  });

// obj was returned and modified:
assert.equal(result, obj);
assert.deepEqual(obj, {
  happy: 'yes',
});
```

- `Object.defineProperties(obj: object, properties: {[k: string|symbol]: PropertyDescriptor}): object` ^[ES5]

The batch version of `Object.defineProperty()`. Each property `p` of the object `properties` specifies one property that is to be added to `obj`: The key of `p` specifies the key of the property, the value of `p` is a descriptor that specifies the attributes of the property.

```
const address1 = Object.defineProperties({}, {
  street: { value: 'Evergreen Terrace', enumerable: true },
  number: { value: 742, enumerable: true },
});
```

- `Object.create(proto: null|object, properties?: {[k: string|symbol]: PropertyDescriptor}): object` ^[ES5]

First, creates an object whose prototype is `proto`. Then, if the optional parameter `properties` has been provided, adds properties to it – in the same manner as `Object.defineProperties()`. Finally, returns the result. For example, the following code snippet produces the same result as the previous snippet:

```
const address2 = Object.create(Object.prototype, {
  street: { value: 'Evergreen Terrace', enumerable: true },
  number: { value: 742, enumerable: true },
});
```

```
});
assert.deepEqual(address1, address2);
```

- `Object.getOwnPropertyDescriptor(obj: object, key: string|symbol): undefined|PropertyDescriptor` ^[ES5]

Returns the descriptor of the own (non-inherited) property of `obj` whose key is `key`. If there is no such property, `undefined` is returned.

```
assert.deepEqual(
  Object.getOwnPropertyDescriptor(Object.prototype, 'toString'),
  {
    value: {}.toString,
    writable: true,
    enumerable: false,
    configurable: true,
  });
assert.equal(
  Object.getOwnPropertyDescriptor({}, 'toString'),
  undefined);
```

- `Object.getOwnPropertyDescriptors(obj: object): {[k: string|symbol]: PropertyDescriptor}` ^[ES2017]

Returns an object where each property key `'k'` of `obj` is mapped to the property descriptor for `obj.k`. The result can be used as input for `Object.defineProperty()` and `Object.create()`.

```
const propertyKey = Symbol('propertyKey');
const obj = {
  [propertyKey]: 'abc',
  get count() { return 123 },
};

const desc = Object.getOwnPropertyDescriptor.bind(Object);
assert.deepEqual(
  Object.getOwnPropertyDescriptors(obj),
  {
    [propertyKey]: {
      value: 'abc',
      writable: true,
      enumerable: true,
      configurable: true
    },
    count: {
      get: desc(obj, 'count').get, // (A)
      set: undefined,
      enumerable: true,
      configurable: true
    }
  });
```

Using `desc()` in line A is a work-around so that `.deepEqual()` works.

9.10 Further reading

The next three chapters provide more details on property attributes:

- §10 “Protecting objects from being changed”
- §11 “Properties: assignment vs. definition”
- §12 “Enumerability of properties”

Chapter 10

Protecting objects from being changed

Contents

10.1 Levels of protection: preventing extensions, sealing, freezing	99
10.2 Preventing extensions of objects	100
10.2.1 Checking whether an object is extensible	100
10.3 Sealing objects	101
10.3.1 Checking whether an object is sealed	102
10.4 Freezing objects	102
10.4.1 Checking whether an object is frozen	103
10.4.2 Freezing is shallow	103
10.4.3 Implementing deep freezing	103
10.5 Further reading	104

In this chapter, we'll look at how objects can be protected from being changed. Examples include: Preventing properties being added and preventing properties being changed.



Required knowledge: property attributes

For this chapter, you should be familiar with property attributes. If you aren't, check out §9 “[Property attributes: an introduction](#)”.

10.1 Levels of protection: preventing extensions, sealing, freezing

JavaScript has three levels of protecting objects:

- *Preventing extensions* makes it impossible to add new properties to an object. We can still delete and change properties, though.

- Method: `Object.preventExtensions(obj)`
- *Sealing* prevents extensions and makes all properties *unconfigurable* (roughly: we can't change how a property works anymore).
 - Method: `Object.seal(obj)`
- *Freezing* seals an object after making all of its properties non-writable. That is, the object is not extensible, all properties are read-only and there is no way to change that.
 - Method: `Object.freeze(obj)`

10.2 Preventing extensions of objects

`Object.preventExtensions<T>(obj: T): T`

This method works as follows:

- If `obj` is not an object, it returns it.
- Otherwise, it changes `obj` so that we can't add properties anymore and returns it.
- The type parameter `<T>` expresses that the result has the same type as the parameter.

Let's use `Object.preventExtensions()` in an example:

```
const obj = { first: 'Jane' };
Object.preventExtensions(obj);
assert.throws(
  () => obj.last = 'Doe',
  /^TypeError: Cannot add property last, object is not extensible$/);
```

We can still delete properties, though:

```
assert.deepEquals(
  Object.keys(obj), ['first']);
delete obj.first;
assert.deepEquals(
  Object.keys(obj), []);
```

10.2.1 Checking whether an object is extensible

`Object.isExtensible(obj: any): boolean`

Checks whether `obj` is extensible – for example:

```
> const obj = {};
> Object.isExtensible(obj)
true
> Object.preventExtensions(obj)
{}
> Object.isExtensible(obj)
false
```

10.3 Sealing objects

`Object.seal<T>(obj: T): T`

Description of this method:

- If `obj` isn't an object, it returns it.
- Otherwise, it prevents extensions of `obj`, makes all of its properties *unconfigurable*, and returns it. The properties being unconfigurable means that they can't be changed, anymore (except for their values): Read-only properties stay read-only, enumerable properties stay enumerable, etc.

The following example demonstrates that sealing makes the object non-extensible and its properties unconfigurable.

```
const obj = {
  first: 'Jane',
  last: 'Doe',
};

// Before sealing
assert.equal(Object.isExtensible(obj), true);
assert.deepEqual(
  Object.getOwnPropertyDescriptors(obj),
  {
    first: {
      value: 'Jane',
      writable: true,
      enumerable: true,
      configurable: true
    },
    last: {
      value: 'Doe',
      writable: true,
      enumerable: true,
      configurable: true
    }
  }
);

Object.seal(obj);

// After sealing
assert.equal(Object.isExtensible(obj), false);
assert.deepEqual(
  Object.getOwnPropertyDescriptors(obj),
  {
    first: {
      value: 'Jane',
      writable: true,
      enumerable: true,
```

```

    configurable: false
  },
  last: {
    value: 'Doe',
    writable: true,
    enumerable: true,
    configurable: false
  }
});

```

We can still change the the value of property `.first`:

```

obj.first = 'John';
assert.deepEqual(
  obj, {first: 'John', last: 'Doe'});

```

But we can't change its attributes:

```

assert.throws(
  () => Object.defineProperty(obj, 'first', { enumerable: false }),
  /^TypeError: Cannot redefine property: first$/);

```

10.3.1 Checking whether an object is sealed

```
Object.isSealed(obj: any): boolean
```

Checks whether `obj` is sealed – for example:

```

> const obj = {};
> Object.isSealed(obj)
false
> Object.seal(obj)
{}
> Object.isSealed(obj)
true

```

10.4 Freezing objects

```
Object.freeze<T>(obj: T): T;
```

- This method immediately returns `obj` if it isn't an object.
- Otherwise, it makes all properties non-writable, seals `obj`, and returns it. That is, `obj` is not extensible, all properties are read-only and there is no way to change that.

```

const point = { x: 17, y: -5 };
Object.freeze(point);

```

```

assert.throws(
  () => point.x = 2,
  /^TypeError: Cannot assign to read only property 'x'/);

```

```

assert.throws(
  () => Object.defineProperty(point, 'x', {enumerable: false}),
  /^TypeError: Cannot redefine property: x$/);

assert.throws(
  () => point.z = 4,
  /^TypeError: Cannot add property z, object is not extensible$/);

```

10.4.1 Checking whether an object is frozen

`Object.isFrozen(obj: any): boolean`

Checks whether obj is frozen – for example:

```

> const point = { x: 17, y: -5 };
> Object.isFrozen(point)
false
> Object.freeze(point)
{ x: 17, y: -5 }
> Object.isFrozen(point)
true

```

10.4.2 Freezing is shallow

`Object.freeze(obj)` only freezes obj and its properties. It does not freeze the values of those properties – for example:

```

const teacher = {
  name: 'Edna Krabappel',
  students: ['Bart'],
};
Object.freeze(teacher);

// We can't change own properties:
assert.throws(
  () => teacher.name = 'Elizabeth Hoover',
  /^TypeError: Cannot assign to read only property 'name'/);

// Alas, we can still change values of own properties:
teacher.students.push('Lisa');
assert.deepEqual(
  teacher, {
    name: 'Edna Krabappel',
    students: ['Bart', 'Lisa'],
  });

```

10.4.3 Implementing deep freezing

If we want deep freezing, we need to implement it ourselves:

```
function deepFreeze(value) {
  if (Array.isArray(value)) {
    for (const element of value) {
      deepFreeze(element);
    }
    Object.freeze(value);
  } else if (typeof value === 'object' && value !== null) {
    for (const v of Object.values(value)) {
      deepFreeze(v);
    }
    Object.freeze(value);
  } else {
    // Nothing to do: primitive values are already immutable
  }
  return value;
}
```

Revisiting the example from the previous section, we can check if `deepFreeze()` really freezes deeply:

```
const teacher = {
  name: 'Edna Krabappel',
  students: ['Bart'],
};
deepFreeze(teacher);

assert.throws(
  () => teacher.students.push('Lisa'),
  /^TypeError: Cannot add property 1, object is not extensible$/);
```

10.5 Further reading

- More information on patterns for preventing data structures from being changed: §15 “Immutable wrappers for collections”

Chapter 11

Properties: assignment vs. definition

Contents

11.1 Assignment vs. definition	106
11.1.1 Assignment	106
11.1.2 Definition	106
11.2 Assignment and definition in theory (optional)	106
11.2.1 Assigning to a property	107
11.2.2 Defining a property	109
11.3 Definition and assignment in practice	110
11.3.1 Only definition allows us to create a property with arbitrary attributes	110
11.3.2 The assignment operator does not change properties in prototypes	111
11.3.3 Assignment calls setters, definition doesn't	111
11.3.4 Inherited read-only properties prevent creating own properties via assignment	113
11.4 Which language constructs use definition, which assignment?	114
11.4.1 The properties of an object literal are added via definition	114
11.4.2 The assignment operator = always uses assignment	114
11.4.3 Public class fields are added via definition	115
11.5 Further reading and sources of this chapter	115

There are two ways of creating or changing a property prop of an object obj:

- *Assigning:* `obj.prop = true`
- *Defining:* `Object.defineProperty(obj, '', {value: true})`

This chapter explains how they work.

**Required knowledge: property attributes and property descriptors**

For this chapter, you should be familiar with property attributes and property descriptors. If you aren't, check out §9 “Property attributes: an introduction”.

11.1 Assignment vs. definition

11.1.1 Assignment

We use the assignment operator `=` to assign a value `value` to a property `.prop` of an object `obj`:

```
obj.prop = value
```

This operator works differently depending on what `.prop` looks like:

- Changing properties: If there is an own data property `.prop`, assignment changes its value to `value`.
- Invoking setters: If there is an own or inherited setter for `.prop`, assignment invokes that setter.
- Creating properties: If there is no own data property `.prop` and no own or inherited setter for it, assignment creates a new own data property.

That is, the main purpose of assignment is making changes. That's why it supports setters.

11.1.2 Definition

To define a property with the key `propKey` of an object `obj`, we use an operation such as the following method:

```
Object.defineProperty(obj, propKey, propDesc)
```

This method works differently depending on what the property looks like:

- Changing properties: If an own property with key `propKey` exists, defining changes its property attributes as specified by the property descriptor `propDesc` (if possible).
- Creating properties: Otherwise, defining creates an own property with the attributes specified by `propDesc` (if possible).

That is, the main purpose of definition is to create an own property (even if there is an inherited setter, which it ignores) and to change property attributes.

11.2 Assignment and definition in theory (optional)

**Property descriptors in the ECMAScript specification**

In specification operations, property descriptors are not JavaScript objects but *Records*, a spec-internal data structure that has *fields*. The keys of fields are written in double brackets. For example, `Desc.[[Configurable]]` accesses the field `.[[Configurable]]` of `Desc`. These records are translated to and from JavaScript objects when interacting with the outside world.

11.2.1 Assigning to a property

The actual work of assigning to a property is handled via [the following operation](#) in the ECMAScript specification:

```
OrdinarySetWithOwnDescriptor(0, P, V, Receiver, ownDesc)
```

These are the parameters:

- 0 is the object that is currently being visited.
- P is the key of the property that we are assigning to.
- V is the value we are assigning.
- Receiver is the object where the assignment started.
- ownDesc is the descriptor of 0[P] or null if that property doesn't exist.

The return value is a boolean that indicates whether or not the operation succeeded. As explained [later in this chapter](#), strict-mode assignment throws a `TypeError` if `OrdinarySetWithOwnDescriptor()` fails.

This is a high-level summary of the algorithm:

- It traverses the prototype chain of `Receiver` until it finds a property whose key is `P`. The traversal is done by calling `OrdinarySetWithOwnDescriptor()` recursively. During recursion, `0` changes and points to the object that is currently being visited, but `Receiver` stays the same.
- Depending on what the traversal finds, an own property is created in `Receiver` (where recursion started) or something else happens.

In more detail, this algorithm works as follows:

- If `ownDesc` is undefined, then we haven't yet found a property with key `P`:
 - If `0` has a prototype parent, then we return `parent.[[Set]](P, V, Receiver)`. This continues our search. The method call usually ends up invoking `OrdinarySetWithOwnDescriptor()` recursively.
 - Otherwise, our search for `P` has failed and we set `ownDesc` as follows:


```
{
    [[Value]]: undefined, [[Writable]]: true,
    [[Enumerable]]: true, [[Configurable]]: true
}
```

 With this `ownDesc`, the next if statement will create an own property in `Receiver`.
- If `ownDesc` specifies a data property, then we have found a property:
 - If `ownDesc.[[Writable]]` is false, return false. This means that any non-writable property `P` (own or inherited!) prevents assignment.
 - Let `existingDescriptor` be `Receiver.[[GetOwnProperty]](P)`. That is, retrieve the descriptor of the property where the assignment started. We now

have:

- * The current object `O` and the current property descriptor `ownDesc` on one hand.
- * The original object `Receiver` and the original property descriptor `existingDescriptor` on the other hand.
- If `existingDescriptor` is not undefined:
 - * (If we get here, then we are still at the beginning of the prototype chain – we only recurse if `Receiver` does not have a property `P`.)
 - * The following two if conditions should never be `true` because `ownDesc` and `existingDesc` should be equal:
 - If `existingDescriptor` specifies an accessor, return `false`.
 - If `existingDescriptor.[[Writable]]` is `false`, return `false`.
 - * Return `Receiver.[[DefineOwnProperty]](P, { [[Value]]: V })`. This internal method performs definition, which we use to change the value of property `Receiver[P]`. The definition algorithm is described in the next subsection.
- Else:
 - * (If we get here, then `Receiver` does not have an own property with key `P`.)
 - * Return `CreateDataProperty(Receiver, P, V)`. (This operation creates an own data property in its first argument.)
- (If we get here, then `ownDesc` describes an accessor property that is own or inherited.)
- Let `setter` be `ownDesc.[[Set]]`.
- If `setter` is undefined, return `false`.
- Perform `Call(setter, Receiver, «V»)`. `Call()` invokes the function object `setter` with this set to `Receiver` and the single parameter `V` (French quotes «» are used for lists in the specification).
- Return `true`.

11.2.1.1 How do we get from an assignment to `OrdinarySetWithOwnDescriptor()`?

Evaluating an assignment without destructuring involves the following steps:

- In the spec, evaluation starts in [the section on the runtime semantics of AssignmentExpression](#). This section handles providing names for anonymous functions, destructuring, and more.
- If there is no destructuring pattern, then `PutValue()` is used to make the assignment.
- For property assignments, `PutValue()` invokes the internal method `.[[Set]]()`.
- For ordinary objects, `.[[Set]]()` calls `OrdinarySet()` (which calls `OrdinarySetWithOwnDescriptor()`) and returns the result.

Notably, `PutValue()` throws a `TypeError` in strict mode if the result of `.[[Set]]()` is `false`.

11.2.2 Defining a property

The actual work of defining a property is handled via [the following operation](#) in the ECMAScript specification:

```
ValidateAndApplyPropertyDescriptor(0, P, extensible, Desc, current)
```

The parameters are:

- The object 0 where we want to define a property. There is a special validation-only mode where 0 is undefined. We are ignoring this mode here.
- The property key P of the property we want to define.
- extensible indicates if 0 is extensible.
- Desc is a property descriptor specifying the attributes we want the property to have.
- current contains the property descriptor of an own property 0[P] if it exists. Otherwise, current is undefined.

The result of the operation is a boolean that indicates if it succeeded. Failure can have different consequences. Some callers ignore the result. Others, such as `Object.defineProperty()`, throw an exception if the result is false.

This is a summary of the algorithm:

- If current is undefined, then property P does not currently exist and must be created.
 - If extensible is false, return false indicating that the property could not be added.
 - Otherwise, check Desc and create either a data property or an accessor property.
 - Return true.
- If Desc doesn't have any fields, return true indicating that the operation succeeded (because no changes had to be made).
- If current.[[Configurable]] is false:
 - (Desc is not allowed to change attributes other than value.)
 - If Desc.[[Configurable]] exists, it must have the same value as current.[[Configurable]]. If not, return false.
 - Same check: Desc.[[Enumerable]]
- Next, we *validate* the property descriptor Desc: Can the attributes described by current be changed to the values specified by Desc? If not, return false. If yes, go on.
 - If the descriptor is *generic* (with no attributes specific to data properties or accessor properties), then validation is successful and we can move on.
 - Else if one descriptor specifies a data property and the other an accessor property:
 - * The current property must be configurable (otherwise its attributes can't be changed as necessary). If not, false is returned.

- * Change the current property from a data property to an accessor property or vice versa. When doing so, the values of `[[Configurable]]` and `[[Enumerable]]` are preserved, all other attributes get **default values** (undefined for object-valued attributes, false for boolean-valued attributes).
- Else if both descriptors specify data properties:
 - * If both `current.[[Configurable]]` and `current.[[Writable]]` are false, then no changes are allowed and `Desc` and `current` must specify the same attributes:
 - (Due to `current.[[Configurable]]` being false, `Desc.[[Configurable]]` and `Desc.[[Enumerable]]` were already checked previously and have the correct values.)
 - If `Desc.[[Writable]]` exists and is true, then return false.
 - If `Desc.[[Value]]` exists and does not have the same value as `current.[[Value]]`, then return false.
 - There is nothing more to do. Return true indicating that the algorithm succeeded.
 - (Note that normally, we can't change any attributes of a non-configurable property other than its value. The one exception to this rule is that we can always go from writable to non-writable. This algorithm handles this exception correctly.)
 - Else (both descriptors specify accessor properties):
 - * If `current.[[Configurable]]` is false, then no changes are allowed and `Desc` and `current` must specify the same attributes:
 - (Due to `current.[[Configurable]]` being false, `Desc.[[Configurable]]` and `Desc.[[Enumerable]]` were already checked previously and have the correct values.)
 - If `Desc.[[Set]]` exists, it must have the same value as `current.[[Set]]`. If not, return false.
 - Same check: `Desc.[[Get]]`
 - There is nothing more to do. Return true indicating that the algorithm succeeded.
 - Set the attributes of the property with key `P` to the values specified by `Desc`. Due to validation, we can be sure that all of the changes are allowed.
 - Return true.

11.3 Definition and assignment in practice

This section describes some consequences of how property definition and assignment work.

11.3.1 Only definition allows us to create a property with arbitrary attributes

If we create an own property via assignment, it always creates properties whose attributes `writable`, `enumerable`, and `configurable` are all true.

```
const obj = {};
obj.dataProp = 'abc';
assert.deepEqual(
  Object.getOwnPropertyDescriptor(obj, 'dataProp'),
  {
    value: 'abc',
    writable: true,
    enumerable: true,
    configurable: true,
  });
```

Therefore, if we want to specify arbitrary attributes, we must use definition.

And while we can create getters and setters inside object literals, we can't add them later via assignment. Here, too, we need definition.

11.3.2 The assignment operator does not change properties in prototypes

Let us consider the following setup, where `obj` inherits the property `prop` from `proto`.

```
const proto = { prop: 'a' };
const obj = Object.create(proto);
```

We can't (destructively) change `proto.prop` by assigning to `obj.prop`. Doing so creates a new own property:

```
assert.deepEqual(
  Object.keys(obj), []);

obj.prop = 'b';

// The assignment worked:
assert.equal(obj.prop, 'b');

// But we created an own property and overrode proto.prop,
// we did not change it:
assert.deepEqual(
  Object.keys(obj), ['prop']);
assert.equal(proto.prop, 'a');
```

The rationale for this behavior is as follows: Prototypes can have properties whose values are shared by all of their descendants. If we want to change such a property in only one descendant, we must do so non-destructively, via overriding. Then the change does not affect the other descendants.

11.3.3 Assignment calls setters, definition doesn't

What is the difference between defining the property `.prop` of `obj` versus assigning to it?

If we define, then our intention is to either create or change an own (non-inherited) property of `obj`. Therefore, definition ignores the inherited setter for `.prop` in the following

example:

```
let setterWasCalled = false;
const proto = {
  get prop() {
    return 'protoGetter';
  },
  set prop(x) {
    setterWasCalled = true;
  },
};
const obj = Object.create(proto);

assert.equal(obj.prop, 'protoGetter');

// Defining obj.prop:
Object.defineProperty(
  obj, 'prop', { value: 'objData' });
assert.equal(setterWasCalled, false);

// We have overridden the getter:
assert.equal(obj.prop, 'objData');
```

If, instead, we assign to `.prop`, then our intention is often to change something that already exists and that change should be handled by the setter:

```
let setterWasCalled = false;
const proto = {
  get prop() {
    return 'protoGetter';
  },
  set prop(x) {
    setterWasCalled = true;
  },
};
const obj = Object.create(proto);

assert.equal(obj.prop, 'protoGetter');

// Assigning to obj.prop:
obj.prop = 'objData';
assert.equal(setterWasCalled, true);

// The getter still active:
assert.equal(obj.prop, 'protoGetter');
```

11.3.4 Inherited read-only properties prevent creating own properties via assignment

What happens if `.prop` is read-only in a prototype?

```
const proto = Object.defineProperty(
  {}, 'prop', {
    value: 'protoValue',
    writable: false,
  });
```

In any object that inherits the read-only `.prop` from `proto`, we can't use assignment to create an own property with the same key – for example:

```
const obj = Object.create(proto);
assert.throws(
  () => obj.prop = 'objValue',
  /TypeError: Cannot assign to read only property 'prop'/);
```

Why can't we assign? The rationale is that overriding an inherited property by creating an own property can be seen as **non-destructively changing** the inherited property. Arguably, if a property is non-writable, we shouldn't be able to do that.

However, defining `.prop` still works and lets us override:

```
Object.defineProperty(
  obj, 'prop', { value: 'objValue' });
assert.equal(obj.prop, 'objValue');
```

Accessor properties that don't have a setter are also considered to be read-only:

```
const proto = {
  get prop() {
    return 'protoValue';
  }
};
const obj = Object.create(proto);
assert.throws(
  () => obj.prop = 'objValue',
  /TypeError: Cannot set property prop of #<Object> which has only a getter$/);
```



The “override mistake”: pros and cons

The fact that read-only properties prevent assignment earlier in the prototype chain, has been given the name *override mistake*:

- It was introduced in ECMAScript 5.1.
- On one hand, this behavior is consistent with how prototypal inheritance and setters work. (So, arguably, it is *not* a mistake.)
- On the other hand, with the behavior, deep-freezing the global object causes unwanted side-effects.

- There was an attempt to change the behavior, but that broke the library Lodash and was abandoned ([pull request on GitHub](#)).
- Background knowledge:
 - [Pull request on GitHub](#)
 - [Wiki page on ECMAScript.org](#) (archived)

11.4 Which language constructs use definition, which assignment?

In this section, we examine where the language uses definition and where it uses assignment. We detect which operation is used by tracking whether or not inherited setters are called. See §11.3.3 “Assignment calls setters, definition doesn’t” for more information.

11.4.1 The properties of an object literal are added via definition

When we create properties via an object literal, JavaScript always uses definition (and therefore never calls inherited setters):

```
let lastSetterArgument;
const proto = {
  set prop(x) {
    lastSetterArgument = x;
  },
};
const obj = {
  __proto__: proto,
  prop: 'abc',
};
assert.equal(lastSetterArgument, undefined);
```

11.4.2 The assignment operator = always uses assignment

The assignment operator = always uses assignment to create or change properties.

```
let lastSetterArgument;
const proto = {
  set prop(x) {
    lastSetterArgument = x;
  },
};
const obj = Object.create(proto);

// Normal assignment:
obj.prop = 'abc';
assert.equal(lastSetterArgument, 'abc');

// Assigning via destructuring:
```

```
[obj.prop] = ['def'];
assert.equal(lastSetterArgument, 'def');
```

11.4.3 Public class fields are added via definition

Alas, even though public class fields have the same syntax as assignment, they do *not* use assignment to create properties, they use definition (like properties in object literals):

```
let lastSetterArgument1;
let lastSetterArgument2;
class A {
  set prop1(x) {
    lastSetterArgument1 = x;
  }
  set prop2(x) {
    lastSetterArgument2 = x;
  }
}
class B extends A {
  prop1 = 'one';
  constructor() {
    super();
    this.prop2 = 'two';
  }
}
new B();

// The public class field uses definition:
assert.equal(lastSetterArgument1, undefined);
// Inside the constructor, we trigger assignment:
assert.equal(lastSetterArgument2, 'two');
```

11.5 Further reading and sources of this chapter

- [Section “Prototype chains”](#) in “JavaScript for impatient programmers”
- [Email by Allen Wirfs-Brock to the es-discuss mailing list](#): “The distinction between assignment and definition [...] was not very important when all ES had was data properties and there was no way for ES code to manipulate property attributes.” [That changed with ECMAScript 5.]

Chapter 12

Enumerability of properties

Contents

12.1 How enumerability affects property-iterating constructs	117
12.1.1 Operations that only consider enumerable properties	118
12.1.2 Operations that consider both enumerable and non- enumerable properties	120
12.1.3 Naming rules for introspective operations	121
12.2 The enumerability of pre-defined and created properties	122
12.3 Use cases for enumerability	122
12.3.1 Use case: Hiding properties from the for-in loop	123
12.3.2 Use case: Marking properties as not to be copied	124
12.3.3 Marking properties as private	127
12.3.4 Hiding own properties from JSON.stringify()	127
12.4 Conclusion	128

Enumerability is an *attribute* of object properties. In this chapter, we take a closer look at how it is used and how it influences operations such as `Object.keys()` and `Object.assign()`.



Required knowledge: property attributes

For this chapter, you should be familiar with property attributes. If you aren't, check out §9 “[Property attributes: an introduction](#)”.

12.1 How enumerability affects property-iterating constructs

To demonstrate how various operations are affected by enumerability, we use the following object `obj` whose prototype is `proto`.

```

const protoEnumSymbolKey = Symbol('protoEnumSymbolKey');
const protoNonEnumSymbolKey = Symbol('protoNonEnumSymbolKey');
const proto = Object.defineProperties({}, {
  protoEnumStringKey: {
    value: 'protoEnumStringKeyValue',
    enumerable: true,
  },
  [protoEnumSymbolKey]: {
    value: 'protoEnumSymbolKeyValue',
    enumerable: true,
  },
  protoNonEnumStringKey: {
    value: 'protoNonEnumStringKeyValue',
    enumerable: false,
  },
  [protoNonEnumSymbolKey]: {
    value: 'protoNonEnumSymbolKeyValue',
    enumerable: false,
  },
});

const objEnumSymbolKey = Symbol('objEnumSymbolKey');
const objNonEnumSymbolKey = Symbol('objNonEnumSymbolKey');
const obj = Object.create(proto, {
  objEnumStringKey: {
    value: 'objEnumStringKeyValue',
    enumerable: true,
  },
  [objEnumSymbolKey]: {
    value: 'objEnumSymbolKeyValue',
    enumerable: true,
  },
  objNonEnumStringKey: {
    value: 'objNonEnumStringKeyValue',
    enumerable: false,
  },
  [objNonEnumSymbolKey]: {
    value: 'objNonEnumSymbolKeyValue',
    enumerable: false,
  },
});

```

12.1.1 Operations that only consider enumerable properties

Table 12.1: Operations that ignore non-enumerable properties.

Operation		String keys	Symbol keys	Inherited
<code>Object.keys()</code>	ES5	✓	✗	✗
<code>Object.values()</code>	ES2017	✓	✗	✗
<code>Object.entries()</code>	ES2017	✓	✗	✗
Spreading <code>{...x}</code>	ES2018	✓	✓	✗
<code>Object.assign()</code>	ES6	✓	✓	✗
<code>JSON.stringify()</code>	ES5	✓	✗	✗
<code>for-in</code>	ES1	✓	✗	✓

The following operations (summarized in tbl. 12.1) only consider enumerable properties:

- `Object.keys()` ^[ES5] returns the keys of enumerable own string-keyed properties.


```
> Object.keys(obj)
[ 'objEnumStringKey' ]
```
- `Object.values()` ^[ES2017] returns the values of enumerable own string-keyed properties.


```
> Object.values(obj)
[ 'objEnumStringKeyValue' ]
```
- `Object.entries()` ^[ES2017] returns key-value pairs for enumerable own string-keyed properties. (Note that `Object.fromEntries()` does accept symbols as keys, but only creates enumerable properties.)


```
> Object.entries(obj)
[ [ 'objEnumStringKey', 'objEnumStringKeyValue' ] ]
```
- Spreading into object literals ^[ES2018] only considers own enumerable properties (with string keys or symbol keys).


```
> const copy = {...obj};
> Reflect.ownKeys(copy)
[ 'objEnumStringKey', objEnumSymbolKey ]
```
- `Object.assign()` ^[ES6] only copies enumerable own properties (with either string keys or symbol keys).


```
> const copy = Object.assign({}, obj);
> Reflect.ownKeys(copy)
[ 'objEnumStringKey', objEnumSymbolKey ]
```
- `JSON.stringify()` ^[ES5] only stringifies enumerable own properties with string keys.


```
> JSON.stringify(obj)
'{"objEnumStringKey":"objEnumStringKeyValue"}'
```
- `for-in` loop ^[ES1] traverses the keys of own and inherited enumerable string-keyed properties.

```
const propKeys = [];
for (const propKey in obj) {
  propKeys.push(propKey);
}
assert.deepEqual(
  propKeys, [ 'objEnumStringKey', 'protoEnumStringKey' ] );
```

for-in is the only built-in operation where enumerability matters for inherited properties. All other operations only work with own properties.

12.1.2 Operations that consider both enumerable and non-enumerable properties

Table 12.2: Operations that consider both enumerable and non-enumerable properties.

Operation		Str. keys	Sym. keys	Inherited
Object.getOwnPropertyNames()	ES5	✓	✗	✗
Object.getOwnPropertySymbols()	ES6	✗	✓	✗
Reflect.ownKeys()	ES6	✓	✓	✗
Object.getOwnPropertyDescriptors()	ES2017	✓	✓	✗

The following operations (summarized in tbl. 12.2) consider both enumerable and non-enumerable properties:

- Object.getOwnPropertyNames() ^[ES5] lists the keys of all own string-keyed properties.

```
> Object.getOwnPropertyNames(obj)
[ 'objEnumStringKey', 'objNonEnumStringKey' ]
```
- Object.getOwnPropertySymbols() ^[ES6] lists the keys of all own symbol-keyed properties.

```
> Object.getOwnPropertySymbols(obj)
[ objEnumSymbolKey, objNonEnumSymbolKey ]
```
- Reflect.ownKeys() ^[ES6] lists the keys of all own properties.

```
> Reflect.ownKeys(obj)
[
  'objEnumStringKey',
  'objNonEnumStringKey',
  objEnumSymbolKey,
  objNonEnumSymbolKey
]
```
- Object.getOwnPropertyDescriptors() ^[ES2017] lists the property descriptors of all own properties.

```

> Object.getOwnPropertyDescriptors(obj)
{
  objEnumStringKey: {
    value: 'objEnumStringValue',
    writable: false,
    enumerable: true,
    configurable: false
  },
  objNonEnumStringKey: {
    value: 'objNonEnumStringValue',
    writable: false,
    enumerable: false,
    configurable: false
  },
  [objEnumSymbolKey]: {
    value: 'objEnumSymbolKeyValue',
    writable: false,
    enumerable: true,
    configurable: false
  },
  [objNonEnumSymbolKey]: {
    value: 'objNonEnumSymbolKeyValue',
    writable: false,
    enumerable: false,
    configurable: false
  }
}

```

12.1.3 Naming rules for introspective operations

Introspection enables a program to examine the structure of values at runtime. It is *metaprogramming*: Normal programming is about writing programs; metaprogramming is about examining and/or changing programs.

In JavaScript, common introspective operations have short names, while rarely used operations have long names. Ignoring non-enumerable properties is the norm, which is why operations that do that have short names and operations that don't, long names:

- `Object.keys()` ignores non-enumerable properties.
- `Object.getOwnPropertyNames()` lists the string keys of all own properties.

However, `Reflect` methods (such as `Reflect.ownKeys()`) deviate from this rule because `Reflect` provides operations that are more “meta” and related to Proxies.

Additionally, the following distinction is made (since ES6, which introduced symbols):

- *Property keys* are either strings or symbols.
- *Property names* are property keys that are strings.
- *Property symbols* are property keys that are symbols.

Therefore, a better name for `Object.keys()` would now be `Object.names()`.

12.2 The enumerability of pre-defined and created properties

In this section, we'll abbreviate `Object.getOwnPropertyDescriptor()` like this:

```
const desc = Object.getOwnPropertyDescriptor.bind(Object);
```

Most data properties are created with the following attributes:

```
{
  writable: true,
  enumerable: false,
  configurable: true,
}
```

That includes:

- Assignment
- Object literals
- Public class fields
- `Object.fromEntries()`

The most important non-enumerable properties are:

- Prototype properties of built-in classes

```
> desc(Object.prototype, 'toString').enumerable
false
```

- Prototype properties created via user-defined classes

```
> desc(class {foo() {}}.prototype, 'foo').enumerable
false
```

- Property `.length` of Arrays:

```
> Object.getOwnPropertyDescriptor([], 'length')
{ value: 0, writable: true, enumerable: false, configurable: false }
```

- Property `.length` of strings (note that all properties of primitive values are read-only):

```
> Object.getOwnPropertyDescriptor('', 'length')
{ value: 0, writable: false, enumerable: false, configurable: false }
```

We'll look at the use cases for enumerability next, which will tell us why some properties are enumerable and others aren't.

12.3 Use cases for enumerability

Enumerability is an inconsistent feature. It does have use cases, but there is always some kind of caveat. In this section, we look at the use cases and the caveats.

12.3.1 Use case: Hiding properties from the for-in loop

The for-in loop traverses *all* enumerable string-keyed properties of an object, own and inherited ones. Therefore, the attribute `enumerable` is used to hide properties that should not be traversed. That was the reason for introducing enumerability in ECMAScript 1.

In general, it is best to avoid for-in. The next two subsections explain why. The following function will help us demonstrate how for-in works.

```
function listPropertiesViaForIn(obj) {  
  const result = [];  
  for (const key in obj) {  
    result.push(key);  
  }  
  return result;  
}
```

12.3.1.1 The caveats of using for-in for objects

for-in iterates over all properties, including inherited ones:

```
const proto = {enumerableProtoProp: 1};  
const obj = {  
  __proto__: proto,  
  enumerableObjProp: 2,  
};  
assert.deepEqual(  
  listPropertiesViaForIn(obj),  
  ['enumerableObjProp', 'enumerableProtoProp']);
```

With normal plain objects, for-in doesn't see inherited methods such as `Object.prototype.toString()` because they are all non-enumerable:

```
const obj = {};  
assert.deepEqual(  
  listPropertiesViaForIn(obj),  
  []);
```

In user-defined classes, all inherited properties are also non-enumerable and therefore ignored:

```
class Person {  
  constructor(first, last) {  
    this.first = first;  
    this.last = last;  
  }  
  getName() {  
    return this.first + ' ' + this.last;  
  }  
}  
  
const jane = new Person('Jane', 'Doe');  
assert.deepEqual(  
  listPropertiesViaForIn(jane),  
  []);
```

```
listPropertiesViaForIn(jane),
['first', 'last']);
```

Conclusion: In objects, `for-in` considers inherited properties and we usually want to ignore those. Then it is better to combine a `for-of` loop with `Object.keys()`, `Object.entries()`, etc.

12.3.1.2 The caveats of using `for-in` for Arrays

The own property `.length` is non-enumerable in Arrays and strings and therefore ignored by `for-in`:

```
> listPropertiesViaForIn(['a', 'b'])
[ '0', '1' ]
> listPropertiesViaForIn('ab')
[ '0', '1' ]
```

However, it is generally not safe to use `for-in` to iterate over the indices of an Array because it considers both inherited and own properties that are not indices. The following example demonstrate what happens if an Array has an own non-index property:

```
const arr1 = ['a', 'b'];
assert.deepEqual(
  listPropertiesViaForIn(arr1),
  ['0', '1']);

const arr2 = ['a', 'b'];
arr2.nonIndexProp = 'yes';
assert.deepEqual(
  listPropertiesViaForIn(arr2),
  ['0', '1', 'nonIndexProp']);
```

Conclusion: `for-in` should not be used for iterating over the indices of an Array because it considers both index properties and non-index properties:

- If you are interested in the keys of an Array, use the Array method `.keys()`:

```
> [...['a', 'b', 'c'].keys()]
[ 0, 1, 2 ]
```
- If you want to iterate over the elements of an Array, use a `for-of` loop, which has the added benefit of also working with other iterable data structures.

12.3.2 Use case: Marking properties as not to be copied

By making properties non-enumerable, we can hide them from some copying operations. Let us first examine two historical copying operations before moving on to more modern copying operations.

12.3.2.1 Historical copying operation: Prototype's `Object.extend()`

[Prototype](#) is a JavaScript framework that was created by Sam Stephenson in February 2005 as part of the foundation for Ajax support in Ruby on Rails.

Prototype's `Object.extend(destination, source)` copies all enumerable own and inherited properties of source into own properties of destination. It is implemented as follows:

```
function extend(destination, source) {
  for (var property in source)
    destination[property] = source[property];
  return destination;
}
```

If we use `Object.extend()` with an object, we can see that it copies inherited properties into own properties and ignores non-enumerable properties (it also ignores symbol-keyed properties). All of this is due to how `for-in` works.

```
const proto = Object.defineProperties({}, {
  enumProtoProp: {
    value: 1,
    enumerable: true,
  },
  nonEnumProtoProp: {
    value: 2,
    enumerable: false,
  },
});
const obj = Object.create(proto, {
  enumObjProp: {
    value: 3,
    enumerable: true,
  },
  nonEnumObjProp: {
    value: 4,
    enumerable: false,
  },
});

assert.deepEqual(
  extend({}, obj),
  {enumObjProp: 3, enumProtoProp: 1});
```

12.3.2.2 Historical copying operation: jQuery's `$.extend()`

jQuery's `$.extend(target, source1, source2, ...)` works similar to `Object.extend()`:

- It copies all enumerable own and inherited properties of source1 into own properties of target.
- Then it does the same with source2.
- Etc.

12.3.2.3 The downsides of enumerability-driven copying

Basing copying on enumerability has several downsides:

- While enumerability is useful for hiding inherited properties, it is mainly used in this manner because we usually only want to copy own properties into own properties. The same effect can be better achieved by ignoring inherited properties.
- Which properties to copy often depends on the task at hand; it rarely makes sense to have a single flag for all use cases. A better choice is to provide a copying operation with a *predicate* (a callback that returns a boolean) that tells it when to ignore properties.
- Enumerability conveniently hides the own property `.length` of Arrays when copying. But that is an incredibly rare exceptional case: a magic property that both influences sibling properties and is influenced by them. If we implement this kind of magic ourselves, we will use (inherited) getters and/or setters, not (own) data properties.

12.3.2.4 `Object.assign()` ^[ES5]

In ES6, `Object.assign(target, source_1, source_2, ...)` can be used to merge the sources into the target. All own enumerable properties of the sources are considered (with either string keys or symbol keys). `Object.assign()` uses a “get” operation to read a value from a source and a “set” operation to write a value to the target.

With regard to enumerability, `Object.assign()` continues the tradition of `Object.extend()` and `$.extend()`. [Quoting Yehuda Katz](#):

`Object.assign` would pave the cowpath of all of the `extend()` APIs already in circulation. We thought the precedent of not copying enumerable methods in those cases was enough reason for `Object.assign` to have this behavior.

In other words: `Object.assign()` was created with an upgrade path from `$.extend()` (and similar) in mind. Its approach is cleaner than `$.extend`’s because it ignores inherited properties.

12.3.2.5 A rare example of non-enumerability being useful when copying

Cases where non-enumerability helps are few. One rare example is [a recent issue that the library fs-extra had](#):

- The built-in Node.js module `fs` has a property `.promises` that contains an object with a Promise-based version of the `fs` API. At the time of the issue, reading `.promise` led to the following warning being logged to the console:

```
ExperimentalWarning: The fs.promises API is experimental
```

- In addition to providing its own functionality, `fs-extra` also re-exports everything that’s in `fs`. For CommonJS modules, that means copying all properties of `fs` into the `module.exports` of `fs-extra` (via `Object.assign()`). And when `fs-extra` did that, it triggered the warning. That was confusing because it happened every time `fs-extra` was loaded.

- A quick fix was to make property `fs.promises` non-enumerable. Afterwards, `fs-extra` ignored it.

12.3.3 Marking properties as private

If we make a property non-enumerable, it can't be seen by `Object.keys()`, the `for-in` loop, etc., anymore. With regard to those mechanisms, the property is private.

However, there are several problems with this approach:

- When copying an object, we normally want to copy private properties. That clashes with making properties non-enumerable that shouldn't be copied (see previous section).
- The property isn't really private. Getting, setting and several other mechanisms make no distinction between enumerable and non-enumerable properties.
- When working with code either as source or interactively, we can't immediately see whether a property is enumerable or not. A naming convention (such as prefixing property names with an underscore) is easier to discover.
- We can't use enumerability to distinguish between public and private methods because methods in prototypes are non-enumerable by default.

12.3.4 Hiding own properties from `JSON.stringify()`

`JSON.stringify()` does not include properties in its output that are non-enumerable. We can therefore use enumerability to determine which own properties should be exported to JSON. This use case is similar to the previous one, marking properties as private. But it is also different because this is more about exporting and slightly different considerations apply. For example: Can an object be completely reconstructed from JSON?

As an alternative to enumerability, an object can implement the method `.toJSON()` and `JSON.stringify()` stringifies whatever that method returns, instead of the object itself. The next example demonstrates how that works.

```
class Point {
  static fromJSON(json) {
    return new Point(json[0], json[1]);
  }
  constructor(x, y) {
    this.x = x;
    this.y = y;
  }
  toJSON() {
    return [this.x, this.y];
  }
}
assert.equal(
  JSON.stringify(new Point(8, -3)),
  '[8,-3]'
);
```

I find `toJSON()` cleaner than enumerability. It also gives us more freedom w.r.t. what the storage format should look like.

12.4 Conclusion

We have seen that almost all applications for non-enumerability are work-arounds that now have other and better solutions.

For our own code, we can usually pretend that enumerability doesn't exist:

- Creating properties via object literals and assignment always creates enumerable properties.
- Prototype properties created via classes are always non-enumerable.

That is, we automatically follow best practices.

Part V

OOP: techniques

Chapter 13

Techniques for instantiating classes

Contents

13.1 The problem: initializing a property asynchronously	131
13.2 Solution: Promise-based constructor	132
13.2.1 Using an immediately-invoked asynchronous arrow function	133
13.3 Solution: static factory method	133
13.3.1 Improvement: private constructor via secret token	134
13.3.2 Improvement: constructor throws, factory method borrows the class prototype	135
13.3.3 Improvement: instances are inactive by default, activated by factory method	135
13.3.4 Variant: separate factory function	136
13.4 Subclassing a Promise-based constructor (optional)	137
13.5 Conclusion	138
13.6 Further reading	138

In this chapter, we examine several approaches for creating instances of classes: Constructors, factory functions, etc. We do so by solving one concrete problem several times. The focus of this chapter is on classes, which is why alternatives to classes are ignored.

13.1 The problem: initializing a property asynchronously

The following container class is supposed to receive the contents of its property `.data` asynchronously. This is our first attempt:

```
class DataContainer {
  #data; // (A)
  constructor() {
    Promise.resolve('downloaded')
```

```

        .then(data => this.#data = data); // (B)
    }
    getData() {
        return 'DATA: '+this.#data; // (C)
    }
}

```

Key issue of this code: Property `.data` is initially undefined.

```

const dc = new DataContainer();
assert.equal(dc.getData(), 'DATA: undefined');
setTimeout(() => assert.equal(
    dc.getData(), 'DATA: downloaded'), 0);

```

In line A, we declare the **private field** `#data` that we use in line B and line C.

The Promise inside the constructor of `DataContainer` is settled asynchronously, which is why we can only see the final value of `.data` if we finish the current task and start a new one, via `setTimeout()`. In other words, the instance of `DataContainer` is not completely initialized, yet, when we first see it.

13.2 Solution: Promise-based constructor

What if we delay access to the instance of `DataContainer` until it is fully initialized? We can achieve that by returning a Promise from the constructor. By default, a constructor returns a new instance of the class that it is part of. We can override that if we explicitly return an object:

```

class DataContainer {
    #data;
    constructor() {
        return Promise.resolve('downloaded')
            .then(data => {
                this.#data = data;
                return this; // (A)
            });
    }
    getData() {
        return 'DATA: '+this.#data;
    }
}
new DataContainer()
    .then(dc => assert.equal( // (B)
        dc.getData(), 'DATA: downloaded'));

```

Now we have to wait until we can access our instance (line B). It is passed on to us after the data is “downloaded” (line A). There are two possible sources of errors in this code:

- The download may fail and produce a rejection.
- An exception may be thrown in the body of the first `.then()` callback.

In either case, the errors become rejections of the Promise that is returned from the constructor.

Pros and cons:

- A benefit of this approach is that we can only access the instance once it is fully initialized. And there is no other way of creating instances of `DataContainer`.
- A disadvantage is that it may be surprising to have a constructor return a Promise instead of an instance.

13.2.1 Using an immediately-invoked asynchronous arrow function

Instead of using the Promise API directly to create the Promise that is returned from the constructor, we can also use an asynchronous arrow function that [we invoke immediately](#):

```
constructor() {
  return (async () => {
    this.#data = await Promise.resolve('downloaded');
    return this;
  })();
}
```

13.3 Solution: static factory method

A *static factory method* of a class `C` creates instances of `C` and is an alternative to using `new C()`. Common names for static factory methods in JavaScript:

- `.create()`: Creates a new instance. Example: `Object.create()`
- `.from()`: Creates a new instance based on a different object, by copying and/or converting it. Example: `Array.from()`
- `.of()`: Creates a new instance by assembling values specified via arguments. Example: `Array.of()`

In the following example, `DataContainer.create()` is a static factory method. It returns Promises for instances of `DataContainer`:

```
class DataContainer {
  #data;
  static async create() {
    const data = await Promise.resolve('downloaded');
    return new this(data);
  }
  constructor(data) {
    this.#data = data;
  }
  getData() {
    return 'DATA: '+this.#data;
  }
}
```

```
DataContainer.create()
  .then(dc => assert.equal(
    dc.getData(), 'DATA: downloaded'));

```

This time, all asynchronous functionality is contained in `.create()`, which enables the rest of the class to be completely synchronous and therefore simpler.

Pros and cons:

- A benefit of this approach is that the constructor becomes simple.
- A disadvantage of this approach is that it's now possible to create instances that are incorrectly set up, via `new DataContainer()`.

13.3.1 Improvement: private constructor via secret token

If we want to ensure that instances are always correctly set up, we must ensure that only `DataContainer.create()` can invoke the constructor of `DataContainer`. We can achieve that via a secret token:

```
const secretToken = Symbol('secretToken');
class DataContainer {
  #data;
  static async create() {
    const data = await Promise.resolve('downloaded');
    return new this(secretToken, data);
  }
  constructor(token, data) {
    if (token !== secretToken) {
      throw new Error('Constructor is private');
    }
    this.#data = data;
  }
  getData() {
    return 'DATA: '+this.#data;
  }
}
DataContainer.create()
  .then(dc => assert.equal(
    dc.getData(), 'DATA: downloaded'));

```

If `secretToken` and `DataContainer` reside in the same module and only the latter is exported, then outside parties don't have access to `secretToken` and therefore can't create instances of `DataContainer`.

Pros and cons:

- Benefit: safe and straightforward.
- Disadvantage: slightly verbose.

13.3.2 Improvement: constructor throws, factory method borrows the class prototype

The following variant of our solution disables the constructor of `DataContainer` and uses a trick to create instances of it another way (line A):

```
class DataContainer {
  static async create() {
    const data = await Promise.resolve('downloaded');
    return Object.create(this.prototype)._init(data); // (A)
  }
  constructor() {
    throw new Error('Constructor is private');
  }
  _init(data) {
    this._data = data;
    return this;
  }
  getData() {
    return 'DATA: ' + this._data;
  }
}
DataContainer.create()
  .then(dc => {
    assert.equal(dc instanceof DataContainer, true); // (B)
    assert.equal(
      dc.getData(), 'DATA: downloaded');
  });
```

Internally, an instance of `DataContainer` is any object whose prototype is `DataContainer.prototype`. That's why we can create instances via `Object.create()` (line A) and that's why `instanceof` works in line B.

Pros and cons:

- Benefit: elegant; `instanceof` works.
- Disadvantages:
 - Creating instances is not completely prevented. To be fair, though, the work-around via `Object.create()` can also be used for our previous solutions.
 - We can't use [private fields](#) and [private methods](#) in `DataContainer`, because those are only set up correctly for instances that were created via the constructor.

13.3.3 Improvement: instances are inactive by default, activated by factory method

Another, more verbose variant is that, by default, instances are switched off via the flag `.#active`. The initialization method `._init()` that switches them on cannot be accessed externally, but `Data.container()` can invoke it:

```

class DataContainer {
  #data;
  static async create() {
    const data = await Promise.resolve('downloaded');
    return new this().#init(data);
  }

  #active = false;
  constructor() {
  }
  #init(data) {
    this.#active = true;
    this.#data = data;
    return this;
  }
  getData() {
    this.#check();
    return 'DATA: ' + this.#data;
  }
  #check() {
    if (!this.#active) {
      throw new Error('Not created by factory');
    }
  }
}
DataContainer.create()
  .then(dc => assert.equal(
    dc.getData(), 'DATA: downloaded'));

```

The flag `#active` is enforced via the private method `#check()` which must be invoked at the beginning of every method.

The major downside of this solution is its verbosity. There is also a risk of forgetting to invoke `#check()` in each method.

13.3.4 Variant: separate factory function

For completeness sake, I'll show another variant: Instead of using a static method as a factory you can also use a separate stand-alone function.

```

const secretToken = Symbol('secretToken');
class DataContainer {
  #data;
  constructor(token, data) {
    if (token !== secretToken) {
      throw new Error('Constructor is private');
    }
    this.#data = data;
  }
  getData() {

```

```

        return 'DATA: '+this.#data;
    }
}

async function createDataContainer() {
    const data = await Promise.resolve('downloaded');
    return new DataContainer(secretToken, data);
}

createDataContainer()
    .then(dc => assert.equal(
        dc.getData(), 'DATA: downloaded'));

```

Stand-alone functions as factories are occasionally useful, but in this case, I prefer a static method:

- The stand-alone function can't access private members of `DataContainer`.
- I prefer the way `DataContainer.create()` looks.

13.4 Subclassing a Promise-based constructor (optional)

In general, subclassing is something to use sparingly.

With a separate factory function, it is relatively easy to extend `DataContainer`.

Alas, extending the class with the Promise-based constructor leads to severe limitations. In the following example, we subclass `DataContainer`. The subclass `SubDataContainer` has its own private field `#moreData` that it initializes asynchronously by hooking into the Promise returned by the constructor of its superclass.

```

class DataContainer {
    #data;
    constructor() {
        return Promise.resolve('downloaded')
            .then(data => {
                this.#data = data;
                return this; // (A)
            });
    }
    getData() {
        return 'DATA: '+this.#data;
    }
}

class SubDataContainer extends DataContainer {
    #moreData;
    constructor() {
        super();
        const promise = this;
        return promise

```

```

        .then(_this => {
            return Promise.resolve('more')
                .then(moreData => {
                    _this.#moreData = moreData;
                    return _this;
                });
        });
    }
    getData() {
        return super.getData() + ', ' + this.#moreData;
    }
}

```

Alas, we can't instantiate this class:

```

assert.rejects(
    () => new SubDataContainer(),
    {
        name: 'TypeError',
        message: 'Cannot write private member #moreData ' +
            'to an object whose class did not declare it',
    }
);

```

Why the failure? A constructor always adds its private fields to its `this`. However, here, `this` in the subconstructor is the Promise returned by the superconstructor (and not the instance of `SubDataContainer` delivered via the Promise).

However, this approach still works if `SubDataContainer` does not have any private fields.

13.5 Conclusion

For the scenario examined in this chapter, I prefer either a Promise-based constructor or a static factory method plus a private constructor via a secret token.

However, the other techniques presented here can still be useful in other scenarios.

13.6 Further reading

- Asynchronous programming:
 - [Chapter “Promises for asynchronous programming”](#) in “JavaScript for impatient programmers”
 - [Chapter “Async functions”](#) in “JavaScript for impatient programmers”
 - [Section “Immediately invoked async arrow functions”](#) in “JavaScript for impatient programmers”
- OOP:
 - [Chapter “Prototype chains and classes”](#) in “JavaScript for impatient programmers”
 - [Blog post “ES proposal: private class fields”](#)

- [Blog post “ES proposal: private methods and accessors in JavaScript classes”](#)

Chapter 14

Copying instances of classes: .clone() vs. copy constructors

Contents

14.1 .clone() methods	141
14.2 Static factory methods	142
14.3 Acknowledgements	143

In this chapter, we look at two techniques for implementing copying for class instances:

- .clone() methods
- So-called *copy constructors*, constructors that receive another instance of the current class and use it to initialize the current instance.

14.1 .clone() methods

This technique introduces one method .clone() per class whose instances are to be copied. It returns a deep copy of this. The following example shows three classes that can be cloned.

```
class Point {
    constructor(x, y) {
        this.x = x;
        this.y = y;
    }
    clone() {
        return new Point(this.x, this.y);
    }
}
class Color {
    constructor(name) {
```

```

    this.name = name;
  }
  clone() {
    return new Color(this.name);
  }
}
class ColorPoint extends Point {
  constructor(x, y, color) {
    super(x, y);
    this.color = color;
  }
  clone() {
    return new ColorPoint(
      this.x, this.y, this.color.clone()); // (A)
  }
}

```

Line A demonstrates an important aspect of this technique: compound instance property values must also be cloned, recursively.

14.2 Static factory methods

A *copy constructor* is a constructor that uses another instance of the current class to set up the current instance. Copy constructors are popular in static languages such as C++ and Java, where you can provide multiple versions of a constructor via *static overloading*. Here, *static* means that the choice which version to use, is made at compile time.

In JavaScript, we must make that decision at runtime and that leads to inelegant code:

```

class Point {
  constructor(...args) {
    if (args[0] instanceof Point) {
      // Copy constructor
      const [other] = args;
      this.x = other.x;
      this.y = other.y;
    } else {
      const [x, y] = args;
      this.x = x;
      this.y = y;
    }
  }
}

```

This is how you'd use this class:

```

const original = new Point(-1, 4);
const copy = new Point(original);
assert.deepEqual(copy, original);

```

Static factory methods are an alternative to constructors and work better in this case because we can directly invoke the desired functionality. (Here, *static* means that these factory methods are class methods.)

In the following example, the three classes `Point`, `Color` and `ColorPoint` each have a static factory method `.from()`:

```
class Point {
  constructor(x, y) {
    this.x = x;
    this.y = y;
  }
  static from(other) {
    return new Point(other.x, other.y);
  }
}
class Color {
  constructor(name) {
    this.name = name;
  }
  static from(other) {
    return new Color(other.name);
  }
}
class ColorPoint extends Point {
  constructor(x, y, color) {
    super(x, y);
    this.color = color;
  }
  static from(other) {
    return new ColorPoint(
      other.x, other.y, Color.from(other.color)); // (A)
  }
}
```

In line A, we once again copy recursively.

This is how `ColorPoint.from()` works:

```
const original = new ColorPoint(-1, 4, new Color('red'));
const copy = ColorPoint.from(original);
assert.deepEqual(copy, original);
```

14.3 Acknowledgements

- [Ron Korvig](#) reminded me to use static factory methods and not overloaded constructors for deep-copying in JavaScript.

Chapter 15

Immutable wrappers for collections

Contents

15.1 Wrapping objects	145
15.1.1 Making collections immutable via wrapping	146
15.2 An immutable wrapper for Maps	146
15.3 An immutable wrapper for Arrays	147

An immutable wrapper for a collection makes that collection immutable by wrapping it in a new object. In this chapter, we examine how that works and why it is useful.

15.1 Wrapping objects

If there is an object whose interface we'd like to reduce, we can take the following approach:

- Create a new object that stores the original in a private field. The new object is said to *wrap* the original object. The new object is called *wrapper*, the original object *wrapped object*.
- The wrapper only forwards some of the method calls it receives to the wrapped object.

This is what wrapping looks like:

```
class Wrapper {
  #wrapped;
  constructor(wrapped) {
    this.#wrapped = wrapped;
  }
  allowedMethod1(...args) {
    return this.#wrapped.allowedMethod1(...args);
  }
}
```

```

    }
    allowedMethod2(...args) {
      return this.#wrapped.allowedMethod2(...args);
    }
  }
}

```

Related software design patterns:

- Wrapping is related to [the Gang of Four design pattern Facade](#).
- We used [forwarding](#) to implement [delegation](#). Delegation means that one object lets another object (the *delegate*) handle some of its work. It is an alternative to inheritance for sharing code.

15.1.1 Making collections immutable via wrapping

To make a collection immutable, we can use wrapping and remove all destructive operations from its interface.

One important use case for this technique is an object that has an internal mutable data structure that it wants to export safely without copying it. The export being “live” may also be a goal. The object can achieve its goals by wrapping the internal data structure and making it immutable.

The next two sections showcase immutable wrappers for Maps and Arrays. They both have the following limitations:

- They are sketches. More work is needed to make them suitable for practical use: Better checks, support for more methods, etc.
- They work shallowly: Each one makes the wrapped object immutable, but not the data it returns. This could be fixed by wrapping some of the results that are returned by methods.

15.2 An immutable wrapper for Maps

Class `ImmutableMapWrapper` produces wrappers for Maps:

```

class ImmutableMapWrapper {
  static _setUpPrototype() {
    // Only forward non-destructive methods to the wrapped Map:
    for (const methodName of ['get', 'has', 'keys', 'size']) {
      ImmutableMapWrapper.prototype[methodName] = function (...args) {
        return this.#wrappedMap[methodName](...args);
      }
    }
  }

  #wrappedMap;
  constructor(wrappedMap) {
    this.#wrappedMap = wrappedMap;
  }
}

```

```

}
ImmutableMapWrapper._setUpPrototype();

```

The setup of the prototype has to be performed by a static method, because we only have access to the private field `._wrappedMap` from inside the class.

This is `ImmutableMapWrapper` in action:

```

const map = new Map([[false, 'no'], [true, 'yes']]);
const wrapped = new ImmutableMapWrapper(map);

// Non-destructive operations work as usual:
assert.equal(
  wrapped.get(true), 'yes');
assert.equal(
  wrapped.has(false), true);
assert.deepEqual(
  [...wrapped.keys()], [false, true]);

// Destructive operations are not available:
assert.throws(
  () => wrapped.set(false, 'never!'),
  /^TypeError: wrapped.set is not a function$/);
assert.throws(
  () => wrapped.clear(),
  /^TypeError: wrapped.clear is not a function$/);

```

15.3 An immutable wrapper for Arrays

For an Array `arr`, normal wrapping is not enough because we need to intercept not just method calls, but also property accesses such as `arr[1] = true`. [JavaScript proxies](#) enable us to do this:

```

const RE_INDEX_PROP_KEY = /^[0-9]+$/;
const ALLOWED_PROPERTIES = new Set([
  'length', 'constructor', 'slice', 'concat']);

function wrapArrayImmutably(arr) {
  const handler = {
    get(target, propKey, receiver) {
      // We assume that propKey is a string (not a symbol)
      if (RE_INDEX_PROP_KEY.test(propKey) // simplified check!
        || ALLOWED_PROPERTIES.has(propKey)) {
        return Reflect.get(target, propKey, receiver);
      }
      throw new TypeError(`Property "${propKey}" can't be accessed`);
    },
    set(target, propKey, value, receiver) {
      throw new TypeError('Setting is not allowed');
    }
  };

```

```

    },
    deleteProperty(target, propKey) {
      throw new TypeError('Deleting is not allowed');
    },
  };
  return new Proxy(arr, handler);
}

```

Let's wrap an Array:

```

const arr = ['a', 'b', 'c'];
const wrapped = wrapArrayImmutably(arr);

// Non-destructive operations are allowed:
assert.deepEqual(
  wrapped.slice(1), ['b', 'c']);
assert.equal(
  wrapped[1], 'b');

// Destructive operations are not allowed:
assert.throws(
  () => wrapped[1] = 'x',
  /^TypeError: Setting is not allowed$/);
assert.throws(
  () => wrapped.shift(),
  /^TypeError: Property "shift" can't be accessed$/);

```

Part VI

Regular expressions

Chapter 16

Regular expressions: lookahead assertions by example

Contents

16.1 Cheat sheet: lookahead assertions	151
16.2 Warnings for this chapter	152
16.3 Example: Specifying what comes before or after a match (positive lookahead)	152
16.4 Example: Specifying what does not come before or after a match (negative lookahead)	153
16.4.1 There are no simple alternatives to negative lookahead assertions	153
16.5 Interlude: pointing lookahead assertions inward	154
16.6 Example: match strings not starting with 'abc'	154
16.7 Example: match substrings that do not contain '.mjs'	155
16.8 Example: skipping lines with comments	155
16.9 Example: smart quotes	156
16.9.1 Supporting escaping via backslashes	156
16.10 Acknowledgements	157
16.11 Further reading	157

In this chapter we use examples to explore lookahead assertions in regular expressions. A lookahead assertion is non-capturing and must match (or not match) what comes before (or after) the current location in the input string.

16.1 Cheat sheet: lookahead assertions

Table 16.1: Overview of available lookahead assertions.

Pattern	Name	
(?=«pattern»)	Positive lookahead	ES3
(?!«pattern»)	Negative lookahead	ES3
(?<=«pattern»)	Positive lookbehind	ES2018
(?<!=«pattern»)	Negative lookbehind	ES2018

There are four lookahead assertions (tbl. 16.1)

- Lookahead assertions (ECMAScript 3):
 - Positive lookahead: (|=«pattern»)| matches if pattern matches what comes after the current location in the input string.
 - Negative lookahead: (?!«pattern»)| matches if pattern does not match what comes after the current location in the input string.
- Lookbehind assertions (ECMAScript 2018):
 - Positive lookbehind: (?<=«pattern»)| matches if pattern matches what comes before the current location in the input string.
 - Negative lookbehind: (?<!=«pattern»)| matches if pattern does not match what comes before the current location in the input string.

16.2 Warnings for this chapter

- The examples show what can be achieved via lookahead assertions. However, regular expression aren't always the best solution. Another technique, such as proper parsing, may be a better choice.
- Lookbehind assertions are a relatively new feature that may not be supported by all JavaScript engines you are targeting.
- Lookaround assertions may affect performance negatively, especially if their patterns match long strings.

16.3 Example: Specifying what comes before or after a match (positive lookahead)

In the following interaction, we extract quoted words:

```
> 'how "are" "you" doing'.match(/(?<=")[a-z]+(?=")/g)
[ 'are', 'you' ]
```

Two lookahead assertions help us here:

- (?<=) “must be preceded by a quote”
- (?=) “must be followed by a quote”

Lookaround assertions are especially convenient for .match() in /g mode, which returns whole matches (capture group 0). Whatever the pattern of a lookahead assertion

matches is not captured. Without lookahead assertions, the quotes show up in the result:

```
> 'how "are" "you" doing'.match(/"([a-z]+)"/g)
[ '"are"', '"you"' ]
```

16.4 Example: Specifying what does not come before or after a match (negative lookahead)

How can we achieve the opposite of what we did in the previous section and extract all unquoted words from a string?

- Input: 'how "are" "you" doing'
- Output: ['how', 'doing']

Our first attempt is to simply convert positive lookahead assertions to negative lookahead assertions. Alas, that fails:

```
> 'how "are" "you" doing'.match(/(?!") [a-z]+(?!")/g)
[ 'how', 'r', 'o', 'doing' ]
```

The problem is that we extract sequences of characters that are not bracketed by quotes. That means that in the string '"are"', the "r" in the middle is considered unquoted, because it is preceded by an "a" and followed by an "e".

We can fix this by stating that prefix and suffix must be neither quote nor letter:

```
> 'how "are" "you" doing'.match(/(?![a-zA-Z]) [a-z]+(?![a-zA-Z])/g)
[ 'how', 'doing' ]
```

Another solution is to demand via `\b` that the sequence of characters `[a-z]+` start and end at word boundaries:

```
> 'how "are" "you" doing'.match(/(?!") \b[a-z]+\b(?!")/g)
[ 'how', 'doing' ]
```

One thing that is nice about negative lookbehind and negative lookahead is that they also work at the beginning or end, respectively, of a string – as demonstrated in the example.

16.4.1 There are no simple alternatives to negative lookahead assertions

Negative lookahead assertions are a powerful tool and usually impossible to emulate via other regular expression means.

If we don't want to use them, we normally have to take a completely different approach. For example, in this case, we could split the string into (quoted and unquoted) words and then filter those:

```
const str = 'how "are" "you" doing';

const allWords = str.match(/"?[a-z]+"?/g);
const unquotedWords = allWords.filter(
```

```
w => !w.startsWith('') || !w.endsWith('');
assert.deepEqual(unquotedWords, ['how', 'doing']);
```

Benefits of this approach:

- It works on older engines.
- It is easy to understand.

16.5 Interlude: pointing lookahead assertions inward

All of the examples we have seen so far have in common that the lookahead assertions dictate what must come before or after the match but without including those characters in the match.

The regular expressions shown in the remainder of this chapter are different: Their lookahead assertions point inward and restrict what's inside the match.

16.6 Example: match strings not starting with 'abc'

Let's assume we want to match all strings that do not start with 'abc'. Our first attempt could be the regular expression `/^(?!abc)/`.

That works well for `.test()`:

```
> /^(?!abc)/.test('xyz')
true
```

However, `.exec()` gives us an empty string:

```
> /^(?!abc)/.exec('xyz')
{ 0: '', index: 0, input: 'xyz', groups: undefined }
```

The problem is that assertions such as lookahead assertions don't expand the matched text. That is, they don't capture input characters, they only make demands about the current location in the input.

Therefore, the solution is to add a pattern that does capture input characters:

```
> /^(?!abc).*/.exec('xyz')
{ 0: 'xyz', index: 0, input: 'xyz', groups: undefined }
```

As desired, this new regular expression rejects strings that are prefixed with 'abc':

```
> /^(?!abc).*/.exec('abc')
null
> /^(?!abc).*/.exec('abcd')
null
```

And it accepts strings that don't have the full prefix:

```
> /^(?!abc).*/.exec('ab')
{ 0: 'ab', index: 0, input: 'ab', groups: undefined }
```

16.7 Example: match substrings that do not contain '.mjs'

In the following example, we want to find

```
import ... from '«module-specifier»';
```

where module-specifier does not end with '.mjs'.

```
const code = `
import {transform} from './util';
import {Person} from './person.mjs';
import {zip} from 'lodash';
`.trim();
assert.deepEqual(
  code.match(/^import .*? from '([^']+(?!\.mjs)';$/umg),
  [
    "import {transform} from './util';",
    "import {zip} from 'lodash';",
  ]);
```

Here, the lookbehind assertion `(?!\.mjs)` acts as a *guard* and prevents that the regular expression matches strings that contain '.mjs' at this location.

16.8 Example: skipping lines with comments

Scenario: We want to parse lines with settings, while skipping comments. For example:

```
const RE_SETTING = /^(?!#)([^\:]*):(.*)$/

const lines = [
  'indent: 2', // setting
  '# Trim trailing whitespace:', // comment
  'whitespace: trim', // setting
];
for (const line of lines) {
  const match = RE_SETTING.exec(line);
  if (match) {
    const key = JSON.stringify(match[1]);
    const value = JSON.stringify(match[2]);
    console.log(`KEY: ${key} VALUE: ${value}`);
  }
}

// Output:
// 'KEY: "indent" VALUE: " 2"'
// 'KEY: "whitespace" VALUE: " trim"'
```

How did we arrive at the regular expression `RE_SETTING`?

We started with the following regular expression for settings:

```
/^([^:]*):(.*)$/
```

Intuitively, it is a sequence of the following parts:

- Start of the line
- Non-colons (zero or more)
- A single colon
- Any characters (zero or more)
- The end of line

This regular expression does reject *some* comments:

```
> /^([^:]*):(.*)$/ .test('# Comment')
false
```

But it accepts others (that have colons in them):

```
> /^([^:]*):(.*)$/ .test('# Comment:')
true
```

We can fix that by prefixing `(?!#)` as a guard. Intuitively, it means: “The current location in the input string must not be followed by the character `#`.”

The new regular expression works as desired:

```
> /^(?!#)([~:]*):(.*)$/ .test('# Comment:')
false
```

16.9 Example: smart quotes

Let’s assume we want to convert pairs of straight double quotes to curly quotes:

- Input: ``"yes" and "no"`
- Output: ``“yes” and “no”``

This is our first attempt:

```
> `The words "must" and "should".`.replace(/"(.)"/g, '`$1`')
'The words "must" and "should".'
```

Only the first quote and the last quote is curly. The problem here is that the `*` quantifier matches *greedily* (as much as possible).

If we put a question mark after the `*`, it matches *reluctantly*:

```
> `The words "must" and "should".`.replace(/"(.*?)"/g, '`$1`')
'The words “must” and “should”.'
```

16.9.1 Supporting escaping via backslashes

What if we want to allow the escaping of quotes via backslashes? We can do that by using the guard `(?<!\\\)` before the quotes:

```
> const regexp = /(?<!\\\)"(.*?)"(?<!\\\)"/g;
> String.raw`"straight\" and "curly"`.replace(regexp, '`$1`')
'\\“straight\\” and “curly”'
```

As a post-processing step, we would still need to do:

```
.replace(/\\"/g, `"`)
```

However, this regular expression can fail when there is a backslash-escaped backslash:

```
> String.raw`Backslash: "\\\".replace(/(?<!\\" (.*) (?<!\\" )/g, '$1')
'Backslash: "\\\"'
```

The second backslash prevented the quotes from becoming curly.

We can fix that if we make our guard more sophisticated (? : makes the group non-capturing):

```
(?<=[^\\](?:\\\\)*)
```

The new guard allows pairs of backslashes before quotes:

```
> const regExp = /(?!<=[^\\](?:\\\\)*)"(.*)"(?!<=[^\\](?:\\\\)*)"/g;
> String.raw`Backslash: "\\\".replace(regExp, '$1')
'Backslash: "\\\"'
```

One issue remains. This guard prevents the first quote from being matched if it appears at the beginning of a string:

```
> const regExp = /(?!<=[^\\](?:\\\\)*)"(.*)"(?!<=[^\\](?:\\\\)*)"/g;
> `"abc"`.replace(regExp, '$1')
'abc'
```

We can fix that by changing the first guard to: (?<=[^\\](?:\\\\)*|^)

```
> const regExp = /(?!<=[^\\](?:\\\\)*|^)"(.*)"(?!<=[^\\](?:\\\\)*)"/g;
> `"abc"`.replace(regExp, '$1')
'abc'
```

16.10 Acknowledgements

- The first regular expression that handles escaped backslashes in front of quotes [was proposed by @jonasraoni on Twitter](#).

16.11 Further reading

- [Chapter “Regular expressions \(RegExp\)”](#) in “JavaScript for impatient programmers”

Chapter 17

Composing regular expressions via `re-template-tag` (bonus)

Contents

17.1 The basics	159
17.2 A tour of the features	160
17.2.1 Main features	160
17.2.2 Details and advanced features	160
17.3 Why is this useful?	160
17.4 <code>re</code> and named capture groups	160

My small library `re-template-tag` provides a template tag function for composing regular expressions. This chapter explains how it works. The intent is to introduce the ideas behind the library (more than the library itself).

17.1 The basics

The library implements the template tag `re` for regular expressions.

Installation:

```
npm install re-template-tag
```

Importing:

```
import {re} from 're-template-tag';
```

Use: The following two expressions produce the same regular expression.

```
assert.deepEqual(  
  re`/abc/gu`,  
  /abc/gu);
```

17.2 A tour of the features

The [unit tests](#) are a good tour of the features of *re-template-tag*.

17.2.1 Main features

You can use *re* to assemble a regular expression from fragments:

```
const RE_YEAR = /([0-9]{4})/;
const RE_MONTH = /([0-9]{2})/;
const RE_DAY = /([0-9]{2})/;
const RE_DATE = re`^${RE_YEAR}-${RE_MONTH}-${RE_DAY}$u`;
assert.equal(RE_DATE.source, '^([0-9]{4})-([0-9]{2})-([0-9]{2})$');
```

Any strings you insert are escaped properly:

```
assert.equal(re`/-${'.'}-/u`.source, '-\\.\\.');
```

Flag-less mode:

```
const regexp = re`abc`;
assert.ok(regexp instanceof RegExp);
assert.equal(regexp.source, 'abc');
assert.equal(regexp.flags, '');

```

17.2.2 Details and advanced features

You can use the backslash as you would inside regular expression literals:

```
assert.equal(re`/\./u`.source, '\\.');
```

Regular expression flags (such as */u* in the previous example) can also be computed:

```
const regexp = re`/abc/${'g'+ 'u'}`;
assert.ok(regexp instanceof RegExp);
assert.equal(regexp.source, 'abc');
assert.equal(regexp.flags, 'gu');
```

17.3 Why is this useful?

- You can compose regular expressions from fragments and document the fragments via comments. That makes regular expressions easier to understand.
- You can define constants for regular expression fragments and reuse them.
- You can define plain text constants via strings and insert them into regular expressions and they are escaped as necessary.

17.4 *re* and named capture groups

re profits from [named capture groups](#) because they make the fragments more independent.

Without named groups:

```
const RE_YEAR = /[0-9]{4}/;
const RE_MONTH = /[0-9]{2}/;
const RE_DAY = /[0-9]{2}/;
const RE_DATE = re`/^${RE_YEAR}-${RE_MONTH}-${RE_DAY}$/u`;

const match = RE_DATE.exec('2017-01-27');
assert.equal(match[1], '2017');
```

With named groups:

```
const RE_YEAR = re`(?<year>[0-9]{4})`;
const RE_MONTH = re`(?<month>[0-9]{2})`;
const RE_DAY = re`(?<day>[0-9]{2})`;
const RE_DATE = re`/${RE_YEAR}-${RE_MONTH}-${RE_DAY}$/u`;

const match = RE_DATE.exec('2017-01-27');
assert.equal(match.groups.year, '2017');
```


Part VII

Miscellaneous topics

Chapter 18

Exploring Promises by implementing them

Contents

18.1 Refresher: states of Promises	166
18.2 Version 1: Stand-alone Promise	166
18.2.1 Method .then()	167
18.2.2 Method .resolve()	168
18.3 Version 2: Chaining .then() calls	169
18.4 Convenience method .catch()	170
18.5 Omitting reactions	170
18.6 The implementation	171
18.7 Version 3: Flattening Promises returned from .then() callbacks . .	172
18.7.1 Returning Promises from a callback of .then()	172
18.7.2 Flattening makes Promise states more complicated	173
18.7.3 Implementing Promise-flattening	173
18.8 Version 4: Exceptions thrown in reaction callbacks	175
18.9 Version 5: Revealing constructor pattern	177



Required knowledge: Promises

For this chapter, you should be roughly familiar with Promises, but much relevant knowledge is also reviewed here. If necessary, you can read [the chapter on Promises in “JavaScript for impatient programmers”](#).

In this chapter, we will approach Promises from a different angle: Instead of using this API, we will create a simple implementation of it. This different angle once helped me greatly with making sense of Promises.

The Promise implementation is the class `ToyPromise`. In order to be easier to understand, it doesn't completely match the API. But it is close enough to still give us much insight into how Promises work.



Repository with code

`ToyPromise` is available on GitHub, in the repository [toy-promise](#).

18.1 Refresher: states of Promises

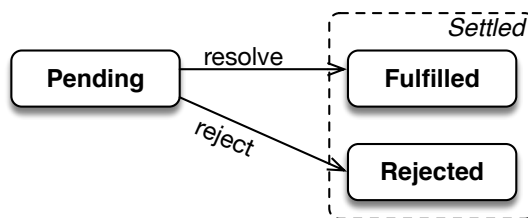


Figure 18.1: The states of a Promise (simplified version): A Promise is initially pending. If we resolve it, it becomes fulfilled. If we reject it, it becomes rejected.

We start with a simplified version of how Promise states work (fig. 18.1):

- A Promise is initially *pending*.
- If a Promise is *resolved* with a value *v*, it becomes *fulfilled* (later, we'll see that resolving can also reject). *v* is now the *fulfillment value* of the Promise.
- If a Promise is *rejected* with an error *e*, it becomes *rejected*. *e* is now the *rejection value* of the Promise.

18.2 Version 1: Stand-alone Promise

Our first implementation is a stand-alone Promise with minimal functionality:

- We can create a Promise.
- We can resolve or reject a Promise and we can only do it once.
- We can register *reactions* (callbacks) via `.then()`. Registering must do the right thing independently of whether the Promise has already been settled or not.
- `.then()` does not support chaining, yet – it does not return anything.

`ToyPromise1` is a class with three prototype methods:

- `ToyPromise1.prototype.resolve(value)`
- `ToyPromise1.prototype.reject(reason)`
- `ToyPromise1.prototype.then(onFulfilled, onRejected)`

That is, `resolve` and `reject` are methods (and not functions handed to a callback parameter of the constructor).

This is how this first implementation is used:

```
// .resolve() before .then()
const tp1 = new ToyPromise1();
tp1.resolve('abc');
tp1.then((value) => {
  assert.equal(value, 'abc');
});

// .then() before .resolve()
const tp2 = new ToyPromise1();
tp2.then((value) => {
  assert.equal(value, 'def');
});
tp2.resolve('def');
```

Fig. 18.2 illustrates how our first `ToyPromise` works.



The diagrams of the data flow in Promises are optional

The motivation for the diagrams is to have a visual explanation for how Promises work. But they are optional. If you find them confusing, you can ignore them and focus on the code.

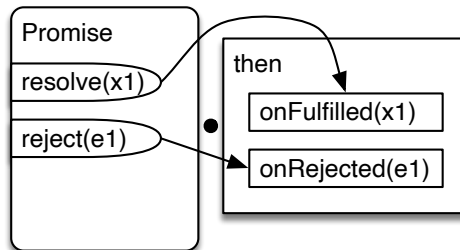


Figure 18.2: `ToyPromise1`: If a Promise is resolved, the provided value is passed on to the *fulfillment reactions* (first arguments of `.then()`). If a Promise is rejected, the provided value is passed on to the *rejection reactions* (second arguments of `.then()`).

18.2.1 Method `.then()`

Let's examine `.then()` first. It has to handle two cases:

- If the Promise is still pending, it queues invocations of `onFulfilled` and `onRejected`. They are to be used later, when the Promise is settled.
- If the Promise is already fulfilled or rejected, `onFulfilled` or `onRejected` can be invoked right away.

```
then(onFulfilled, onRejected) {
  const fulfillmentTask = () => {
    if (typeof onFulfilled === 'function') {
      onFulfilled(this._promiseResult);
    }
  };
}
```

```

    }
  };
  const rejectionTask = () => {
    if (typeof onRejected === 'function') {
      onRejected(this._promiseResult);
    }
  };
  switch (this._promiseState) {
    case 'pending':
      this._fulfillmentTasks.push(fulfillmentTask);
      this._rejectionTasks.push(rejectionTask);
      break;
    case 'fulfilled':
      addToTaskQueue(fulfillmentTask);
      break;
    case 'rejected':
      addToTaskQueue(rejectionTask);
      break;
    default:
      throw new Error();
  }
}

```

The previous code snippet uses the following helper function:

```

function addToTaskQueue(task) {
  setTimeout(task, 0);
}

```

Promises must always settle asynchronously. That's why we don't directly execute tasks, we add them to the task queue of the event loop (of browsers, Node.js, etc.). Note that the real Promise API doesn't use normal tasks (like `setTimeout()`), it uses *microtasks*, which are tightly coupled with the current normal task and always execute directly after it.

18.2.2 Method `.resolve()`

`.resolve()` works as follows: If the Promise is already settled, it does nothing (ensuring that a Promise can only be settled once). Otherwise, the state of the Promise changes to 'fulfilled' and the result is cached in `this.promiseResult`. Next, all fulfillment reactions that have been enqueued so far, are invoked.

```

resolve(value) {
  if (this._promiseState !== 'pending') return this;
  this._promiseState = 'fulfilled';
  this._promiseResult = value;
  this._clearAndEnqueueTasks(this._fulfillmentTasks);
  return this; // enable chaining
}

_clearAndEnqueueTasks(tasks) {
  this._fulfillmentTasks = undefined;
}

```

```

    this._rejectionTasks = undefined;
    tasks.map(addToTaskQueue);
  }

```

`reject()` is similar to `resolve()`.

18.3 Version 2: Chaining `.then()` calls

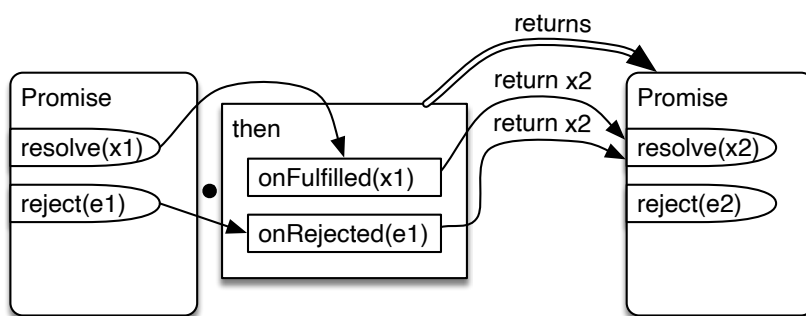


Figure 18.3: `ToyPromise2` chains `.then()` calls: `.then()` now returns a `Promise` that is resolved by whatever value is returned by the fulfillment reaction or the rejection reaction.

The next feature we implement is chaining (fig. 18.3): A value that we return from a fulfillment reaction or a rejection reaction can be handled by a fulfillment reaction in a following `.then()` call. (In the next version, chaining will become much more useful, due to special support for returning Promises.)

In the following example:

- First `.then()`: We return a value in a fulfillment reaction.
- Second `.then()`: We receive that value via a fulfillment reaction.

```

new ToyPromise2()
  .resolve('result1')
  .then(x => {
    assert.equal(x, 'result1');
    return 'result2';
  })
  .then(x => {
    assert.equal(x, 'result2');
  });

```

In the following example:

- First `.then()`: We return a value in a rejection reaction.
- Second `.then()`: We receive that value via a fulfillment reaction.

```

new ToyPromise2()
  .reject('error1')
  .then(null,

```

```

x => {
  assert.equal(x, 'error1');
  return 'result2';
})
.then(x => {
  assert.equal(x, 'result2');
});

```

18.4 Convenience method `.catch()`

The new version introduces a convenience method `.catch()` that makes it easier to only provide a rejection reaction. Note that only providing a fulfillment reaction is already easy – we simply omit the second parameter of `.then()` (see previous example).

The previous example looks nicer if we use it (line A):

```

new ToyPromise2()
  .reject('error1')
  .catch(x => { // (A)
    assert.equal(x, 'error1');
    return 'result2';
  })
  .then(x => {
    assert.equal(x, 'result2');
  });

```

The following two method invocations are equivalent:

```

.catch(rejectionReaction)
.then(null, rejectionReaction)

```

This is how `.catch()` is implemented:

```

catch(onRejected) { // [new]
  return this.then(null, onRejected);
}

```

18.5 Omitting reactions

The new version also forwards fulfillments if we omit a fulfillment reaction and it forwards rejections if we omit a rejection reaction. Why is that useful?

The following example demonstrates passing on rejections:

```

someAsyncFunction()
  .then(fulfillmentReaction1)
  .then(fulfillmentReaction2)
  .catch(rejectionReaction);

```

`rejectionReaction` can now handle the rejections of `someAsyncFunction()`, `fulfillmentReaction1`, and `fulfillmentReaction2`.

The following example demonstrates passing on fulfillments:

```
someAsyncFunction()
  .catch(rejectionReaction)
  .then(fulfillmentReaction);
```

If `someAsyncFunction()` rejects its Promise, `rejectionReaction` can fix whatever is wrong and return a fulfillment value that is then handled by `fulfillmentReaction`.

If `someAsyncFunction()` fulfills its Promise, `fulfillmentReaction` can also handle it because the `.catch()` is skipped.

18.6 The implementation

How is all of this handled under the hood?

- `.then()` returns a Promise that is resolved with what either `onFulfilled` or `onRejected` return.
- If `onFulfilled` or `onRejected` are missing, whatever they would have received is passed on to the Promise returned by `.then()`.

Only `.then()` changes:

```
then(onFulfilled, onRejected) {
  const resultPromise = new ToyPromise2(); // [new]

  const fulfillmentTask = () => {
    if (typeof onFulfilled === 'function') {
      const returned = onFulfilled(this._promiseResult);
      resultPromise.resolve(returned); // [new]
    } else { // [new]
      // `onFulfilled` is missing
      // => we must pass on the fulfillment value
      resultPromise.resolve(this._promiseResult);
    }
  };

  const rejectionTask = () => {
    if (typeof onRejected === 'function') {
      const returned = onRejected(this._promiseResult);
      resultPromise.resolve(returned); // [new]
    } else { // [new]
      // `onRejected` is missing
      // => we must pass on the rejection value
      resultPromise.reject(this._promiseResult);
    }
  };

  ...
}
```

```

    return resultPromise; // [new]
  }

```

`.then()` creates and returns a new Promise (first line and last line of the method). Additionally:

- `fulfillmentTask` works differently. This is what now happens after fulfillment:
 - If `onFulfilled` was provided, it is called and its result is used to resolve `resultPromise`.
 - If `onFulfilled` is missing, we use the fulfillment value of the current Promise to resolve `resultPromise`.
- `rejectionTask` works differently. This is what now happens after rejection:
 - If `onRejected` was provided, it is called and its result is used to *resolve* `resultPromise`. Note that `resultPromise` is not rejected: We are assuming that `onRejected()` fixed whatever problem there was.
 - If `onRejected` is missing, we use the rejection value of the current Promise to reject `resultPromise`.

18.7 Version 3: Flattening Promises returned from `.then()` callbacks

18.7.1 Returning Promises from a callback of `.then()`

Promise-flattening is mostly about making chaining more convenient: If we want to pass on a value from one `.then()` callback to the next one, we return it in the former. After that, `.then()` puts it into the Promise that it has already returned.

This approach becomes inconvenient if we return a Promise from a `.then()` callback. For example, the result of a Promise-based function (line A):

```

asyncFunc1()
  .then((result1) => {
    assert.equal(result1, 'Result of asyncFunc1()');
    return asyncFunc2(); // (A)
  })
  .then((result2Promise) => {
    result2Promise
      .then((result2) => { // (B)
        assert.equal(
          result2, 'Result of asyncFunc2()');
      });
  });

```

This time, putting the value returned in line A into the Promise returned by `.then()` forces us to unwrap that Promise in line B. It would be nice if instead, the Promise returned in line A replaced the Promise returned by `.then()`. How exactly that could be done is not immediately clear, but if it worked, it would let us write our code like this:

```

asyncFunc1()
  .then((result1) => {

```

```

    assert.equal(result1, 'Result of asyncFunc1()');
    return asyncFunc2(); // (A)
  })
  .then((result2) => {
    // result2 is the fulfillment value, not the Promise
    assert.equal(
      result2, 'Result of asyncFunc2()');
  });

```

In line A, we returned a Promise. Thanks to Promise-flattening, `result2` is the fulfillment value of that Promise, not the Promise itself.

18.7.2 Flattening makes Promise states more complicated



Flattening Promises in the ECMAScript specification

In the ECMAScript specification, the details of flattening Promises are described in [section “Promise Objects”](#).

How does the Promise API handle flattening?

If a Promise *P* is resolved with a Promise *Q*, then *P* does not wrap *Q*, *P* “becomes” *Q*: State and settlement value of *P* are now always the same as *Q*’s. That helps us with `.then()` because `.then()` resolves the Promise it returns with the value returned by one of its callbacks.

How does *P* become *Q*? By *locking in* on *Q*: *P* becomes externally unresolvable and a settlement of *Q* triggers a settlement of *P*. Lock-in is an additional invisible Promise state that makes states more complicated.

The Promise API has one additional feature: *Q* doesn’t have to be a Promise, only a so-called *thenable*. A thenable is an object with a method `.then()`. The reason for this added flexibility is to enable different Promise implementations to work together (which mattered when Promises were first added to the language).

Fig. 18.4 visualizes the new states.

Note that the concept of *resolving* has also become more complicated. Resolving a Promise now only means that it can’t be settled directly, anymore:

- Resolving may reject a Promise: We can resolve a Promise with a rejected Promise.
- Resolving may not even settle a Promise: We can resolve a Promise with another one that is always pending.

The ECMAScript specification puts it this way: “An unresolved Promise is always in the pending state. A resolved Promise may be pending, fulfilled, or rejected.”

18.7.3 Implementing Promise-flattening

Fig. 18.5 shows how `ToyPromise3` handles flattening.

We detect thenables via this function:

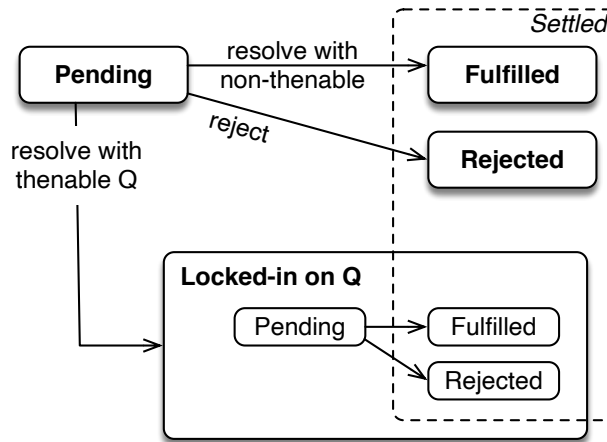


Figure 18.4: All states of a Promise: Promise-flattening introduces the invisible pseudo-state “locked-in”. That state is reached if a Promise *P* is resolved with a thenable *Q*. Afterwards, state and settlement value of *P* is always the same as those of *Q*.

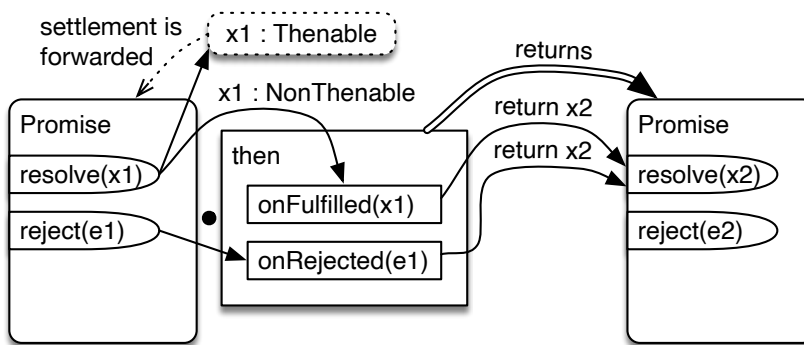


Figure 18.5: ToyPromise3 flattens resolved Promises: If the first Promise is resolved with a thenable *x1*, it locks in on *x1* and is settled with the settlement value of *x1*. If the first Promise is resolved with a non-thenable value, everything works as it did before.

```
function isThenable(value) { // [new]
  return typeof value === 'object' && value !== null
    && typeof value.then === 'function';
}
```

To implement lock-in, we introduce a new boolean flag `._alreadyResolved`. Setting it to `true` deactivates `.resolve()` and `.reject()` – for example:

```
resolve(value) { // [new]
  if (this._alreadyResolved) return this;
  this._alreadyResolved = true;

  if (isThenable(value)) {
    // Forward fulfillments and rejections from `value` to `this`.
    // The callbacks are always executed asynchronously
    value.then(
      (result) => this._doFulfill(result),
      (error) => this._doReject(error));
  } else {
    this._doFulfill(value);
  }

  return this; // enable chaining
}
```

If `value` is a thenable then we lock the current Promise in on it:

- If `value` is fulfilled with a result, the current Promise is also fulfilled with that result.
- If `value` is rejected with an error, the current Promise is also rejected with that error.

The settling is performed via the private methods `._doFulfill()` and `._doReject()`, to get around the protection via `._alreadyResolved`.

`._doFulfill()` is relatively simple:

```
_doFulfill(value) { // [new]
  assert.ok(!isThenable(value));
  this._promiseState = 'fulfilled';
  this._promiseResult = value;
  this._clearAndEnqueueTasks(this._fulfillmentTasks);
}
```

`.reject()` is not shown here. Its only new functionality is that it now also obeys `._alreadyResolved`.

18.8 Version 4: Exceptions thrown in reaction callbacks

As our final feature, we'd like our Promises to handle exceptions in user code as rejections (fig. 18.6). In this chapter, "user code" means the two callback parameters of `.then()`.

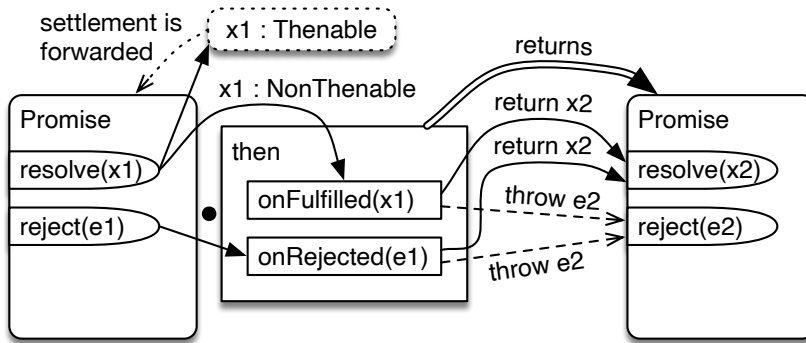


Figure 18.6: ToyPromise4 converts exceptions in Promise reactions to rejections of the Promise returned by `.then()`.

```
new ToyPromise4()
  .resolve('a')
  .then((value) => {
    assert.equal(value, 'a');
    throw 'b'; // triggers a rejection
  })
  .catch((error) => {
    assert.equal(error, 'b');
  })
```

`.then()` now runs the Promise reactions `onFulfilled` and `onRejected` safely, via the helper method `._runReactionSafely()` – for example:

```
const fulfillmentTask = () => {
  if (typeof onFulfilled === 'function') {
    this._runReactionSafely(resultPromise, onFulfilled); // [new]
  } else {
    // `onFulfilled` is missing
    // => we must pass on the fulfillment value
    resultPromise.resolve(this._promiseResult);
  }
};
```

`._runReactionSafely()` is implemented as follows:

```
._runReactionSafely(resultPromise, reaction) { // [new]
  try {
    const returned = reaction(this._promiseResult);
    resultPromise.resolve(returned);
  } catch (e) {
    resultPromise.reject(e);
  }
}
```

18.9 Version 5: Revealing constructor pattern

We are skipping one last step: If we wanted to turn `ToyPromise` into an actual `Promise` implementation, we'd still need to implement [the revealing constructor pattern](#): JavaScript Promises are not resolved and rejected via methods, but via functions that are handed to the *executor*, the callback parameter of the constructor.

```
const promise = new Promise(  
  (resolve, reject) => { // executor  
    // ...  
  });
```

If the executor throws an exception, then `promise` is rejected.

Chapter 19

Metaprogramming with Proxies (early access)

Contents

19.1 Overview	180
19.2 Programming versus metaprogramming	181
19.2.1 Kinds of metaprogramming	181
19.3 Proxies explained	182
19.3.1 An example	183
19.3.2 Function-specific traps	183
19.3.3 Intercepting method calls	184
19.3.4 Revocable Proxies	185
19.3.5 Proxies as prototypes	185
19.3.6 Forwarding intercepted operations	186
19.3.7 Pitfall: not all objects can be wrapped transparently by Proxies	187
19.4 Use cases for Proxies	190
19.4.1 Tracing property accesses (get, set)	191
19.4.2 Warning about unknown properties (get, set)	193
19.4.3 Negative Array indices (get)	194
19.4.4 Data binding (set)	195
19.4.5 Accessing a restful web service (method calls)	196
19.4.6 Revocable references	197
19.4.7 Implementing the DOM in JavaScript	199
19.4.8 More use cases	199
19.4.9 Libraries that are using Proxies	200
19.5 The design of the Proxy API	200
19.5.1 Stratification: keeping base level and meta level separate	200
19.5.2 Virtual objects versus wrappers	202
19.5.3 Transparent virtualization and handler encapsulation	202
19.5.4 The meta object protocol and Proxy traps	203

19.5.5 Enforcing invariants for Proxies	205
19.6 FAQ: Proxies	209
19.6.1 Where is the enumerate trap?	209
19.7 Reference: the Proxy API	209
19.7.1 Creating Proxies	209
19.7.2 Handler methods	209
19.7.3 Invariants of handler methods	210
19.7.4 Operations that affect the prototype chain	212
19.7.5 Reflect	213
19.8 Conclusion	215
19.9 Further reading	215

19.1 Overview

Proxies enable us to intercept and customize operations performed on objects (such as getting properties). They are a *metaprogramming* feature.

In the following example:

- proxy is an empty object.
- handler can intercept operations that are performed on proxy, by implementing certain methods.
- If the handler does not intercept an operation, it is forwarded to target.

We are only intercepting one operation – get (getting properties):

```
const logged = [];

const target = {size: 0};
const handler = {
  get(target, propKey, receiver) {
    logged.push('GET ' + propKey);
    return 123;
  }
};

const proxy = new Proxy(target, handler);
```

When we get the property `proxy.size`, the handler intercepts that operation:

```
assert.equal(
  proxy.size, 123);

assert.deepEqual(
  logged, ['GET size']);
```

See [the reference for the complete API](#) for a list of operations that can be intercepted.

19.2 Programming versus metaprogramming

Before we can get into what Proxies are and why they are useful, we first need to understand what *metaprogramming* is.

In programming, there are levels:

- At the *base level* (also called: *application level*), code processes user input.
- At the *meta level*, code processes base level code.

Base and meta level can be different languages. In the following meta program, the metaprogramming language is JavaScript and the base programming language is Java.

```
const str = 'Hello' + '!'.repeat(3);
console.log('System.out.println("' + str + '")');
```

Metaprogramming can take different forms. In the previous example, we have printed Java code to the console. Let's use JavaScript as both metaprogramming language and base programming language. The classic example for this is the `eval()` function, which lets us evaluate/compile JavaScript code on the fly. In the interaction below, we use it to evaluate the expression `5 + 2`.

```
> eval('5 + 2')
7
```

Other JavaScript operations may not look like metaprogramming, but actually are, if we look closer:

```
// Base level
const obj = {
  hello() {
    console.log('Hello!');
  },
};

// Meta level
for (const key of Object.keys(obj)) {
  console.log(key);
}
```

The program is examining its own structure while running. This doesn't look like metaprogramming, because the separation between programming constructs and data structures is fuzzy in JavaScript. All of the `Object.*` methods can be considered metaprogramming functionality.

19.2.1 Kinds of metaprogramming

Reflective metaprogramming means that a program processes itself. Kiczales et al. [2] distinguish three kinds of reflective metaprogramming:

- **Introspection:** We have read-only access to the structure of a program.
- **Self-modification:** We can change that structure.
- **Intercession:** We can redefine the semantics of some language operations.

Let's look at examples.

Example: introspection. `Object.keys()` performs introspection (see previous example).

Example: self-modification. The following function `moveProperty` moves a property from a source to a target. It performs self-modification via the bracket operator for property access, the assignment operator and the `delete` operator. (In production code, we'd probably use **property descriptors** for this task.)

```
function moveProperty(source, propertyName, target) {
  target[propertyName] = source[propertyName];
  delete source[propertyName];
}
```

This is how `moveProperty()` is used:

```
const obj1 = { color: 'blue' };
const obj2 = {};

moveProperty(obj1, 'color', obj2);

assert.deepEqual(
  obj1, {});

assert.deepEqual(
  obj2, { color: 'blue' });
```

ECMAScript 5 doesn't support intercession; Proxies were created to fill that gap.

19.3 Proxies explained

Proxies bring intercession to JavaScript. They work as follows. There are many operations that we can perform on an object `obj` – for example:

- Getting the property `prop` of an object `obj` (`obj.prop`)
- Checking whether an object `obj` has a property `prop` (`'prop' in obj`)

Proxies are special objects that allow us to customize some of these operations. A Proxy is created with two parameters:

- **handler:** For each operation, there is a corresponding handler method that – if present – performs that operation. Such a method *intercepts* the operation (on its way to the target) and is called a *trap* – a term borrowed from the domain of operating systems.
- **target:** If the handler doesn't intercept an operation, then it is performed on the target. That is, it acts as a fallback for the handler. In a way, the Proxy wraps the target.

Note: The verb form of “intercession” is “to intercede”. Interceding is bidirectional in nature. Intercepting is unidirectional in nature.

19.3.1 An example

In the following example, the handler intercepts the operations `get` and `has`.

```
const logged = [];

const target = {};
const handler = {
  /** Intercepts: getting properties */
  get(target, propKey, receiver) {
    logged.push(`GET ${propKey}`);
    return 123;
  },

  /** Intercepts: checking whether properties exist */
  has(target, propKey) {
    logged.push(`HAS ${propKey}`);
    return true;
  }
};

const proxy = new Proxy(target, handler);
```

If we get a property (line A) or use the `in` operator (line B), the handler intercepts those operations:

```
assert.equal(proxy.age, 123); // (A)
assert.equal('hello' in proxy, true); // (B)

assert.deepEqual(
  logged, [
    'GET age',
    'HAS hello',
  ]);
```

The handler doesn't implement the trap `set` (setting properties). Therefore, setting `proxy.age` is forwarded to `target` and leads to `target.age` being set:

```
proxy.age = 99;
assert.equal(target.age, 99);
```

19.3.2 Function-specific traps

If the target is a function, two additional operations can be intercepted:

- `apply`: Making a function call. Triggered via:
 - `proxy(...)`
 - `proxy.call(...)`
 - `proxy.apply(...)`
- `construct`: Making a constructor call. Triggered via:
 - `new proxy(...)`

The reason for only enabling these traps for function targets is simple: Otherwise, we wouldn't be able to forward the operations `apply` and `construct`.

19.3.3 Intercepting method calls

If we want to intercept method calls via a Proxy, we are facing a challenge: There is no trap for method calls. Instead, a method call is viewed as a sequence of two operations:

- A `get` to retrieve a function
- An `apply` to call that function

Therefore, if we want to intercept method calls, we need to intercept two operations:

- First, we intercept the `get` and return a function.
- Second, we intercept the invocation of that function.

The following code demonstrates how that is done.

```
const traced = [];

function traceMethodCalls(obj) {
  const handler = {
    get(target, propKey, receiver) {
      const origMethod = target[propKey];
      return function (...args) { // implicit parameter `this`!
        const result = origMethod.apply(this, args);
        traced.push(propKey + JSON.stringify(args)
          + ' -> ' + JSON.stringify(result));
        return result;
      };
    }
  };
  return new Proxy(obj, handler);
}
```

We are not using a Proxy for the second interception; we are simply wrapping the original method in a function.

Let's use the following object to try out `traceMethodCalls()`:

```
const obj = {
  multiply(x, y) {
    return x * y;
  },
  squared(x) {
    return this.multiply(x, x);
  },
};

const tracedObj = traceMethodCalls(obj);
assert.equal(
  tracedObj.squared(9), 81);
```

```

assert.deepEqual(
  traced, [
    'multiply[9,9] -> 81',
    'squared[9] -> 81',
  ]);

```

Even the call `this.multiply()` inside `obj.squared()` is traced! That's because this keeps referring to the Proxy.

This is not the most efficient solution. One could, for example, cache methods. Furthermore, Proxies themselves have an impact on performance.

19.3.4 Revocable Proxies

Proxies can be *revoked* (switched off):

```
const {proxy, revoke} = Proxy.revocable(target, handler);
```

After we call the function `revoke` for the first time, any operation we apply to proxy causes a `TypeError`. Subsequent calls of `revoke` have no further effect.

```

const target = {}; // Start with an empty object
const handler = {}; // Don't intercept anything
const {proxy, revoke} = Proxy.revocable(target, handler);

// `proxy` works as if it were the object `target`:
proxy.city = 'Paris';
assert.equal(proxy.city, 'Paris');

revoke();

assert.throws(
  () => proxy.prop,
  /^TypeError: Cannot perform 'get' on a proxy that has been revoked$/
);

```

19.3.5 Proxies as prototypes

A Proxy proto can become the prototype of an object `obj`. Some operations that begin in `obj` may continue in `proto`. One such operation is `get`.

```

const proto = new Proxy({}, {
  get(target, propertyKey, receiver) {
    console.log('GET ' + propertyKey);
    return target[propertyKey];
  }
});

const obj = Object.create(proto);
obj.weight;

```

```
// Output:
// 'GET weight'
```

The property `weight` can't be found in `obj`, which is why the search continues in `proto` and the trap `get` is triggered there. There are more operations that affect prototypes; they are listed at the end of this chapter.

19.3.6 Forwarding intercepted operations

Operations whose traps the handler doesn't implement are automatically forwarded to the target. Sometimes there is some task we want to perform in addition to forwarding the operation. For example, intercepting and logging all operations, without preventing them from reaching the target:

```
const handler = {
  deleteProperty(target, propKey) {
    console.log('DELETE ' + propKey);
    return delete target[propKey];
  },
  has(target, propKey) {
    console.log('HAS ' + propKey);
    return propKey in target;
  },
  // Other traps: similar
}
```

19.3.6.1 Improvement: using `Reflect.*`

For each trap, we first log the name of the operation and then forward it by performing it manually. JavaScript has the module-like object `Reflect` that helps with forwarding.

For each trap:

```
handler.trap(target, arg_1, ..., arg_n)
```

`Reflect` has a method:

```
Reflect.trap(target, arg_1, ..., arg_n)
```

If we use `Reflect`, the previous example looks as follows.

```
const handler = {
  deleteProperty(target, propKey) {
    console.log('DELETE ' + propKey);
    return Reflect.deleteProperty(target, propKey);
  },
  has(target, propKey) {
    console.log('HAS ' + propKey);
    return Reflect.has(target, propKey);
  },
}
```

```
    // Other traps: similar
  }
```

19.3.6.2 Improvement: implementing the handler with Proxy

Now what each of the traps does is so similar that we can implement the handler via a Proxy:

```
const handler = new Proxy({}, {
  get(target, trapName, receiver) {
    // Return the handler method named trapName
    return (...args) => {
      console.log(trapName.toUpperCase() + ' ' + args[1]);
      // Forward the operation
      return Reflect[trapName](...args);
    };
  },
});
```

For each trap, the Proxy asks for a handler method via the get operation and we give it one. That is, all of the handler methods can be implemented via the single meta-method get. It was one of the goals for the Proxy API to make this kind of virtualization simple.

Let's use this Proxy-based handler:

```
const target = {};
const proxy = new Proxy(target, handler);

proxy.distance = 450; // set
assert.equal(proxy.distance, 450); // get

// Was `set` operation correctly forwarded to `target`?
assert.equal(
  target.distance, 450);

// Output:
// 'SET distance'
// 'GETOWNPROPERTYDESCRIPTOR distance'
// 'DEFINEPROPERTY distance'
// 'GET distance'
```

19.3.7 Pitfall: not all objects can be wrapped transparently by Proxies

A Proxy object can be seen as intercepting operations performed on its target object – the Proxy wraps the target. The Proxy's handler object is like an observer or listener for the Proxy. It specifies which operations should be intercepted by implementing corresponding methods (get for reading a property, etc.). If the handler method for an operation is missing then that operation is not intercepted. It is simply forwarded to the target.

Therefore, if the handler is the empty object, the Proxy should transparently wrap the target. Alas, that doesn't always work.

19.3.7.1 Wrapping an object affects this

Before we dig deeper, let's quickly review how wrapping a target affects this:

```
const target = {
  myMethod() {
    return {
      thisIsTarget: this === target,
      thisIsProxy: this === proxy,
    };
  }
};
const handler = {};
const proxy = new Proxy(target, handler);
```

If we call `target.myMethod()` directly, this points to target:

```
assert.deepEqual(
  target.myMethod(), {
    thisIsTarget: true,
    thisIsProxy: false,
  });
```

If we invoke that method via the Proxy, this points to proxy:

```
assert.deepEqual(
  proxy.myMethod(), {
    thisIsTarget: false,
    thisIsProxy: true,
  });
```

That is, if the Proxy forwards a method call to the target, this is not changed. As a consequence, the Proxy continues to be in the loop if the target uses `this`, e.g., to make a method call.

19.3.7.2 Objects that can't be wrapped transparently

Normally, Proxies with empty handlers wrap targets transparently: we don't notice that they are there and they don't change the behavior of the targets.

If, however, a target associates information with `this` via a mechanism that is not controlled by Proxies, we have a problem: things fail, because different information is associated depending on whether the target is wrapped or not.

For example, the following class `Person` stores private information in the WeakMap `_name` (more information on this technique is given in [JavaScript for impatient programmers](#)):

```
const _name = new WeakMap();
class Person {
  constructor(name) {
    _name.set(this, name);
  }
  get name() {
```

```

    return _name.get(this);
  }
}

```

Instances of `Person` can't be wrapped transparently:

```

const jane = new Person('Jane');
assert.equal(jane.name, 'Jane');

const proxy = new Proxy(jane, {});
assert.equal(proxy.name, undefined);

```

`jane.name` is different from the wrapped `proxy.name`. The following implementation does not have this problem:

```

class Person2 {
  constructor(name) {
    this._name = name;
  }
  get name() {
    return this._name;
  }
}

const jane = new Person2('Jane');
assert.equal(jane.name, 'Jane');

const proxy = new Proxy(jane, {});
assert.equal(proxy.name, 'Jane');

```

19.3.7.3 Wrapping instances of built-in constructors

Instances of most built-in constructors also use a mechanism that is not intercepted by Proxies. They therefore can't be wrapped transparently, either. We can see that if we use an instance of `Date`:

```

const target = new Date();
const handler = {};
const proxy = new Proxy(target, handler);

assert.throws(
  () => proxy.getFullYear(),
  /^TypeError: this is not a Date object\.$/
);

```

The mechanism that is unaffected by Proxies is called *internal slots*. These slots are property-like storage associated with instances. The specification handles these slots as if they were properties with names in square brackets. For example, the following method is internal and can be invoked on all objects `O`:

```

O.[[GetPrototypeOf]]()

```

In contrast to properties, accessing internal slots is not done via normal “get” and “set” operations. If `.getFullYear()` is invoked via a Proxy, it can’t find the internal slot it needs on this and complains via a `TypeError`.

For Date methods, [the language specification states](#):

Unless explicitly defined otherwise, the methods of the Date prototype object defined below are not generic and the `this` value passed to them must be an object that has a `[[DateValue]]` internal slot that has been initialized to a time value.

19.3.7.4 A work-around

As a work-around, we can change how the handler forwards method calls and selectively set `this` to the target and not the Proxy:

```
const handler = {
  get(target, propKey, receiver) {
    if (propKey === 'getFullYear') {
      return target.getFullYear.bind(target);
    }
    return Reflect.get(target, propKey, receiver);
  },
};
const proxy = new Proxy(new Date('2030-12-24'), handler);
assert.equal(proxy.getFullYear(), 2030);
```

The drawback of this approach is that none of the operations that the method performs on `this` go through the Proxy.

19.3.7.5 Arrays can be wrapped transparently

In contrast to other built-ins, Arrays can be wrapped transparently:

```
const p = new Proxy(new Array(), {});

p.push('a');
assert.equal(p.length, 1);

p.length = 0;
assert.equal(p.length, 0);
```

The reason for Arrays being wrappable is that, even though property access is customized to make `.length` work, Array methods don’t rely on internal slots – they are generic.

19.4 Use cases for Proxies

This section demonstrates what Proxies can be used for. That will give us the opportunity to see the API in action.

19.4.1 Tracing property accesses (get, set)

Let's assume we have a function `tracePropertyAccesses(obj, propKeys)` that logs whenever a property of `obj`, whose key is in the Array `propKeys`, is set or got. In the following code, we apply that function to an instance of the class `Point`:

```
class Point {
  constructor(x, y) {
    this.x = x;
    this.y = y;
  }
  toString() {
    return `Point(${this.x}, ${this.y})`;
  }
}

// Trace accesses to properties `x` and `y`
const point = new Point(5, 7);
const tracedPoint = tracePropertyAccesses(point, ['x', 'y']);
```

Getting and setting properties of the traced object `p` has the following effects:

```
assert.equal(tracedPoint.x, 5);
tracedPoint.x = 21;

// Output:
// 'GET x'
// 'SET x=21'
```

Intriguingly, tracing also works whenever `Point` accesses the properties because this now refers to the traced object, not to an instance of `Point`:

```
assert.equal(
  tracedPoint.toString(),
  'Point(21, 7)');

// Output:
// 'GET x'
// 'GET y'
```

19.4.1.1 Implementing `tracePropertyAccesses()` without Proxies

Without Proxies we'd implement `tracePropertyAccesses()` as follows. We replace each property with a getter and a setter that traces accesses. The setters and getters use an extra object, `propData`, to store the data of the properties. Note that we are destructively changing the original implementation, which means that we are metaprogramming.

```
function tracePropertyAccesses(obj, propKeys, log=console.log) {
  // Store the property data here
  const propData = Object.create(null);

  // Replace each property with a getter and a setter
```

```

propKeys.forEach(function (propKey) {
  propData[propKey] = obj[propKey];
  Object.defineProperty(obj, propKey, {
    get: function () {
      log('GET ' + propKey);
      return propData[propKey];
    },
    set: function (value) {
      log('SET ' + propKey + '=' + value);
      propData[propKey] = value;
    },
  });
});
return obj;
}

```

The parameter `log` makes it easier to unit-test this function:

```

const obj = {};
const logged = [];
tracePropertyAccesses(obj, ['a', 'b'], x => logged.push(x));

obj.a = 1;
assert.equal(obj.a, 1);

obj.c = 3;
assert.equal(obj.c, 3);

assert.deepEqual(
  logged, [
    'SET a=1',
    'GET a',
  ]);

```

19.4.1.2 Implementing `tracePropertyAccesses()` with a Proxy

Proxies give us a simpler solution. We intercept property getting and setting and don't have to change the implementation.

```

function tracePropertyAccesses(obj, propKeys, log=console.log) {
  const propKeySet = new Set(propKeys);
  return new Proxy(obj, {
    get(target, propKey, receiver) {
      if (propKeySet.has(propKey)) {
        log('GET ' + propKey);
      }
      return Reflect.get(target, propKey, receiver);
    },
    set(target, propKey, value, receiver) {
      if (propKeySet.has(propKey)) {

```

```

        log('SET '+propKey+'='+value);
    }
    return Reflect.set(target, propKey, value, receiver);
},
});
}

```

19.4.2 Warning about unknown properties (get, set)

When it comes to accessing properties, JavaScript is very forgiving. For example, if we try to read a property and misspell its name, we don't get an exception – we get the result `undefined`.

We can use Proxies to get an exception in such a case. This works as follows. We make the Proxy a prototype of an object. If a property isn't found in the object, the get trap of the Proxy is triggered:

- If the property doesn't even exist in the prototype chain after the Proxy, it really is missing and we throw an exception.
- Otherwise, we return the value of the inherited property. We do so by forwarding the get operation to the target (the Proxy gets its prototype from the target).

This is an implementation of this approach:

```

const propertyCheckerHandler = {
  get(target, propKey, receiver) {
    // Only check string property keys
    if (typeof propKey === 'string' && !(propKey in target)) {
      throw new ReferenceError('Unknown property: ' + propKey);
    }
    return Reflect.get(target, propKey, receiver);
  }
};
const PropertyChecker = new Proxy({}, propertyCheckerHandler);

```

Let's use PropertyChecker for an object:

```

const jane = {
  __proto__: PropertyChecker,
  name: 'Jane',
};

// Own property:
assert.equal(
  jane.name,
  'Jane');

// Typo:
assert.throws(
  () => jane.nmae,
  /^ReferenceError: Unknown property: nmae$/);

```

```
// Inherited property:
assert.equal(
  jane.toString(),
  '[object Object]');
```

19.4.2.1 PropertyChecker as a class

If we turn PropertyChecker into a constructor, we can use it for classes via extends:

```
// We can't change .prototype of classes, so we are using a function
function PropertyChecker2() {}
PropertyChecker2.prototype = new Proxy({}, propertyCheckerHandler);

class Point extends PropertyChecker2 {
  constructor(x, y) {
    super();
    this.x = x;
    this.y = y;
  }
}

const point = new Point(5, 7);
assert.equal(point.x, 5);
assert.throws(
  () => point.z,
  /^ReferenceError: Unknown property: z/);
```

This is the prototype chain of point:

```
const p = Object.getPrototypeOf.bind(Object);
assert.equal(p(point), Point.prototype);
assert.equal(p(p(point)), PropertyChecker2.prototype);
assert.equal(p(p(p(point))), Object.prototype);
```

19.4.2.2 Preventing the accidental creation of properties

If we are worried about accidentally *creating* properties, we have two options:

- We can either wrap a Proxy around objects that traps set.
- Or we can make an object obj non-extensible via `Object.preventExtensions(obj)`, which means that JavaScript doesn't let us add new (own) properties to obj.

19.4.3 Negative Array indices (get)

Some Array methods let us refer to the last element via `-1`, to the second-to-last element via `-2`, etc. For example:

```
> ['a', 'b', 'c'].slice(-1)
[ 'c' ]
```

Alas, that doesn't work when accessing elements via the bracket operator (`[]`). We can, however, use Proxies to add that capability. The following function `createArray()` creates Arrays that support negative indices. It does so by wrapping Proxies around Array instances. The Proxies intercept the `get` operation that is triggered by the bracket operator.

```
function createArray(...elements) {
  const handler = {
    get(target, propKey, receiver) {
      if (typeof propKey === 'string') {
        const index = Number(propKey);
        if (index < 0) {
          propKey = String(target.length + index);
        }
      }
      return Reflect.get(target, propKey, receiver);
    }
  };
  // Wrap a proxy around the Array
  return new Proxy(elements, handler);
}
const arr = createArray('a', 'b', 'c');
assert.equal(
  arr[-1], 'c');
assert.equal(
  arr[0], 'a');
assert.equal(
  arr.length, 3);
```

19.4.4 Data binding (set)

Data binding is about syncing data between objects. One popular use case are widgets based on the MVC (Model View Controller) pattern: With data binding, the *view* (the widget) stays up-to-date if we change the *model* (the data visualized by the widget).

To implement data binding, we have to observe and react to changes made to an object. The following code snippet is a sketch of how observing changes could work for Arrays.

```
function createObservedArray(callback) {
  const array = [];
  return new Proxy(array, {
    set(target, propertyKey, value, receiver) {
      callback(propertyKey, value);
      return Reflect.set(target, propertyKey, value, receiver);
    }
  });
}
const observedArray = createObservedArray(
  (key, value) => console.log(
    `${JSON.stringify(key)} = ${JSON.stringify(value)}`));
```

```

observedArray.push('a');

// Output:
// '"0" = "a"'
// '"length" = 1'

```

19.4.5 Accessing a restful web service (method calls)

A Proxy can be used to create an object on which arbitrary methods can be invoked. In the following example, the function `createWebService()` creates one such object, `service`. Invoking a method on `service` retrieves the contents of the web service resource with the same name. Retrieval is handled via a Promise.

```

const service = createWebService('http://example.com/data');
// Read JSON data in http://example.com/data/employees
service.employees().then((jsonStr) => {
  const employees = JSON.parse(jsonStr);
  // ...
});

```

The following code is a quick and dirty implementation of `createWebService` without Proxies. We need to know beforehand what methods will be invoked on `service`. The parameter `propKeys` provides us with that information; it holds an Array with method names.

```

function createWebService(baseUrl, propKeys) {
  const service = {};
  for (const propKey of propKeys) {
    service[propKey] = () => {
      return httpGet(baseUrl + '/' + propKey);
    };
  }
  return service;
}

```

With Proxies, `createWebService()` is simpler:

```

function createWebService(baseUrl) {
  return new Proxy({}, {
    get(target, propKey, receiver) {
      // Return the method to be called
      return () => httpGet(baseUrl + '/' + propKey);
    }
  });
}

```

Both implementations use the following function to make HTTP GET requests (how it works is explained in [JavaScript for impatient programmers](#)).

```

function httpGet(url) {
  return new Promise(
    (resolve, reject) => {

```

```

const xhr = new XMLHttpRequest();
xhr.onload = () => {
  if (xhr.status === 200) {
    resolve(xhr.responseText); // (A)
  } else {
    // Something went wrong (404, etc.)
    reject(new Error(xhr.statusText)); // (B)
  }
}
xhr.onerror = () => {
  reject(new Error('Network error')); // (C)
};
xhr.open('GET', url);
xhr.send();
});
}

```

19.4.6 Revocable references

Revocable references work as follows: A client is not allowed to access an important resource (an object) directly, only via a reference (an intermediate object, a wrapper around the resource). Normally, every operation applied to the reference is forwarded to the resource. After the client is done, the resource is protected by *revoking* the reference, by switching it off. Henceforth, applying operations to the reference throws exceptions and nothing is forwarded, anymore.

In the following example, we create a revocable reference for a resource. We then read one of the resource's properties via the reference. That works, because the reference grants us access. Next, we revoke the reference. Now the reference doesn't let us read the property, anymore.

```

const resource = { x: 11, y: 8 };
const { reference, revoke } = createRevocableReference(resource);

// Access granted
assert.equal(reference.x, 11);

revoke();

// Access denied
assert.throws(
  () => reference.x,
  /^TypeError: Cannot perform 'get' on a proxy that has been revoked/
);

```

Proxies are ideally suited for implementing revocable references, because they can intercept and forward operations. This is a simple Proxy-based implementation of `createRevocableReference`:

```

function createRevocableReference(target) {

```

```

let enabled = true;
return {
  reference: new Proxy(target, {
    get(target, propKey, receiver) {
      if (!enabled) {
        throw new TypeError(
          `Cannot perform 'get' on a proxy that has been revoked`);
      }
      return Reflect.get(target, propKey, receiver);
    },
    has(target, propKey) {
      if (!enabled) {
        throw new TypeError(
          `Cannot perform 'has' on a proxy that has been revoked`);
      }
      return Reflect.has(target, propKey);
    },
    // (Remaining methods omitted)
  }),
  revoke: () => {
    enabled = false;
  },
};
}

```

The code can be simplified via the Proxy-as-handler technique from the previous section. This time, the handler basically is the Reflect object. Thus, the get trap normally returns the appropriate Reflect method. If the reference has been revoked, a `TypeError` is thrown, instead.

```

function createRevocableReference(target) {
  let enabled = true;
  const handler = new Proxy({}, {
    get(_handlerTarget, trapName, receiver) {
      if (!enabled) {
        throw new TypeError(
          `Cannot perform '${trapName}' on a proxy`
          + ` that has been revoked`);
      }
      return Reflect[trapName];
    }
  });
  return {
    reference: new Proxy(target, handler),
    revoke: () => {
      enabled = false;
    },
  };
}

```

However, we don't have to implement revocable references ourselves because Proxies can be revoked. This time, the revoking happens in the Proxy, not in the handler. All the handler has to do is forward every operation to the target. As we have seen that happens automatically if the handler doesn't implement any traps.

```
function createRevocableReference(target) {  
  const handler = {}; // forward everything  
  const { proxy, revoke } = Proxy.revocable(target, handler);  
  return { reference: proxy, revoke };  
}
```

19.4.6.1 Membranes

Membranes build on the idea of revocable references: Libraries for safely running untrusted code wrap a membrane around that code to isolate it and to keep the rest of the system safe. Objects pass the membrane in two directions:

- The untrusted code may receive objects (“dry objects”) from the outside.
- Or it may hand objects (“wet objects”) to the outside.

In both cases, revocable references are wrapped around the objects. Objects returned by wrapped functions or methods are also wrapped. Additionally, if a wrapped wet object is passed back into a membrane, it is unwrapped.

Once the untrusted code is done, all of the revocable references are revoked. As a result, none of its code on the outside can be executed anymore and outside objects that it references, cease to work as well. The [Caja Compiler](#) is “a tool for making third party HTML, CSS and JavaScript safe to embed in your website”. It uses membranes to achieve this goal.

19.4.7 Implementing the DOM in JavaScript

The browsers' Document Object Model (DOM) is usually implemented as a mix of JavaScript and C++. Implementing it in pure JavaScript is useful for:

- Emulating a browser environment, e.g. to manipulate HTML in Node.js. [jsdom](#) is one library that does that.
- Making the DOM faster (switching between JavaScript and C++ costs time).

Alas, the standard DOM can do things that are not easily replicated in JavaScript. For example, most DOM collections are live views on the current state of the DOM that change dynamically whenever the DOM changes. As a result, pure JavaScript implementations of the DOM are not very efficient. One of the reasons for adding Proxies to JavaScript was to enable more efficient DOM implementations.

19.4.8 More use cases

There are more use cases for Proxies. For example:

- Remoting: Local placeholder objects forward method invocations to remote objects. This use case is similar to the web service example.

- Data access objects for databases: Reading and writing to the object reads and writes to the database. This use case is similar to the web service example.
- Profiling: Intercept method invocations to track how much time is spent in each method. This use case is similar to the tracing example.

19.4.9 Libraries that are using Proxies

- *Immer* (by Michel Weststrate) helps with non-destructively updating data. The changes that should be applied are specified by invoking methods, setting properties, setting Array elements, etc. of a (potentially nested) *draft state*. Draft states are implemented via Proxies.
- *MobX* lets you observe changes to data structures such as objects, Arrays and class instances. That is implemented via Proxies.
- *Alpine.js* (by Caleb Porzio) is a frontend library that implements data binding via Proxies.
- *on-change* (by Sindre Sorhus) observes changes to an object (via Proxies) and reports them.
- *Env utility* (by Nicholas C. Zakas) lets you access environment variables via properties and throws exceptions if they don't exist. That is implemented via Proxies.
- *LDflex* (by Ruben Verborgh and Ruben Taelman) provides a query language for Linked Data (think Semantic Web). The fluid query API is implemented via Proxies.

19.5 The design of the Proxy API

In this section, we go deeper into how Proxies work and why they work that way.

19.5.1 Stratification: keeping base level and meta level separate

Firefox used to support a limited form of interceding metaprogramming for a while: If an object `0` had a method named `__noSuchMethod__`, it was notified whenever a method was invoked on `0` that didn't exist. The following code demonstrates how that worked:

```
const calc = {
  __noSuchMethod__: function (methodName, args) {
    switch (methodName) {
      case 'plus':
        return args.reduce((a, b) => a + b);
      case 'times':
        return args.reduce((a, b) => a * b);
      default:
        throw new TypeError('Unsupported: ' + methodName);
    }
  }
};
```

```
// All of the following method calls are implemented via
// __noSuchMethod__().
assert.equal(
  calc.plus(3, 5, 2), 10);
assert.equal(
  calc.times(2, 3, 4), 24);

assert.equal(
  calc.plus('Parts', ' of ', 'a', ' string'),
  'Parts of a string');
```

Thus, `__noSuchMethod__` works similarly to a Proxy trap. In contrast to Proxies, the trap is an own or inherited method of the object whose operations we want to intercept. The problem with that approach is that base level (normal methods) and meta level (`__noSuchMethod__`) are mixed. Base-level code may accidentally invoke or see a meta level method and there is the possibility of accidentally defining a meta level method.

Even in standard ECMAScript, base level and meta level are sometimes mixed. For example, the following metaprogramming mechanisms can fail, because they exist at the base level:

- `obj.hasOwnProperty(propKey)`: This call can fail if a property in the prototype chain overrides the built-in implementation. For example, in the following code, `obj` causes a failure:

```
const obj = { hasOwnProperty: null };
assert.throws(
  () => obj.hasOwnProperty('width'),
  /^TypeError: obj.hasOwnProperty is not a function/
);
```

These are safe ways of invoking `.hasOwnProperty()`:

```
assert.equal(
  Object.prototype.hasOwnProperty.call(obj, 'width'), false);

// Abbreviated version:
assert.equal(
  {}.hasOwnProperty.call(obj, 'width'), false);
```

- `func.call(...)`, `func.apply(...)`: For both methods, problem and solution are the same as with `.hasOwnProperty()`.
- `obj.__proto__`: In plain objects, `__proto__` is a special property that lets us get and set the prototype of the receiver. Hence, when we use plain objects as dictionaries, we must **avoid `__proto__` as a property key**.

By now, it should be obvious that making (base level) property keys special is problematic. Therefore, Proxies are *stratified*: Base level (the Proxy object) and meta level (the handler object) are separate.

19.5.2 Virtual objects versus wrappers

Proxies are used in two roles:

- As *wrappers*, they *wrap* their targets, they control access to them. Examples of wrappers are: revocable resources and tracing via Proxies.
- As *virtual objects*, they are simply objects with special behavior and their targets don't matter. An example is a Proxy that forwards method calls to a remote object.

An earlier design of the Proxy API conceived Proxies as purely virtual objects. However, it turned out that even in that role, a target was useful, to enforce invariants (which are explained later) and as a fallback for traps that the handler doesn't implement.

19.5.3 Transparent virtualization and handler encapsulation

Proxies are shielded in two ways:

- It is impossible to determine whether an object is a Proxy or not (*transparent virtualization*).
- We can't access a handler via its Proxy (*handler encapsulation*).

Both principles give Proxies considerable power for impersonating other objects. One reason for enforcing *invariants* (as explained later) is to keep that power in check.

If we do need a way to tell Proxies apart from non-Proxies, we have to implement it ourselves. The following code is a module `lib.mjs` that exports two functions: one of them creates Proxies, the other one determines whether an object is one of those Proxies.

```
// lib.mjs
const proxies = new WeakSet();

export function createProxy(obj) {
  const handler = {};
  const proxy = new Proxy(obj, handler);
  proxies.add(proxy);
  return proxy;
}

export function isProxy(obj) {
  return proxies.has(obj);
}
```

This module uses the data structure `WeakSet` for keeping track of Proxies. `WeakSet` is ideally suited for this purpose, because it doesn't prevent its elements from being garbage-collected.

The next example shows how `lib.mjs` can be used.

```
// main.mjs
import { createProxy, isProxy } from './lib.mjs';

const proxy = createProxy({});
```

```
assert.equal(isProxy(proxy), true);
assert.equal(isProxy({}), false);
```

19.5.4 The meta object protocol and Proxy traps

In this section, we examine how JavaScript is structured internally and how the set of Proxy traps was chosen.

In the context of programming languages and API design, a *protocol* is a set of interfaces plus rules for using them. The ECMAScript specification describes how to execute JavaScript code. It includes a [protocol for handling objects](#). This protocol operates at a meta level and is sometimes called the *meta object protocol* (MOP). The JavaScript MOP consists of own internal methods that all objects have. “Internal” means that they exist only in the specification (JavaScript engines may or may not have them) and are not accessible from JavaScript. The names of internal methods are written in double square brackets.

The internal method for getting properties is called `.[[Get]]()`. If we use double underscores instead of double brackets, this method would roughly be implemented as follows in JavaScript.

```
// Method definition
__Get__(propKey, receiver) {
  const desc = this.__GetOwnProperty__(propKey);
  if (desc === undefined) {
    const parent = this.__GetPrototypeOf__();
    if (parent === null) return undefined;
    return parent.__Get__(propKey, receiver); // (A)
  }
  if ('value' in desc) {
    return desc.value;
  }
  const getter = desc.get;
  if (getter === undefined) return undefined;
  return getter.__Call__(receiver, []);
}
```

The MOP methods called in this code are:

- `.[[GetOwnProperty]]` (trap `getOwnPropertyDescriptor`)
- `.[[GetPrototypeOf]]` (trap `getPrototypeOf`)
- `.[[Get]]` (trap `get`)
- `.[[Call]]` (trap `apply`)

In line A we can see why Proxies in a prototype chain find out about `get` if a property isn’t found in an “earlier” object: If there is no own property whose key is `propKey`, the search continues in the prototype parent of this.

Fundamental versus derived operations. We can see that `.[[Get]]()` calls other MOP operations. Operations that do that are called *derived*. Operations that don’t depend on other operations are called *fundamental*.

19.5.4.1 The meta object protocol of Proxies

The [meta object protocol of Proxies](#) is different from that of normal objects. For normal objects, derived operations call other operations. For Proxies, each operation (regardless of whether it is fundamental or derived) is either intercepted by a handler method or forwarded to the target.

Which operations should be interceptable via Proxies?

- One possibility is to only provide traps for fundamental operations.
- The alternative is to include some derived operations.

The upside of doing the latter is that it increases performance and is more convenient. For example, if there weren't a trap for `get`, we'd have to implement its functionality via `getOwnPropertyDescriptor`.

A downside of including derived traps is that that can lead to Proxies behaving inconsistently. For example, `get` may return a value that is different from the value in the descriptor returned by `getOwnPropertyDescriptor`.

19.5.4.2 Selective interception: Which operations should be interceptable?

Interception by Proxies is *selective*: we can't intercept every language operation. Why were some operations excluded? Let's look at two reasons.

First, *stable* operations are not well suited for interception. An operation is *stable* if it always produces the same results for the same arguments. If a Proxy can trap a stable operation, it can become unstable and thus unreliable. [Strict equality](#) (`===`) is one such stable operation. It can't be trapped and its result is computed by treating the Proxy itself as just another object. Another way of maintaining stability is by applying an operation to the target instead of the Proxy. As explained later, when we look at how invariants are enforced for Proxies, this happens when `Object.getPrototypeOf()` is applied to a Proxy whose target is non-extensible.

A second reason for not making more operations interceptable is that interception means executing custom code in situations where that normally isn't possible. The more this interleaving of code happens, the harder it is to understand and debug a program. It also affects performance negatively.

19.5.4.3 Traps: `get` versus `invoke`

If we want to create virtual methods via Proxies, we have to return functions from a `get` trap. That raises the question: why not introduce an extra trap for method invocations (e.g. `invoke`)? That would enable us to distinguish between:

- Getting properties via `obj.prop` (trap `get`)
- Invoking methods via `obj.prop()` (trap `invoke`)

There are two reasons for not doing so.

First, not all implementations distinguish between `get` and `invoke`. For example, [Apple's JavaScriptCore](#) doesn't.

Second, extracting a method and invoking it later via `.call()` or `.apply()` should have the same effect as invoking the method via `dispatch`. In other words, the following two variants should work equivalently. If there were an extra trap `invoke`, then that equivalence would be harder to maintain.

```
// Variant 1: call via dynamic dispatch
const result1 = obj.m();

// Variant 2: extract and call directly
const m = obj.m;
const result2 = m.call(obj);
```

19.5.4.3.1 Use cases for `invoke`

Some things can only be done if we are able to distinguish between `get` and `invoke`. Those things are therefore impossible with the current Proxy API. Two examples are: auto-binding and intercepting missing methods. Let's examine how one would implement them if Proxies supported `invoke`.

Auto-binding. By making a Proxy the prototype of an object `obj`, we can automatically bind methods:

- Retrieving the value of a method `m` via `obj.m` returns a function whose `this` is bound to `obj`.
- `obj.m()` performs a method call.

Auto-binding helps with using methods as callbacks. For example, variant 2 from the previous example becomes simpler:

```
const boundMethod = obj.m;
const result = boundMethod();
```

Intercepting missing methods. `invoke` lets a Proxy emulate the previously mentioned `__noSuchMethod__` mechanism. The Proxy would again become the prototype of an object `obj`. It would react differently depending on how an unknown property `prop` is accessed:

- If we read that property via `obj.prop`, no interception happens and `undefined` is returned.
- If we make the method call `obj.prop()` then the Proxy intercepts and, e.g., notifies a callback.

19.5.5 Enforcing invariants for Proxies

Before we look at what invariants are and how they are enforced for Proxies, let's review how objects can be protected via non-extensibility and non-configurability.

19.5.5.1 Protecting objects

There are two ways of protecting objects:

- Non-extensibility protects objects: If an object is non-extensible, we can't add properties and we can't change its prototype.

- Non-configurability protects properties (or rather, their attributes):
 - The boolean attribute `writable` controls whether a property’s value can be changed.
 - The boolean attribute `configurable` controls whether a property’s attributes can be changed.

For more information on this topic, see §10 “Protecting objects from being changed”.

19.5.5.2 Enforcing invariants

Traditionally, non-extensibility and non-configurability are:

- Universal: They work for all objects.
- Monotonic: Once switched on, they can’t be switched off again.

These and other characteristics that remain unchanged in the face of language operations are called *invariants*. It is easy to violate invariants via Proxies because they are not intrinsically bound by non-extensibility etc. The Proxy API prevents that from happening by checking the target object and the results of handler methods.

The next two subsections describe four invariants. An exhaustive list of invariants is given at the end of this chapter.

19.5.5.3 Two invariants that are enforced via the target object

The following two invariants involve non-extensibility and non-configurability. These are enforced by using the target object for bookkeeping: results returned by handler methods have to be mostly in sync with the target object.

- Invariant: If `Object.preventExtensions(obj)` returns `true` then all future calls must return `false` and `obj` must now be non-extensible.
 - Enforced for Proxies by throwing a `TypeError` if the handler returns `true`, but the target object is not extensible.
- Invariant: Once an object has been made non-extensible, `Object.isExtensible(obj)` must always return `false`.
 - Enforced for Proxies by throwing a `TypeError` if the result returned by the handler is not the same (after coercion) as `Object.isExtensible(target)`.

19.5.5.4 Two invariants that are enforced by checking return values

The following two invariants are enforced by checking return values:

- Invariant: `Object.isExtensible(obj)` must return a boolean.
 - Enforced for Proxies by coercing the value returned by the handler to a boolean.
- Invariant: `Object.getOwnPropertyDescriptor(obj, ...)` must return an object or `undefined`.
 - Enforced for Proxies by throwing a `TypeError` if the handler doesn’t return an appropriate value.

19.5.5.5 Benefits of invariants

Enforcing invariants has the following benefits:

- Proxies work like all other objects with regard to extensibility and configurability. Therefore, universality is maintained. This is achieved without preventing Proxies from virtualizing (impersonating) protected objects.
- A protected object can't be misrepresented by wrapping a Proxy around it. Misrepresentation can be caused by bugs or by malicious code.

The next two sections give examples of invariants being enforced.

19.5.5.6 Example: the prototype of a non-extensible target must be represented faithfully

In response to the `getPrototypeOf` trap, the Proxy must return the target's prototype if the target is non-extensible.

To demonstrate this invariant, let's create a handler that returns a prototype that is different from the target's prototype:

```
const fakeProto = {};
const handler = {
  getPrototypeOf(t) {
    return fakeProto;
  }
};
```

Faking the prototype works if the target is extensible:

```
const extensibleTarget = {};
const extProxy = new Proxy(extensibleTarget, handler);

assert.equal(
  Object.getPrototypeOf(extProxy), fakeProto);
```

We do, however, get an error if we fake the prototype for a non-extensible object.

```
const nonExtensibleTarget = {};
Object.preventExtensions(nonExtensibleTarget);
const nonExtProxy = new Proxy(nonExtensibleTarget, handler);

assert.throws(
  () => Object.getPrototypeOf(nonExtProxy),
  {
    name: 'TypeError',
    message: "'getPrototypeOf' on proxy: proxy target is"
      + " non-extensible but the trap did not return its"
      + " actual prototype",
  });
```

19.5.5.7 Example: non-writable non-configurable target properties must be represented faithfully

If the target has a non-writable non-configurable property, then the handler must return that property's value in response to a get trap. To demonstrate this invariant, let's create a handler that always returns the same value for properties.

```
const handler = {
  get(target, propKey) {
    return 'abc';
  }
};
const target = Object.defineProperties(
  {}, {
    manufacturer: {
      value: 'Iso Autoveicoli',
      writable: true,
      configurable: true
    },
    model: {
      value: 'Isetta',
      writable: false,
      configurable: false
    },
  });
const proxy = new Proxy(target, handler);
```

Property `target.manufacturer` is not both non-writable and non-configurable, which means that the handler is allowed to pretend that it has a different value:

```
assert.equal(
  proxy.manufacturer, 'abc');
```

However, property `target.model` is both non-writable and non-configurable. Therefore, we can't fake its value:

```
assert.throws(
  () => proxy.model,
  {
    name: 'TypeError',
    message: "'get' on proxy: property 'model' is a read-only and"
      + " non-configurable data property on the proxy target but"
      + " the proxy did not return its actual value (expected"
      + " 'Isetta' but got 'abc')",
  });
```

19.6 FAQ: Proxies

19.6.1 Where is the `enumerate` trap?

ECMAScript 6 originally had a trap `enumerate` that was triggered by `for-in` loops. But it was recently removed, to simplify Proxies. `Reflect.enumerate()` was removed, as well. ([Source: TC39 notes](#))

19.7 Reference: the Proxy API

This section is a quick reference for the Proxy API:

- The global object `Proxy`
- The global object `Reflect`

The reference uses the following custom type:

```
type PropertyKey = string | symbol;
```

19.7.1 Creating Proxies

There are two ways to create Proxies:

- `const proxy = new Proxy(target, handler)`
Creates a new Proxy object with the given target and the given handler.
- `const {proxy, revoke} = Proxy.revocable(target, handler)`
Creates a Proxy that can be revoked via the function `revoke`. `revoke` can be called multiple times, but only the first call has an effect and switches proxy off. Afterwards, any operation performed on proxy leads to a `TypeError` being thrown.

19.7.2 Handler methods

This subsection explains what traps can be implemented by handlers and what operations trigger them. Several traps return boolean values. For the traps `has` and `isExtensible`, the boolean is the result of the operation. For all other traps, the boolean indicates whether the operation succeeded or not.

Traps for all objects:

- `defineProperty(target, propKey, propDesc): boolean`
– `Object.defineProperty(proxy, propKey, propDesc)`
- `deleteProperty(target, propKey): boolean`
– `delete proxy[propKey]`
– `delete proxy.someProp`
- `get(target, propKey, receiver): any`
– `receiver[propKey]`
– `receiver.someProp`
- `getOwnPropertyDescriptor(target, propKey): undefined|PropDesc`
– `Object.getOwnPropertyDescriptor(proxy, propKey)`
- `getPrototypeOf(target): null|object`
– `Object.getPrototypeOf(proxy)`

- `has(target, propKey): boolean`
 - `propKey` in `proxy`
- `isExtensible(target): boolean`
 - `Object.isExtensible(proxy)`
- `ownKeys(target): Array<PropertyKey>`
 - `Object.getOwnPropertyNames(proxy)` (only uses string keys)
 - `Object.getOwnPropertySymbols(proxy)` (only uses symbol keys)
 - `Object.keys(proxy)` (only uses enumerable string keys; enumerability is checked via `Object.getOwnPropertyDescriptor`)
- `preventExtensions(target): boolean`
 - `Object.preventExtensions(proxy)`
- `set(target, propKey, value, receiver): boolean`
 - `receiver[propKey] = value`
 - `receiver.someProp = value`
- `setPrototypeOf(target, proto): boolean`
 - `Object.setPrototypeOf(proxy, proto)`

Traps for functions (available if target is a function):

- `apply(target, thisArgument, argumentsList): any`
 - `proxy.apply(thisArgument, argumentsList)`
 - `proxy.call(thisArgument, ...argumentsList)`
 - `proxy(...argumentsList)`
- `construct(target, argumentsList, newTarget): object`
 - `new proxy(...argumentsList)`

19.7.2.1 Fundamental operations versus derived operations

The following operations are *fundamental*, they don't use other operations to do their work: `apply`, `defineProperty`, `deleteProperty`, `getOwnPropertyDescriptor`, `getPrototypeOf`, `isExtensible`, `ownKeys`, `preventExtensions`, `setPrototypeOf`

All other operations are *derived*, they can be implemented via fundamental operations. For example, `get` can be implemented by iterating over the prototype chain via `getPrototypeOf` and calling `getOwnPropertyDescriptor` for each chain member until either an own property is found or the chain ends.

19.7.3 Invariants of handler methods

Invariants are safety constraints for handlers. This subsection documents what invariants are enforced by the Proxy API and how. Whenever we read “the handler must do X” below, it means that a `TypeError` is thrown if it doesn't. Some invariants restrict return values, others restrict parameters. The correctness of a trap's return value is ensured in two ways:

- If a boolean is expected, coercion is used to convert non-booleans to legal values.
- In all other cases, an illegal value cause a `TypeError`.

This is the complete list of invariants that are enforced:

- `apply(target, thisArgument, argumentsList): any`

- No invariants are enforced.
 - Only active if the target is callable.
- `construct(target, argumentsList, newTarget): object`
 - The result returned by the handler must be an object (not null or any other primitive value).
 - Only active if the target is constructible.
- `defineProperty(target, propKey, propDesc): boolean`
 - If the target is not extensible, then we can't add new properties.
 - If `propDesc` sets the attribute `configurable` to false, then the target must have a non-configurable own property whose key is `propKey`.
 - If `propDesc` sets both attributes `configurable` and `writable` to false, then the target must have an own property with the key `propKey` that is non-configurable and non-writable.
 - If the target has an own property with the key `propKey`, then `propDesc` must be compatible with that property: If we redefine the target property with the descriptor, no exception must be thrown.
- `deleteProperty(target, propKey): boolean`
 - A property can't be reported as deleted if:
 - * The target has a non-configurable own property with key `propKey`.
 - * The target is non-extensible and has a own property with key `propKey`.
- `get(target, propKey, receiver): any`
 - If the target has an own, non-writable, non-configurable data property whose key is `propKey`, then the handler must return that property's value.
 - If the target has an own, non-configurable, getter-less accessor property, then the handler must return undefined.
- `getOwnPropertyDescriptor(target, propKey): undefined|PropDesc`
 - The handler must return either undefined or an object.
 - Non-configurable own properties of the target can't be reported as non-existent by the handler.
 - If the target is non-extensible, then exactly the target's own properties must be reported by the handler as existing.
 - If the handler reports a property as non-configurable, then that property must be a non-configurable own property of the target.
 - If the handler reports a property as non-configurable and non-writable, then that property must be a non-configurable non-writable own property of the target.
- `getPrototypeOf(target): null|object`
 - The result must be either null or an object.
 - If the target object is not extensible, then the handler must return the prototype of the target object.
- `has(target, propKey): boolean`
 - Non-configurable own properties of the target can't be reported as non-existent.
 - If the target is non-extensible, then no own property of the target can be reported as non-existent.
- `isExtensible(target): boolean`
 - After coercion to boolean, the value returned by the handler must be the same as `target.isExtensible()`.

- `ownKeys(target): Array<PropertyKey>`
 - The handler must return an object, which treated as Array-like and converted into an Array.
 - The resulting Array must not contain duplicate entries.
 - Each element of the result must be either a string or a symbol.
 - The result must contain the keys of all non-configurable own properties of the target.
 - If the target is not extensible, then the result must contain exactly the keys of the own properties of the target (and no other values).
- `preventExtensions(target): boolean`
 - The handler must only return a truthy value (indicating a successful change) if `target.isExtensible()` is false.
- `set(target, propKey, value, receiver): boolean`
 - The property can't be changed if the target has a non-writable, non-configurable data property whose key is `propKey`. In that case, `value` must be the value of that property or a `TypeError` is thrown.
 - The property can't be set in any way if the corresponding own target property is a non-configurable accessor without a setter.
- `setPrototypeOf(target, proto): boolean`
 - If the target is not extensible, the prototype can't be changed. This is enforced as follows: If the target is not extensible and the handler returns a truthy value (indicating a successful change), then `proto` must be the same as the prototype of the target. Otherwise, a `TypeError` is thrown.



Invariants in the ECMAScript specification

In the spec, the invariants are listed in section [“Proxy Object Internal Methods and Internal Slots”](#).

19.7.4 Operations that affect the prototype chain

The following operations of normal objects perform operations on objects in the prototype chain. Therefore, if one of the objects in that chain is a Proxy, its traps are triggered. The specification implements the operations as internal own methods (that are not visible to JavaScript code). But in this section, we pretend that they are normal methods that have the same names as the traps. The parameter `target` becomes the receiver of the method call.

- `target.get(propertyKey, receiver)`
If `target` has no own property with the given key, `get` is invoked on the prototype of `target`.
- `target.has(propertyKey)`
Similarly to `get`, `has` is invoked on the prototype of `target` if `target` has no own property with the given key.
- `target.set(propertyKey, value, receiver)`
Similarly to `get`, `set` is invoked on the prototype of `target` if `target` has no own property with the given key.

All other operations only affect own properties, they have no effect on the prototype chain.



Internal operations in the ECMAScript specification

In the spec, these (and other) operations are described in section “[Ordinary Object Internal Methods and Internal Slots](#)”.

19.7.5 Reflect

The global object `Reflect` implements all interceptable operations of the JavaScript meta object protocol as methods. The names of those methods are the same as those of the handler methods, which, *as we have seen*, helps with forwarding operations from the handler to the target.

- `Reflect.apply(target, thisArgument, argumentsList)`: any
Similar to `Function.prototype.apply()`.
- `Reflect.construct(target, argumentsList, newTarget=target)`: object
The new operator as a function. `target` is the constructor to invoke, the optional parameter `newTarget` points to the constructor that started the current chain of constructor calls.
- `Reflect.defineProperty(target, propertyKey, propDesc)`: boolean
Similar to `Object.defineProperty()`.
- `Reflect.deleteProperty(target, propertyKey)`: boolean
The delete operator as a function. It works slightly differently, though: It returns `true` if it successfully deleted the property or if the property never existed. It returns `false` if the property could not be deleted and still exists. The only way to protect properties from deletion is by making them non-configurable. In sloppy mode, the delete operator returns the same results. But in strict mode, it throws a `TypeError` instead of returning `false`.
- `Reflect.get(target, propertyKey, receiver=target)`: any
A function that gets properties. The optional parameter `receiver` points to the object where the getting started. It is needed when `get` reaches a getter later in the prototype chain. Then it provides the value for `this`.
- `Reflect.getOwnPropertyDescriptor(target, propertyKey)`: `undefined|PropDesc`
Same as `Object.getOwnPropertyDescriptor()`.
- `Reflect.getPrototypeOf(target)`: `null|object`
Same as `Object.getPrototypeOf()`.
- `Reflect.has(target, propertyKey)`: boolean
The `in` operator as a function.
- `Reflect.isExtensible(target)`: boolean
Same as `Object.isExtensible()`.
- `Reflect.ownKeys(target)`: `Array<PropertyKey>`
Returns all own property keys in an Array: the string keys and symbol keys of all

own enumerable and non-enumerable properties.

- `Reflect.preventExtensions(target): boolean`
Similar to `Object.preventExtensions()`.
- `Reflect.set(target, propertyKey, value, receiver=target): boolean`
A function that sets properties.
- `Reflect.setPrototypeOf(target, proto): boolean`
The new standard way of setting the prototype of an object. The current non-standard way, that works in most engines, is to set the special property `__proto__`.

Several methods have boolean results. For `.has()` and `.isExtensible()`, they are the results of the operation. For the remaining methods, they indicate whether the operation succeeded.

19.7.5.1 Use cases for Reflect besides forwarding

Apart from forwarding operations, **why is Reflect useful [4]**?

- Different return values: Reflect duplicates the following methods of `Object`, but its methods return booleans indicating whether the operation succeeded (where the `Object` methods return the object that was modified).
 - `Object.defineProperty(obj, propKey, propDesc): object`
 - `Object.preventExtensions(obj): object`
 - `Object.setPrototypeOf(obj, proto): object`
- Operators as functions: The following Reflect methods implement functionality that is otherwise only available via operators:
 - `Reflect.construct(target, argumentsList, newTarget=target): object`
 - `Reflect.deleteProperty(target, propertyKey): boolean`
 - `Reflect.get(target, propertyKey, receiver=target): any`
 - `Reflect.has(target, propertyKey): boolean`
 - `Reflect.set(target, propertyKey, value, receiver=target): boolean`
- Shorter version of `apply()`: If we want to be completely safe about invoking the method `apply()` on a function, we can't do so via dynamic dispatch, because the function may have an own property with the key `'apply'`:

```
func.apply(thisArg, argArray) // not safe
Function.prototype.apply.call(func, thisArg, argArray) // safe
```

Using `Reflect.apply()` is shorter than the safe version:

```
Reflect.apply(func, thisArg, argArray)
```

- No exceptions when deleting properties: the `delete` operator throws in strict mode if we try to delete a non-configurable own property. `Reflect.deleteProperty()` returns `false` in that case.

19.7.5.2 `Object.*` versus `Reflect.*`

Going forward, `Object` will host operations that are of interest to normal applications, while `Reflect` will host operations that are more low-level.

19.8 Conclusion

This concludes our in-depth look at the Proxy API. One thing to be aware of is that Proxies slow down code. That may matter if performance is critical.

On the other hand, performance is often not crucial and it is nice to have the metaprogramming power that Proxies give us.

Acknowledgements:

- Allen Wirfs-Brock pointed out the pitfall explained in §19.3.7 “Pitfall: not all objects can be wrapped transparently by Proxies”.
- The idea for §19.4.3 “Negative Array indices (get)” comes from a [blog post](#) by [Hemanth.HM](#).
- André Jaenisch contributed to the list of libraries that use Proxies.

19.9 Further reading

- [1] “[On the design of the ECMAScript Reflection API](#)” by Tom Van Cutsem and Mark Miller. Technical report, 2012. [Important source of this chapter.]
- [2] “[The Art of the Metaobject Protocol](#)” by Gregor Kiczales, Jim des Rivieres and Daniel G. Bobrow. Book, 1991.
- [3] “[Putting Metaclasses to Work: A New Dimension in Object-Oriented Programming](#)” by Ira R. Forman and Scott H. Danforth. Book, 1999.
- [4] “[Harmony-reflect: Why should I use this library?](#)” by Tom Van Cutsem. [Explains why `Reflect` is useful.]

Chapter 20

The property `.name` of functions (bonus)

Contents

20.1 Names of functions	218
20.1.1 Function names are used in stack traces	218
20.2 Constructs that provide names for functions	219
20.2.1 Function declarations	219
20.2.2 Variable declarations and anonymous functions	219
20.2.3 Assignments and anonymous functions	219
20.2.4 Default values and anonymous functions	219
20.2.5 Named function expressions	220
20.2.6 Methods in object literals	220
20.2.7 Methods in class definitions	221
20.2.8 Names of classes	222
20.2.9 Default exports and anonymous constructs	222
20.2.10 Other programming constructs	222
20.3 Things to look out for with names of functions	223
20.3.1 The name of a function is always assigned at creation	223
20.3.2 Caveat: minification	223
20.4 Changing the names of functions	223
20.5 The function property <code>.name</code> in the ECMAScript specification	224

In this chapter, we look at the names of functions:

- All functions have the property `.name`
- This property is useful for debugging (its value shows up in stack traces) and some metaprogramming tasks (picking a function by name etc.).

20.1 Names of functions

The property `.name` of a function contains the function's name:

```
function myBeautifulFunction() {}
assert.equal(myBeautifulFunction.name, 'myBeautifulFunction');
```

Prior to ECMAScript 6, this property was already supported by most engines. With ES6, it became part of the language standard.

The language has become surprisingly good at finding names for functions, even when they are initially anonymous (e.g., arrow functions).

20.1.1 Function names are used in stack traces

One benefit of function names is that they show up in stack traces. In the following example, the argument of `callFunc()` doesn't get a name, which is why there is no function name in the second line of the stack trace.

```
1  function callFunc(func) {
2    return func();
3  }
4
5  function func1() {
6    throw new Error('Problem');
7  }
8
9  function func2() {
10   // The following arrow function doesn't have a name
11   callFunc(() => func1());
12 }
13
14 function func3() {
15   func2();
16 }
17
18 try {
19   func3();
20 } catch (err) {
21   assert.equal(err.stack, `
22   Error: Problem
23     at func1 (ch_function-names.mjs:6:9)
24     at ch_function-names.mjs:11:18
25     at callFunc (ch_function-names.mjs:2:10)
26     at func2 (ch_function-names.mjs:11:3)
27     at func3 (ch_function-names.mjs:15:3)
28     at ch_function-names.mjs:19:3
29   `.trim());
30 }
31 }
```

20.2 Constructs that provide names for functions

The following sections describe how `.name` is set up automatically by various programming constructs.

20.2.1 Function declarations

Function declarations have had a `.name` for a long time – for example:

```
function funcDecl() {}  
assert.equal(funcDecl.name, 'funcDecl');
```

20.2.2 Variable declarations and anonymous functions

A function created by an anonymous function expression picks up a name if it is the initialization value of a variable declaration:

```
let letFunc = function () {};  
assert.equal(letFunc.name, 'letFunc');  
  
const constFunc = function () {};  
assert.equal(constFunc.name, 'constFunc');  
  
var varFunc = function () {};  
assert.equal(varFunc.name, 'varFunc');
```



Arrow functions also get names

With regard to names, arrow functions are like anonymous function expressions:

```
const arrowFunc = () => {};  
assert.equal(arrowFunc.name, 'arrowFunc');
```

From now on, whenever you see an anonymous function expression, you can assume that an arrow function works the same way.

20.2.3 Assignments and anonymous functions

Even with a normal assignment, `.name` is set up properly:

```
let assignedFunc;  
assignedFunc = function () {};  
assert.equal(assignedFunc.name, 'assignedFunc');
```

20.2.4 Default values and anonymous functions

If a function is a default value, it gets its name from its variable or parameter:

```
const [arrayDestructured = function () {}] = [];  
assert.equal(arrayDestructured.name, 'arrayDestructured');
```

```
const {prop: objectDestructured} = function () {};
assert.equal(objectDestructured.name, 'objectDestructured');

function createParamDefault(paramDefault = function () {}) {
  return paramDefault;
}
assert.equal(createParamDefault().name, 'paramDefault');
```

20.2.5 Named function expressions

The name of a named function expression is used to set up the `.name` property:

```
const varName = function funcName() {};
assert.equal(varName.name, 'funcName');
```

Because it comes first, the function expression's name `funcName` takes precedence over other names (in this case, the name `varName` of the variable). However, the name of a function expression is still only a variable inside the function expression:

```
const varName = function funcName() {
  assert.equal(funcName.name, 'funcName');
};
varName();
assert.throws(
  () => funcName,
  /^ReferenceError: funcName is not defined$/
);
```

20.2.6 Methods in object literals

If a function is the value of a property, it gets its name from that property. It doesn't matter if that happens via a method definition (line A), a traditional property definition (line B), a property definition with a computed property key (line C) or a property value shorthand (line D).

```
function shortHand() {}
const obj = {
  methodDef() {}, // (A)
  propDef: function () {}, // (B)
  ['computed' + 'Key']: function () {}, // (C)
  shortHand, // (D)
};
assert.equal(obj.methodDef.name, 'methodDef');
assert.equal(obj.propDef.name, 'propDef');
assert.equal(obj.computedKey.name, 'computedKey');
assert.equal(obj.shortHand.name, 'shortHand');
```

The names of getters are prefixed with `'get'`, the names of setters are prefixed with `'set'`:

```

const obj = {
  get myGetter() {},
  set mySetter(value) {},
};
const getter = Object.getOwnPropertyDescriptor(obj, 'myGetter').get;
assert.equal(getter.name, 'get myGetter');

const setter = Object.getOwnPropertyDescriptor(obj, 'mySetter').set;
assert.equal(setter.name, 'set mySetter');

```

20.2.7 Methods in class definitions

The naming of methods in class definitions is similar to object literals. For example, here is a class declaration (naming works the same in class expressions):

```

class ClassDecl {
  methodDef() {}
  ['computed' + 'Key']() {} // computed property key

  static staticMethod() {}
}
assert.equal(ClassDecl.prototype.methodDef.name, 'methodDef');
assert.equal(new ClassDecl().methodDef.name, 'methodDef');

assert.equal(ClassDecl.prototype.computedKey.name, 'computedKey');

assert.equal(ClassDecl.staticMethod.name, 'staticMethod');

```

Getters and setters again have the name prefixes 'get' and 'set', respectively:

```

class ClassDecl {
  get myGetter() {}
  set mySetter(value) {}
}
const getter = Object.getOwnPropertyDescriptor(
  ClassDecl.prototype, 'myGetter').get;
assert.equal(getter.name, 'get myGetter');

const setter = Object.getOwnPropertyDescriptor(
  ClassDecl.prototype, 'mySetter').set;
assert.equal(setter.name, 'set mySetter');

```

20.2.7.1 Methods whose keys are symbols

The key of a method can be a symbol. The `.name` property of such a method is still a string:

- If the symbol has a description, the method's name is the description in square brackets.
- Otherwise, the method's name is the empty string ('').

```

const keyWithDesc = Symbol('keyWithDesc');
const keyWithoutDesc = Symbol();

const obj = {
  [keyWithDesc]() {},
  [keyWithoutDesc]() {},
};
assert.equal(obj[keyWithDesc].name, '[keyWithDesc]');
assert.equal(obj[keyWithoutDesc].name, '');

```

20.2.8 Names of classes

Under the hood, classes are functions. Property `.name` of such functions is also set up correctly:

```

class ClassDecl {}
assert.equal(ClassDecl.name, 'ClassDecl');

const AnonClassExpr = class {};
assert.equal(AnonClassExpr.name, 'AnonClassExpr');

const NamedClassExpr = class ClassName {};
assert.equal(NamedClassExpr.name, 'ClassName');

```

20.2.9 Default exports and anonymous constructs

All of the following statements set `.name` to `'default'`:

```

// Anonymous function expressions
export default function () {}
export default (function () {});

// Anonymous class expressions
export default class {}
export default (class {});

// Arrow functions
export default () => {};

```

20.2.10 Other programming constructs

- Generator functions and generator methods get their names the same way that normal functions and methods do. There are no asterisks in their names:

```

function* genFuncDecl() {}
assert.equal(genFuncDecl.name, 'genFuncDecl');

```

- `new Function()` produces functions whose `.name` is `'anonymous'`. [A webkit bug](#) describes why that is necessary on the web.

```

assert.equal(new Function().name, 'anonymous');

```

- `func.bind()` produces a function whose `.name` is `'bound '+func.name`:

```
function func(x) {
  return x
}
const bound = func.bind(undefined, 123);
assert.equal(bound.name, 'bound func');
```

20.3 Things to look out for with names of functions

20.3.1 The name of a function is always assigned at creation

A function only gets a name if one of the previously mentioned patterns is used during its creation. Using one of the patterns later doesn't change anything – for example:

```
function functionFactory() {
  return function () {}; // (A)
}
const func = functionFactory(); // (B)
assert.equal(func.name, ''); // anonymous
```

Function `func` is created in line A, where it also receives its non-name, the empty string `''`. Using an assignment in line B doesn't update the initial name. Missing function names could conceivably be updated later, but doing so would impact performance negatively.

20.3.2 Caveat: minification

Function names are subject to minification, which means that they will usually change in minified code. Depending on what we want to do, we may have to manage function names via strings (which are not minified) or we may have to tell our minifier what names not to minify.

20.4 Changing the names of functions



More information on property attributes

`Object.getOwnPropertyDescriptor()` and `Object.defineProperty()` work with property attributes. For more information on those, see §9 “Property attributes: an introduction”.

These are the attributes of property `.name`:

```
const func = function () {};
assert.deepEqual(
  Object.getOwnPropertyDescriptor(func, 'name'),
  {
    value: 'func',
    writable: false,
    enumerable: false,
```

```
    configurable: true,
  });
```

The property not being writable means that we can't change its value via assignment:

```
assert.throws(
  () => func.name = 'differentName',
  /^TypeError: Cannot assign to read only property 'name'/
);
```

The property is, however, configurable, which means that we can change it by re-defining it:

```
Object.defineProperty(
  func, 'name', {
    value: 'differentName',
  });
assert.equal(func.name, 'differentName');
```

20.5 The function property `.name` in the ECMAScript specification

- The spec operation `SetFunctionName()` sets up the property `.name`. Search for its name in the spec to find out where that happens. The third parameter specifies a name prefix:
 - `Getters and setters` prefix 'get' and 'set'.
 - `Function.prototype.bind()` prefixes 'bound'.
- Anonymous function expressions initially not having a property `.name` can be seen by looking at how functions are created in the spec:
 - The names of named function expressions are set up via `SetFunctionName()`. That operation is not invoked when [creating anonymous function expressions](#).
 - The names of function declarations are set up [elsewhere](#), when entering a scope (early activation).
 - Additionally, the spec [explicitly states](#): “Anonymous functions objects that do not have a contextual name associated with them by this specification do not have a "name" own property but inherit the "name" property of %Function.prototype%.”
- [When an arrow function is created](#), no name is set up, either (`SetFunctionName()` is not invoked).

Most JavaScript engines diverge from the spec and do create own properties `.name` for anonymous functions:

```
> Reflect.ownKeys(() => {})
[ 'length', 'name' ]
```